

Stat 218 Exam 1 Field Guide

FRPs, Kinds and the Big 3+1 in frplib 0.3.10. Drawn from the course text (v3.1.1, mainly chapters 1 to 8), lectures 5 to 13, homework 1 to 6 and the Oct 3 practice set with their posted solutions, and the v0.3.10 cheatsheet.

How to use this. A one-page map of how to approach any problem, with the seven steps and a mind map from the wording to the tool, follows this table (0). Find the wording of the problem in the table below and jump to that section (section numbers are links, and the PDF outline lists every section). Copy the template, swap in the problem's Kinds and statistic, and run it. Check the result with 6.1 before you write it down. If Ctrl-F on your own word finds nothing, try the list of alternate search words in 11. If the problem is a paragraph of words and you do not know where to start, go to 7.1. A timed plan for the practice set, and the knobs to turn when a problem is a variant of one of its nine, are in 4.10. Two routes for a problem that runs until something happens, with the checks that catch a silent slip, are in 4.11.

Every code block here was executed in frplib 0.3.10 while this file was built, except the Fruit Pattern lines in 9.3. Those, and two other slow computations quoted in the text (8.5, 9.5), were run once on the same files and recorded. Homework 4 and 5 run on the course's own hw4.py, hw4_tcboo.py and hw5.py, and homework 2 and 3 on hw2.py and hw3.py. The teal lines that start with # are what the playground prints for the statement above them, minus the box it draws around a Kind or an expectation. Where a Kind is written on one line as value: weight, its tree has been folded to save space. If frp --version on your machine shows less than 0.3.10, 6.4 lists the few lines that will not run there.

Typing and pasting. A block can be pasted as is, or typed line by line: there is no blank line inside a function (in the playground a blank line enters the input) and there is one after it. The playground echoes a result only when an expression is entered on its own, so after pasting a block that defines things, enter the one line whose result you want. If your PDF viewer drops leading spaces when you copy, re-indent the function bodies by hand.

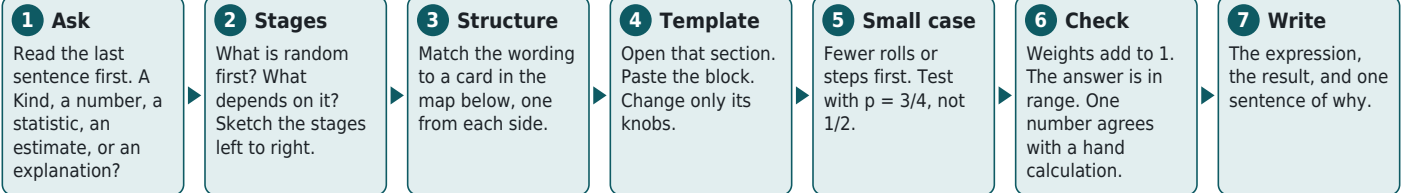
What changed on 7 Oct. Rebuilt on frplib 0.3.10 and checked against the posted solutions for the practice set and for homework 4, 5 and 6, lecture 13, the v0.3.10 cheatsheet and the text v3.1.1. Section 4 now leads with the posted practice answers, and the answer to 7(d) changed (4.7). The third of the Big 3 is now called Updating with Observations, as in the current text (1.2). New: waiting for a pattern (3.18), homework 4 (8), homework 6 (10), the cheatsheet names that fail in frplib 0.3.10 (6.2), and what differs on frplib 0.3.2 (6.4). Renumbered: homework 5 moved from section 8 to section 9, and the search words from section 9 to section 11. Later the same day homework 4 and 5 were rerun on your hw4.py, hw4_tcboo.py and hw5.py in place of stand-ins: every number quoted from those posted solutions was reproduced, and the robot problem turned out to have two readings with different answers (8.3), now quoted from the homework 4 handout. Added last: homework 1 to 3 (12), run on your hw2.py and hw3.py. The posted lines there that do not run on frplib 0.3.10, or do not do what the handout asks, are flagged with a line that works. Added 8 Oct: until problems, three checks, and what each slip looks like when you run it (4.11).

When the problem says	Reach for	Section
roll, flip or draw n times independently	<code>k ** n</code> , then a statistic	3.1
n is large, or the Kind takes forever to compute	<code>fast_join_pow(Sum, k, n)</code> , the monoidal trick	3.2
you observe that, given that, suppose we learn	update with the observation: <code>k (cond)</code> and <code>psi @ k (cond)</code>	3.3
first one thing happens, then depending on it another	a conditional Kind and <code>>></code>	3.4
what does the result tell us about the earlier stage (positive test, which box, which coin)	Bayes: join, update, project. <code>bayes(v, a, t)</code>	3.5
state, at each step, after n steps, a walk, a series of bets, a game	<code>evolve(start, next_state, n)</code>	3.6
how long until, before his sixth search, by the 16th attempt	put a clock in the state and freeze terminal states	3.6
a random number of things, each with a random amount; find the total	<code>fast_join_pow(Sum, k, n)</code> as a conditional Kind, then <code>// kind_N</code>	3.7
the future looks like the beginning; waiting for a pattern of flips	fixed point with <code>iterate</code>	3.8
flip or roll until something happens, with no fixed number of steps	a table of moves, then <code>evolve</code> (or restart patterns and <code>iterate</code>), and three checks	4.11
how long until H T T H T first appears; which of two patterns comes first	state = position along the pattern (lecture 13)	3.18

When the problem says	Reach for	Section
probability of reaching A before B, expected time until, exact answer wanted	first-step equations	3.9
price, expected, predict, does the bet favor you or the casino	$E(\dots)$, or a weighted average by hand	3.10
probability that	$E(k \text{ ^ condition})$	3.10
which value of p fits the data best, maximum likelihood	likelihood = canonical weight on the observed value	3.11
null hypothesis, reject or retain, threshold	Kind of the test statistic when the null is true	3.12
for a general p , in terms of r	<code>symbol('p')</code>	3.13
are X and Y independent	<code>Kind.equal(joint, kx * ky)</code>	3.14
entropy, information, bits	<code>k.entropy</code> , <code>mutual_information</code>	3.15
list, define, describe, true or false, what is the role of	the short-answer bank and its self-test	1.1 to 1.14
give a statistic that	the statistics toolkit	2.2
write a factory for a Kind, a statistic or a condition	<code>@kind_factory</code> and friends	2.8
a dice game that continues until it is won or lost (craps)	state with an outcome flag	3.16
which option is best: where to stand, which die, when to stop, switch or stay	compute the expectation for every option and take the largest	3.17
a long word problem; what are the stages, what is the state	the translation steps and the table of state designs	7.1, 7.2
the Kind is too big to compute; describe its values and weights	reason about which branches survive	7.3
by hand: canonical weights, a Bayes table, a weighted average, an equation for E	worked arithmetic and first-step equations	5.4, 5.6
draw the Kind tree or the wiring diagram	the pictures and how to draw them	5.5
which coin, obscured words, a robot's path, the urn game, immortals (homework 4)	homework 4, with the posted solutions	8.1 to 8.6
supplier and quality, funny dice, fruit, a likelihood from a path (homework 5)	homework 5, with the posted solutions	9.1 to 9.5
kernel workout, two-headed coin, boarding pass, dicey wait, twelve coins (homework 6)	homework 6, with the posted solutions	10.1 to 10.5
a pentagon, Alice and Bob meeting, a rod cut into a triangle, a random forest, a gambler, boxes with prizes, a salesman (homework 1 to 3)	homework 1 to 3, with the posted solutions	12.1 to 12.11
<code>frplib</code> raises an error, or a number looks wrong	checks and fixes	6.1, 6.2
getting ready: what to test with the wifi off, what is allowed	the offline dry run	6.4

0 The map: how to approach any problem

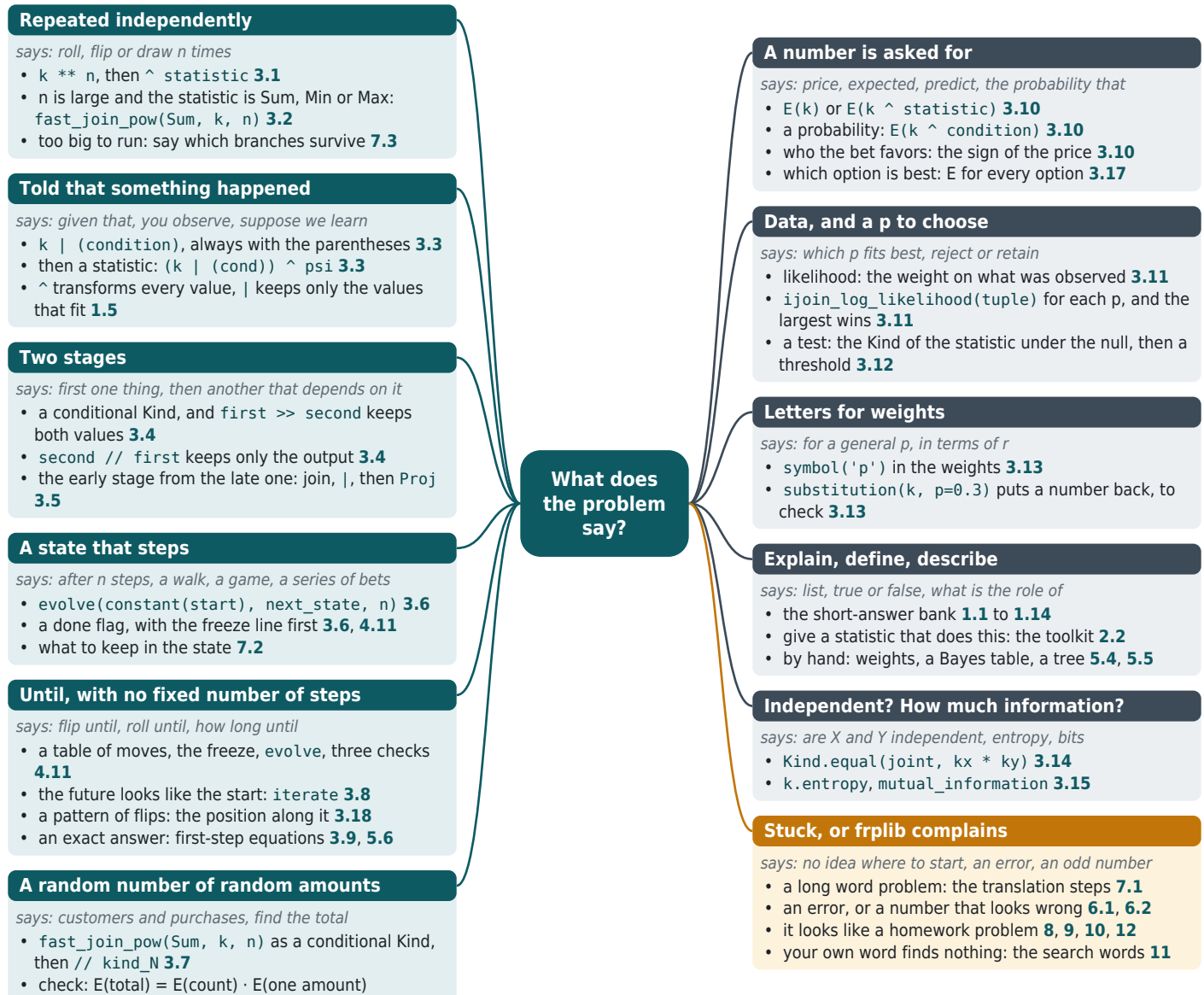
Every problem, the same seven steps



Out of time on a problem: write the expression you would run and one sentence of reasoning, then move on (4.10).

Step 3: from the wording to the tool

Teal cards on the left: how the randomness is built. Grey cards on the right: what is being asked. Most problems take one card from each side. Each line ends with the section to open, and the numbers are links.



Contents

0 The map: how to approach any problem	3
1 Short-answer bank	5
1.1 FRPs and Kinds 5 · 1.2 The Big 3+1 5 · 1.3 Types and vocabulary 6 · 1.4 The modeling framework 6 · 1.5 Transform versus update 7 · 1.6 Joins, conditioning, independence 7 · 1.7 Observations and Bayes's Rule 7 · 1.8 Expectations 8 · 1.9 Kernels and the distribution operator 8 · 1.10 Likelihood and testing 9 · 1.11 State, the Markov property, monoidal statistics 9 · 1.12 Information and entropy 10 · 1.13 Reading notation: holes, arrows, Iverson braces 10 · 1.14 Self-test: short answers to write by hand 11	
2 frplib essentials	13
2.1 Making Kinds 13 · 2.2 Statistics 14 · 2.3 Operators and precedence 16 · 2.4 Looking inside a Kind 16 · 2.5 Conditional Kinds 17 · 2.6 FRPs, clones, samples 18 · 2.7 Procedures: @kind and @frp 18 · 2.8 Factories and helper functions 19 · 2.9 Predict the output: a drill 20	
3 Pattern templates	22
3.1 Independent trials, then ask a question 22 · 3.2 Large powers: the monoidal trick 22 · 3.3 Updating with an observation 23 · 3.4 Two stages: conditional Kinds and joins 24 · 3.5 Bayes's Rule: learn the early stage from the later one 24 · 3.6 Evolving a state 25 · 3.7 A random number of random amounts 26 · 3.8 Fixed points: when the future looks like the beginning 27 · 3.9 First-step equations for exact answers 27 · 3.10 Expectations and probabilities 28 · 3.11 Likelihood and the maximum likelihood estimate 28 · 3.12 Hypothesis test threshold 29 · 3.13 Symbolic weights 31 · 3.14 Checking independence 32 · 3.15 Entropy and mutual information 32 · 3.16 A game that runs until it resolves: craps 32 · 3.17 Decision problems: which option is best 33 · 3.18 Waiting for a pattern: the position along it (lecture 13) 35	
4 The Oct 3 practice set, worked	37
4.1 Problem 1: short answers 37 · 4.2 Problem 2: short answers 38 · 4.3 Problem 3: unbalanced rolls 38 · 4.4 Problem 4: Dr. Jones 39 · 4.5 Problem 5: odd heads run 40 · 4.6 Problem 6: double or nothing 42 · 4.7 Problem 7: customer purchases 42 · 4.8 Problem 8: pricing the payoff 43 · 4.9 Problem 9: asymmetric binary channel 44 · 4.10 The set in 90 minutes, and how to adapt each answer 45 · 4.11 Until problems: two routes, three checks, what each slip looks like 47 · 4.12 A two-state Markov chain: the state only, or the whole path 49	
5 By-hand math	50
5.1 Sums and counting 50 · 5.2 Kinds worth knowing by heart 50 · 5.3 Probability rules, stated as expectations of events 50 · 5.4 Worked by hand 51 · 5.5 Drawing Kind trees and wiring diagrams 52 · 5.6 First-step equations by hand 53	
6 Checks and fixes	54
6.1 Ten-second checks before you write an answer down 54 · 6.2 Error messages and what they mean 54 · 6.3 Finding things during the exam 55 · 6.4 Before the exam: an offline dry run 56	
7 From words to a model	57
7.1 The translation, step by step 57 · 7.2 Choosing the state 59 · 7.3 When the Kind is too big: which branches survive 60 · 7.4 The lecture 11 workshop problems, reduced to their models 61	
8 Homework 4	63
8.1 Problem 1: Which Coin? 63 · 8.2 Problem 2: Obscured Words 63 · 8.3 Problem 3: The Robot's Journey 65 · 8.4 Problem 4: Urn Game 66 · 8.5 Problem 5: Hypothesis Testing 1 68 · 8.6 Problem 6: There Can Be Only One 69	
9 Homework 5	70
9.1 Problem 1: Bayes practice (WidgetCorp) 70 · 9.2 Problem 2: Funny Dice 72 · 9.3 Problem 3: Fruit Pattern 73 · 9.4 Problem 4: Markov likelihood 73 · 9.5 Problem 5: the runs test 74	
10 Homework 6	75
10.1 Problem 1: kernel and distribution workout 75 · 10.2 Problem 2: intuition and evidence 76 · 10.3 Problem 3: the misplaced boarding pass 77 · 10.4 Problem 4: a dicey wait 77 · 10.5 Problem 5: optimal weighings 79	
11 Search words	80
12 Homework 1 to 3	83
12.1 Homework 1: FRPs, Kinds, functions 83 · 12.2 Homework 2, problem 1: statistic workout 84 · 12.3 Homework 2, problem 2: frplib workout 85 · 12.4 Homework 2, problem 3: Meeting Up 85 · 12.5 Homework 2, problem 4: craps at the Lazy C, part 1 86 · 12.6 Homework 2, problem 5: the likelihood principle 87 · 12.7 Homework 3, problem 1: joins workout 87 · 12.8 Homework 3, problem 2: Making Bank 90 · 12.9 Homework 3, problem 3: Find it First! 91 · 12.10 Homework 3, problem 4: craps, part 2 91 · 12.11 Homework 3, problem 5: Wandering Salesman 92	

1 Short-answer bank

Answers in the course's own vocabulary. Tags like (practice 2a) point at the Oct 3 practice question that asks for that item.

1.1 FRPs and Kinds

FRP (Fixed Random Payoff). A device that produces one value, one time. *Random*: before that the value is uncertain. *Fixed*: once produced it never changes. *Payoff*: the value is a tuple of numbers we can treat as a payoff. A *fresh* FRP has not produced its value yet. It is *activated* when the value is produced, which pushing its button does, and *observed* once the value has been shown to us.

Kind. Everything we know about an FRP's value before we see it. A tree with the empty tuple $()$ at the root, a value at every node and a weight on every edge. The leaves are the possible values.

Same Kind, different values. Two FRPs of the same Kind can show different values (unless the Kind has a single leaf). We make the same predictions about both.

The four rules of a valid Kind. Completeness Rule: every path from root to leaf has the same length. **Level Rule:** the nodes on one level hold distinct values of the same dimension. **Prefix Rule:** every node is a strict prefix of each node below it on its path. **Positivity Rule:** all weights are positive.

Dimension, size, width, type. Dimension d is the number of components in each value. Size is the number of leaves (possible values). Width is the number of edges from root to leaf: width 1 is *compact*, width d is *fully unfolded*. The type is $0 \rightarrow d$. A join power $k ** n$ has size $\text{size}(k)^n$ and dimension $n \cdot \text{dim}(k)$.

Canonical form. (1) Order the leaves in increasing lexicographic order. (2) At every non-leaf node, normalize the branch weights to sum to 1. (3) Make the tree compact: one level, each leaf weighted by the product of the normalized weights along its path. Two Kinds are equivalent exactly when they have the same canonical form. Canonical weights are probabilities.

What does not change a Kind. Reordering branches, or multiplying all the weights leaving one node by the same positive constant. Weights 1, 1, 2 and $1/4, 1/4, 1/2$ describe the same Kind.

Unfolding a compact Kind. Group the leaves that share a prefix, put the sum of their weights on the edge to the prefix node, and divide each leaf's weight by that sum.

What the weights mean. In a large demo of FRPs of one Kind, the proportion showing a value approaches that value's canonical weight.

Function. A rule that sends each allowed input to exactly one output. Several inputs may share an output and some outputs may go unused. It is not a function if some input has no output, has two outputs, or the rule never returns for it.

1.2 The Big 3+1

The list (practice 2a). Transforming with Statistics, Building with Joins, Updating with Observations, plus Predicting with Expectations. Lectures 6 to 10 and the text before v3.1 called the third one Constraining with Observations. It is the same operation, and the posted solutions, the current text and the cheatsheet all say Updating.

Operation	On FRPs	On Kinds	We use it to
Transform with a statistic. $X \hat{=} \psi, \psi(X)$	A new FRP whose value is $\psi(v)$ when X 's value is v .	Apply ψ to every leaf value, then combine branches that now have the same value, adding their weights.	Ask a question, extract a feature.
Build with a join. $X \gg C, X * Y, X ** n$	Get X 's value, use it to select the next-stage FRP, output both values joined.	Attach to each leaf u the target Kind for u . Weights multiply along the path.	Build a system in stages. The only operation that adds randomness.
Update with an observation. $X \mid (\text{cond})$	A new FRP that only produces values for which the condition is true.	Erase every branch where the condition is false. Survivors keep their relative weights. Renormalize.	Account for information, real or hypothetical.
+1 Expectation. $E(X)$	The risk-neutral price of each component.	The average of the values weighted by the canonical weights.	Predict.

Consistency. Operating on FRPs and then taking the Kind gives the same result as operating on the Kinds: $\text{kind}(\psi(X)) = \psi(\text{kind}(X))$, $\text{kind}(X \triangleright C) = \text{kind}(X) \triangleright \text{kind}(C)$, $\text{kind}(X \mid \zeta) = \text{kind}(X) \mid \zeta$, $E(X) = E(\text{kind}(X))$. We describe a system with FRPs and analyze it with Kinds, in parallel.

Three interpretations of each of the Big 3. Physical (a wiring diagram: adapter, selector switch, observation processor), algebraic (the operators $\wedge$, $>>$, $|$), and procedural (an FRP or Kind procedure with yield, given, return). The three wiring diagrams are drawn in 5.5.

Transforming an FRP (practice 1e). The new FRP gets the value from the original FRP, applies the statistic, and the result is its value: when X produces v , $\psi(X)$ has value $\psi(v)$. It is activated when the original is activated, and vice versa. Physically, X 's output is wired through an adapter that computes ψ . Its Kind comes from $\text{kind}(X)$ by transforming every leaf value and merging leaves that land on the same value, adding their weights.

Updating a Kind (practice 1f). We erase the branches that are inconsistent with the observation: those whose value makes the condition false.

1.3 Types and vocabulary

Fill in the blanks (practice 2b).

A ...	is a function that maps	to
statistic	values (tuples of numbers)	values
condition	values	a Boolean, written 1 for true and 0 for false
conditional Kind	values	Kinds
conditional FRP	values	FRPs

Codimension and dimension of a statistic. The dimensions of its input and of its output. `typeof(Fork(Count, Min, Mean, Max))` prints `[1..) -> 4`: any input of dimension 1 or more, output of dimension 4.

Type $m \rightarrow n$ of a conditional Kind. It takes inputs of dimension m , and the joined value (`input :: target`) has dimension n , so each target Kind has dimension $n - m$. An ordinary Kind of dimension n is a conditional Kind of type $0 \rightarrow n$, and likewise for FRPs.

Valid conditional Kind (practice 1d). For every input in its domain it must return a Kind, and all the targets must have the same dimension. The function in practice 1d returns the number c when $a = 0$ and a dictionary otherwise, so it is not valid: False.

Compatible. A statistic is compatible with a Kind or FRP when every possible value is a legal input to it.

Event. An FRP whose only possible values are 0 and 1. It *occurs* when its value is 1. $\{X > 2\}$ is the event $X \wedge (_ > 2)$.

Probability. The expectation of an event, which equals the canonical weight on its 1 branch. A number between 0 and 1.

Components. $X = U_1 :: U_2$ means the U 's are projections of X onto consecutive blocks of its components. Scalar components have dimension 1.

Marginal. The Kind of a projection of X , such as $\text{kind}(\text{Proj}[1](X))$: what we know about that component on its own.

Clones. Independent FRPs with the same Kind. `clone(X)` and `frp(k)` each give a fresh one.

1.4 The modeling framework

Data FRP (practice 2e). The FRP that represents all the relevant quantities we collect or observe as we watch a random system evolve. We build it from simpler pieces (Element FRPs) whose structure and interaction are defined in our model, and we transform it in various ways to address our questions (Feature FRPs). Its Kind holds our knowledge and assumptions about the system, and every prediction is computed from it.

Element FRPs. The building blocks (one roll, one flip, one draw). Their Kinds come straight from our assumptions about the system.

Feature FRPs. Derived from the Data FRP with statistics and observations. Each one expresses a question. Predicting its value answers the question.

The pipeline. Element FRPs $\rightarrow$ combine with joins $\rightarrow$ Data FRP $\rightarrow$ derive with statistics and observations $\rightarrow$ Feature FRPs $\rightarrow$ predict with expectations $\rightarrow$ answers.

Model. A collection of possible Kinds for the Data FRP of a random system (text, section 1.5.4). It comes from our assumptions about the Kinds of the Element FRPs and about how they combine and interact. When the Kinds in the collection are labeled one to one by values such as p or (g, s) , those values are the *parameters* of the model.

1.5 Transform versus update

(practice 2f) $k \wedge \phi$ **transforms**: it is the Kind of the FRP produced by passing the value of an FRP with Kind k through the statistic. Because ϕ is a condition, that FRP is an event: the result has dimension 1 with values 0 and 1, and the weight on 1 is the probability that ϕ holds. It asks whether ϕ is true. $k \mid \phi$ **updates** k with the observation that ϕ is true, erasing the branches that are inconsistent with it. The result has the same kind of values as k , but only the branches where ϕ is true, with their weights renormalized. We use it when we have observed that the condition holds, in reality or hypothetically. The link between them: $E(k \wedge \phi)$ is the total canonical weight of the branches that $k \mid \phi$ keeps. Worked numbers are in 4.2.

1.6 Joins, conditioning, independence

Join $X \triangleright C$, written $X \gg C$. The value is $x :: y$, the whole history. Kernel: $(k \triangleright m)(u :: v) = k(u) \cdot m(v \mid u)$. Along a path the normalized weights multiply.

Independent join $X * Y$, written $X * Y$. A join where the second-stage Kind is the same whatever the first stage produced. $\text{kind}(X * Y) = \text{kind}(X) * \text{kind}(Y)$, and the weight of (x, y) is $k_X(x) \cdot k_Y(y)$.

Conditioning, written $m \text{ // } k$. Join, then drop the input: $(k \gg m) \wedge \text{Proj}[(d+1):]$ with $d = \dim(k)$. It averages the target Kinds $m(u)$ using the weights of k , and gives the Kind of the later stage alone.

Independence (practice 1b). For an FRP $X :: Y$, the components X and Y are independent when observing the value of one does not change our knowledge or predictions about the other: $\text{kind}(Y \mid X = x) = \text{kind}(Y)$ for every possible x (checking one direction is enough). Equivalently $\text{kind}(X :: Y) = \text{kind}(X) * \text{kind}(Y)$: we can analyze the whole system by combining separate analyses of the parts.

Quick independence facts. If $Y = \psi(X)$ with ψ not constant, X and Y are dependent. A constant FRP is independent of everything. Statistics of independent FRPs are independent. An FRP built from its own fresh Element FRPs is independent of the rest. Pairwise independent does not imply jointly independent (two flips and their parity).

Independence is an assumption. It is a statement about how the parts of the system are generated, and it is what licenses $*$ and $**$.

Clean join. $X \triangleright C$ is a clean join when X is independent of every target $C(v)$: the new randomness depends on the earlier stages only through the input value handed to the conditional FRP. Clean is the normal case, and it is the condition under which $\text{kind}(X \triangleright C) = \text{kind}(X) \triangleright \text{kind}(C)$. An independent join is clean. Not clean: a conditional FRP whose target is $\psi(X)$ for that same X , because the target then depends on X directly.

Dimension, values and size of a join. For k of dimension d and m of type $d \rightarrow n$, the join $k \triangleright m$ has dimension n . Its values are $u :: v$ for each value u of k and each value v of the target $m(u)$, and its size is the sum of the sizes of the targets $m(u)$ over the values u of k .

Recovering a stage from a join. The early stage is a projection: $k = (k \gg m) \wedge \text{Proj}[: (d+1)]$. The later stage alone is $(k \gg m) \wedge \text{Proj}[(d+1):]$, which is $m \text{ // } k$.

1.7 Observations and Bayes's Rule

Observation. The assertion that a condition is true, because we saw it or because we are asking what if. Real and hypothetical observations are handled identically. The hardware is an *observation processor* (the older text: constraint processor).

Updating a Kind with an observation. Erase the branches where the condition is false. The remaining branches keep their relative weights. Renormalize for canonical form. Kernel: $(k \mid \zeta)(v) = k(v) \zeta(v) / \sum_z k(z) \zeta(z)$. The result is the *updated* Kind (older wording: constrained).

Edge cases. $k \mid \text{top}$ is k itself, because nothing is erased. If no branch satisfies the condition the result is the empty Kind.

A conditional Kind packages updated Kinds. For a join $a \triangleright s$, the target $s(u)$ is the Kind of the later stage given that the early stage equals u : $s(u) = \text{later}(a \triangleright s \mid \text{early} = u)$.

Bayes's Rule. Infer the early stage from an observation of the later stage. Three steps: **join** $a \triangleright t$, **update** with the observed later value, **project** onto the early components. Kernel: $r(u \mid v) = a(u) t(v \mid u) / \sum_{u'} a(u') t(v \mid u')$.

Why a positive test can mean little. When the condition is rare, the false positives from the large healthy group outnumber the true positives from the small sick group.

1.8 Expectations

Arbitrage price. A price c such that paying c per FRP for a large enough batch of that Kind gives a profit with essential certainty.

Risk-neutral price. The smallest value that is larger than every arbitrage price. Risk-neutral because it accounts only for the typical payoff, not for the size of the losses we risk.

Expectation. For a scalar FRP, $E(X)$ is its risk-neutral price. For dimension n it is the tuple of the component prices, so $E(X)$ has the same dimension as X . It depends only on the Kind.

The four rules. **Constancy** $E(c) = c$. **Scaling** $E(sX) = s \cdot E(X)$. **Ordering** if $X \leq Y$ always, then $E(X) \leq E(Y)$. **Additivity** $E(X_1 + \dots + X_n) = E(X_1) + \dots + E(X_n)$, with no independence needed.

Computing it. $E(X) = p_1 v_1 + \dots + p_n v_n$ with canonical weights p . With unnormalized weights w : $(w_1 v_1 + \dots + w_n v_n) / (w_1 + \dots + w_n)$.

Mass-balancing equation. $E(\psi(X)) = \sum_u k_X(u) \psi(u)$. You never have to build the Kind of $\psi(X)$ first.

Affine statistics commute with E. $E(a + bX) = a + b \cdot E(X)$, and $E(\text{Sum}(Y)) = \text{Sum}(E(Y))$ is True. Other statistics do not commute: $E(X^2)$ is not $E(X)^2$ in general. Jensen: if ψ is convex, $E(\psi(X)) \geq \psi(E(X))$.

Variance. $\text{Var}(X) = E(X^2) - E(X)^2$, which is never negative.

Measures of uncertainty. Three ways to quantify how hard a Kind is to predict, each equal to 0 for a constant FRP. Mean absolute deviation: $\text{MAD}(Y) = E(|Y - E(Y)|)$, our prediction of how far Y lands from its expectation. Variance: $\text{Var}(Y) = E((Y - E(Y))^2)$, in squared units; its square root is the standard deviation. Entropy (1.12), which looks only at the weights.

Lecture puzzles. If U takes the values -1 and 1 then U^2 is the constant 1, so $E(U^2) = 1$ for any weights. $E(X | X = 4) = 4$. FRPs with the same Kind have the same expectation, but FRPs with the same expectation need not have the same Kind.

Independent join. $E(X * Y) = E(X) \cdot E(Y)$.

Best prediction. $E(X)$ is the constant c that minimizes the predicted squared error $E|X - c|^2$.

Kinds from expectations. Knowing $E(\psi(X))$ for every statistic ψ determines $\text{kind}(X)$. The indicator statistics $\{x = v\}$ alone recover each weight.

Events as arithmetic. $1 - V$ is *not* V . VW is V and W . $V + W - VW$ is V or W . $V \leq W$ means W occurs whenever V does. $V_1 + \dots + V_n$ counts how many occur.

Updating equation. $E(X | C) \cdot E\{C\} = E(X \cdot \{C\})$, so $E(X | C) = E(X \cdot \{C\}) / E\{C\}$.

Fair and unfair bets. If the expectation of your payoff is negative the bet favors the house. Zero is fair.

Symmetry plus additivity. If $A_1, \dots, A_5$ all have the same Kind and always add up to 48, each has expectation 48/5 (the aces example).

1.9 Kernels and the distribution operator

Kernel. The function that packages a Kind: $k(v)$ is the canonical weight on v , and 0 if v is not a possible value. For a conditional Kind m , $m(v | u)$ is the kernel of the target $m(u)$ at v . In `frplib`: `k.kernel(v)`.

Big 3+1 in kernel form	Formula
Transform, $k' = \psi(k)$	$k'(v) = \sum_u k(u) \cdot \{v = \psi(u)\}$
Join	$(k \triangleright m)(u :: v) = k(u) \cdot m(v u)$
Update	$(k \zeta)(v) = k(v) \zeta(v) / \sum_z k(z) \zeta(z)$
Bayes	$r(u v) = a(u) t(v u) / \sum_{u'} a(u') t(v u')$
Expectation	$E(\psi(k)) = \sum_u k(u) \psi(u)$

Distribution operator. $D_X(\psi) = E(\psi(X))$. You hand it a question and it returns the predicted answer. $D_X = D_Y$ exactly when $\text{kind}(X) = \text{kind}(Y)$. In `frplib`: `D_(X)(psi)`.

Reading them off the page. A kernel can be written with Iverson braces: uniform on 1 to 99 is $k(n) = (1/99) \cdot \{n \in [1 \dots 99]\}$. A distribution operator written out, $D_X(\psi) = (1/16) \psi(-2) + (1/8) \psi(-1) + \dots$, lists the canonical weights (the coefficients) and the values (the arguments of ψ). To get $E(\psi(X))$, plug the statistic in and add up. Homework 6 problem 1 does both (10.1).

1.10 Likelihood and testing

Probabilistic inference. The system is fully specified and we predict the outcomes of running it. One Kind, many possible values not yet observed. The canonical weight on a value is its probability.

Statistical inference. We do not know which Kind in a family describes the system, but we have observed data, and we use the data to learn about the parameter. Many candidate Kinds, one observed value. The same canonical weight, now read as a score for each candidate Kind, is called its likelihood.

Family of Kinds (the model). Several candidate Kinds for the Data FRP, indexed by a parameter θ .

Likelihood function. For observed data x and a model K , $L_x(k) = k(x)$ for each Kind k in K : the canonical weight that k puts on the value we actually observed, and 0 if x is impossible under k . With a parameter θ labeling the Kinds we write $L(\theta) = k_\theta(\text{observed})$. The log-likelihood is $\ell_x(k) = \log L_x(k)$.

Kernel versus likelihood. The same number $k(x)$ read two ways. Kernel: the Kind is fixed and the value varies (the forward, probability direction, from a model to predictions about data). Likelihood: the observed value is fixed and the Kind varies (the backward, statistics direction, from data to the nature of the system).

Likelihood principle. All the information the data carry about the parameter is contained in the likelihood.

Maximum likelihood estimate. The θ that maximizes $L(\theta)$, or equivalently $\log L(\theta)$. It is the model under which what we saw is least surprising. A heuristic that works well in practice, not a guarantee. The text's example: ten flips showing six heads have $L(p) = p^6(1-p)^4$, which is largest at $p = 0.6$.

Independent join. If the data are $\langle x_1, \dots, x_n \rangle$ from $k \text{ ** } n$, then $L = k(x_1) \cdot k(x_2) \cdots k(x_n)$, and the log-likelihood is a sum.

General join. L is the product of the normalized weights along the observed path: $k(v_0) \cdot m_1(v_1 | v_0) \cdot m_2(v_2 | v_0 :: v_1) \cdots$

More data. More independent observations make the comparison between candidate models sharper, because more data reduces the sampling variability.

Hypothesis test. Null hypothesis: the default model. **Alternative:** the model that needs evidence. **Test statistic** ψ : larger values count as evidence against the null (for two single models, the likelihood ratio $L_{\text{alt}} / L_{\text{null}}$). **Threshold** t : chosen so that when the null is true, $E\{\psi(B) > t\} \leq \alpha$. Reject the null if the observed ψ exceeds t , otherwise retain it. Retaining does not show that the null is true.

Type I error rate. The α in that rule: the proportion of data sets generated under the null for which the test rejects it anyway. There are two ways to be wrong, rejecting a null that is true and retaining a null that is false, and the threshold trades one against the other. The procedure is not symmetric: it takes strong evidence to reject the null, and anything less retains it (homework 4, 8.5).

1.11 State, the Markov property, monoidal statistics

State. Information that fully, and as concisely as possible, describes the system's configuration at a moment: enough to say what can happen next and to answer the question asked. There is usually more than one good choice.

Markov property. The next state depends only on the current state, not on how the system got there. Given the present, the future and the past are independent. That is why `state = next_state // state` works.

Augmenting the state. Add a step counter, a visited flag, or a frozen terminal value to answer questions about the history.

Position along a pattern (lecture 13). For "how long until a given pattern of flips first appears" the minimal state is the position along the pattern: how many of its leading symbols are matched right now. The transitions and the resulting first-step equations are in 3.18.

Monoidal statistic. One that can be computed in pieces and the pieces combined. For Sum, Min and Max the rule is $\psi(a :: b) = \psi(\psi(a) :: \psi(b))$, so $\psi(k \text{ ** } n)$ can be built from ψ of smaller powers without ever forming $k \text{ ** } n$ (Alice and Bob's trick). The text also names Count, and a Fork of monoidal statistics, as monoidal. Mean and range are not monoidal, but they follow from Sum and from Fork(Min, Max). What `fast_join_pow` really handles is in 3.2.

Equilibrium. For many systems the Kind of the state stops changing after enough steps, whatever the starting state (the long-run or stationary Kind). Find it by evolving for many steps.

Three ways to answer long-run questions. Solve an exact equation for the answer, compute an approximation and make its error small (evolve the Kind until the unresolved weight is negligible), or simulate the system with FRPs and tabulate.

Dirichlet problem. The general first-step equation (3.9). The states split into boundary sets S_i where f is a known constant c_i , and the rest where $f(s) = \alpha + \beta \cdot E(f(k_s))$, with k_s the Kind of the next state from s . Probability of hitting one boundary set before another: boundary values 1 and 0, $\alpha = 0$, $\beta = 1$. Expected time to reach the boundary: boundary value 0, $\alpha = 1$, $\beta = 1$.

Markov decryption, the idea. To break a substitution cipher, score a candidate decoding d by $L(d)$, the product of the English two-letter (bigram) frequencies along the decoded text. Run a random system on decodings: propose d' by swapping

two letters, move to d' with probability $\min(1, L(d') / L(d))$, otherwise stay, and keep the best-scoring d seen so far. The system climbs toward high scores, and sometimes steps down at random, without trying every possible substitution.

1.12 Information and entropy

Information. Seeing the value v gives $-\lg k(v) = \lg(1 / k(v))$ bits. Rare values surprise more, and independent observations add.

Entropy. $H(Y) = E(-\lg k_Y(Y)) = -\sum p \cdot \lg p$. The average information gained by observing Y . It depends only on the weights, not on the values.

What it measures. The average number of yes or no questions needed to guess the value; the average bits needed to store a value; the minimum random bits needed to generate a value; the minimum bits per value to transmit many clones. No encoding does better than H .

Properties. $H \geq 0$. $H = 0$ only for a constant. Uniform on n values gives $\lg n$, the largest entropy any Kind on those n values can have. $H(X * Y) = H(X) + H(Y)$. For an event with probability p : $h(p) = -p \cdot \lg p - (1 - p) \cdot \lg(1 - p)$. If ψ is one to one on the values of Y then $H(\psi(Y)) = H(Y)$; in general a statistic can only lose entropy.

Conditional entropy. Pointwise: $H(Y | X = x)$ is the entropy of the target Kind at x , a statistic of x . Average: $H(Y | X)$ is the expectation of that statistic over X , a number. "Conditional entropy" alone means the average one.

Chain rule and mutual information. $H(X, Y) = H(X) + H(Y | X)$. $I(Y; X) = H(Y) - H(Y | X) = H(X) + H(Y) - H(X, Y)$. It is never negative, it is symmetric, it is 0 exactly when X and Y are independent, and it is at most $\min(H(X), H(Y))$. When $Y = \psi(X)$ is a function of X , $H(Y | X) = 0$ and so $I(X; \psi(X)) = H(\psi(X))$ (homework 6, 10.5).

KL divergence. $KL(k_1; k_2) = \sum k_1(u) \cdot \lg(k_1(u) / k_2(u))$: the cost in bits of using k_2 as the model when k_1 is correct. Never negative and 0 only when the two Kinds are equivalent, not symmetric, infinite if k_2 gives weight 0 to a value k_1 can produce. Mutual information is a divergence: $I(X; Y) = KL(\text{kind}(X :: Y); \text{kind}(X) * \text{kind}(Y))$, the cost of treating the components as independent.

Codes (text, section 47.1). A prefix (instantaneous) code is one in which no codeword is the beginning of another, so a stream can be decoded one codeword at a time. Its codeword lengths satisfy the Kraft inequality $\sum 2^{-\ell} \leq 1$. Lengths $\ell = \lceil -\lg k(s) \rceil$ give an expected length between H and $H + 1$.

Counting outcomes. An observation with m possible outcomes gives at most $\lg m$ bits. Telling 24 possibilities apart takes $\lg 24 = 4.585$ bits, and a balance with three outcomes gives at most $\lg 3 = 1.585$ bits a weighing, so two weighings ($3^2 = 9$ outcomes) cannot do it and three (27) might (10.5).

Entropy as a limit on communication. To send the values of many clones of X in binary, the amortized number of bits per value can never be below $H(X)$, and good encoders get arbitrarily close. For a bit that is 1 with probability 0.5, 0.9, 0.99 that is 1, 0.469 and 0.081 bits per bit.

Golomb coding (lecture 10). A way to send n bits that are mostly 1s ($p > 1/2$) by sending the lengths of the runs of 1s instead. Each run ends at a 0, and two 0s in a row give a run of length 0. Choose $m = \lceil -1 / \lg p \rceil$ and $b = \lceil \lg m \rceil$. Write a run length as $\ell = qm + r$ with $0 \leq r < m$. Send q in unary (q ones, then a zero), then r : if $r < 2^{b+1} - m$ send r as a b -bit number, otherwise send $r + 2^{b+1} - m$ as a $(b + 1)$ -bit number. The message starts with m itself (a 3-bit number j , then m in 2^j bits, least significant bit first) and ends with the last raw bit z , which tells the decoder whether the final run really ended in a 0.

Golomb example. $p = 0.9$ gives $m = 7$, $b = 2$ and cutoff $2^3 - 7 = 1$. A run of $17 = 2 \cdot 7 + 3$ has $q = 2$ and $r = 3$, so it is sent as 110 followed by $3 + 1 = 4$ in three bits, 100: the six bits 110100. A run of 0 is sent as 0 followed by 00. With `lecture10.py` in the working directory, from `lecture10 import *` gives `bits(n, p)`, `RunLengths`, `golomb_encode(p)`, `golomb_decode` and `to_golomb(run, m, b)`.

1.13 Reading notation: holes, arrows, Iverson braces

Hole (practice 2d). $(10 \square^2 - 2)$ is the function that puts its argument in the hole. At 1 it gives $10 \cdot 1 - 2 = 8$. In `frplib` the hole is two underscores: `10 * __ ** 2 - 2`.

Arrow. $\langle y \rangle \mapsto \sin(\pi y/6)$ at 1 is $\sin(\pi/6) = 1/2$.

Iverson braces. $\{\text{condition}\}$ is 1 when the condition is true and 0 when it is false. So $f(x) = 3 \cdot \{-10 < x < 2\} + 2 \cdot \{x > 0\} - 10 \cdot \{x < 1\} - \{x = 4\}$ at 1 is $3 + 2 - 0 - 0 = 5$.

Inlined statistics. X^2 , $2X - 1$ and $\{X > 2\}$ mean the transforms of X by those functions.

Trig values. $\sin(\pi/6) = 1/2$, $\sin(\pi/4) \approx 0.7071$, $\sin(\pi/3) \approx 0.8660$, $\sin(\pi/2) = 1$. Cosine runs the other way: $\cos(\pi/3) = 1/2$.

1.14 Self-test: short answers to write by hand

Prompts in the style of practice problems 1 and 2, each with an answer that says enough in a sentence or two. Cover the right-hand column, write your own answer on paper, then compare. The definitions behind them are in **1.1** to **1.13**.

Prompt	An answer that says enough
How does an FRP differ from its Kind?	The FRP is the device, the random quantity itself: it produces one value, once. The Kind is what we know about that value before we see it: the possible values and their weights. Many FRPs can share one Kind.
Two FRPs have the same Kind. Do they show the same value?	Not necessarily. The same Kind means we make the same predictions about both. Each produces its own value, and the values can differ unless the Kind is a constant.
What do the weights mean? Must they add to 1?	Only their ratios matter. In a large demo, the share of FRPs showing a value approaches that value's share of the total weight. They need not add to 1. Canonical form rescales them so that they do.
Why put a Kind in canonical form?	Two Kinds are the same exactly when their canonical forms match, and canonical weights are probabilities that can be read off and averaged directly.
Which of the Big 3 can add branches to a Kind?	Only a join, because it is the only operation that adds randomness. A transform keeps or merges branches. An observation keeps or erases them.
When does $k \wedge \psi$ have fewer branches than k ?	When ψ sends two or more values of k to the same output. Those branches merge and their weights add.
$X * Y$ versus $X \gg C$	Both are joins, and both keep every stage's value. In $X * Y$ the second stage has the same Kind whatever the first produced. In $X \gg C$ the first value selects which FRP runs second.
$k \gg m$ versus $m // k$	$k \gg m$ keeps the whole history, input then target. $m // k$ drops the input and leaves the later stage alone: the targets $m(u)$ averaged with the weights of k .
What does an observation do to a Kind, and what does it never do?	It erases the branches where the condition is false, and we renormalize. It never adds a branch and never changes the relative weights of the branches that remain.
Is a hypothetical observation handled differently from a real one?	No. "Suppose we learned that" and "we saw that" are the same operation.
$k \wedge \phi$ versus $k \phi$ for a condition ϕ (practice 2f)	Transforming asks whether ϕ is true: the result is the Kind of an event, on 0 and 1, whose weight on 1 is the probability of ϕ . Updating takes ϕ as observed, in reality or hypothetically: the result is k 's own values, restricted to where ϕ holds, renormalized.
What does it mean for X and Y to be independent (practice 1b)? How would you check?	Observing the value of one does not change the Kind of the other. Equivalently $\text{kind}(X :: Y) = \text{kind}(X) * \text{kind}(Y)$, which is the check: <code>Kind.equal(joint, kx * ky)</code> .
Can X and $\psi(X)$ be independent?	Only if $\psi(X)$ is a constant. Otherwise observing X tells us $\psi(X)$ exactly.
What is the role of the Data FRP (practice 2e)?	It represents all the relevant quantities we collect or observe as the system evolves. It is built from simpler Element FRPs, whose structure and interaction our model defines, and we transform it into Feature FRPs to address our questions.
When we transform an FRP with a statistic, when is the new FRP activated (practice 1e)?	When the original is, and vice versa. The new FRP takes the original's value, applies the statistic, and the result is its value.
What makes a conditional Kind valid (practice 1d)?	It returns a Kind for every input in its domain, and all the targets have the same dimension. Returning a number, a tuple or a dictionary for some input makes it invalid.
A conditional Kind takes inputs of dimension 2 and its targets have dimension 1. Its type?	$2 \rightarrow 3$. The joined value, input followed by target, has dimension 3.
State Bayes's Rule with the Big 3.	Join the early stage with the conditional Kind of the later stage, update with the observed later value, project onto the early components.
What is a probability in this framework?	The expectation of an event (an FRP whose values are 0 and 1). It equals the canonical weight on 1.
What is an expectation, in words?	The risk-neutral price of the FRP, our best prediction of its value. It is computed as the average of the values weighted by the canonical weights.

Prompt	An answer that says enough
Which rules for E need independence?	None of the four. $E(X + Y) = E(X) + E(Y)$ always holds. Independence is what gives $E(XY) = E(X) \cdot E(Y)$ and $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$.
Probability versus likelihood (kernel versus likelihood)	The same number $k_\theta(v)$ read two ways. With the Kind fixed it is the probability of the value v . With the observed v fixed it is a score for each candidate Kind.
What does retaining the null hypothesis mean?	The data did not give enough evidence to reject it. It does not show the null is true.
What is the state of a system? What is the Markov property?	The state is the information needed to say what can happen next and to answer the question, and no more. Markov: given the current state, the next state does not depend on how the system got there, so <code>next_state // state</code> is all we need.
What is a monoidal statistic, and why do we care?	One that can be computed in pieces and the pieces combined: $\text{Sum}(a :: b) = \text{Sum}(\text{Sum}(a) :: \text{Sum}(b))$, and the same for Min and Max (the text names Count as well). It gives $\psi(k ** n)$ from smaller powers without building $k ** n$.
What does entropy measure?	The average information, in bits, gained by observing the value: $H = -\sum p \cdot \lg p$. It depends only on the weights, is 0 for a constant and is largest, $\lg n$, for uniform on n values.
In the pattern-waiting examples, what is the fixed point?	The Kind k of the remaining wait is unchanged by "take a step, then either finish or start over": $k = F(k)$. We find it by iterating F , or solve the matching equation for $E(k)$ (5.6). For a general pattern the state is the position along it (3.18).

True or false	Verdict and the reason to give
Transforming a Kind with a statistic can increase its size.	False. Each value has exactly one output, so branches can merge but never split.
Updating with an observation changes the relative weights of the branches that remain.	False. Survivors keep their relative weights. Only the normalization changes.
$E(X^2) = E(X)^2$.	False in general. They differ by $\text{Var}(X)$, so they agree only for a constant.
$E(X + Y) = E(X) + E(Y)$ only when X and Y are independent.	False. Additivity holds for any X and Y .
Every branch of $k ** n$ has the same weight.	Only when k is uniform. In general a branch's weight is the product of the base weights of its components.
$k \mid \phi$ has the same dimension as k , and $k \wedge \phi$ has dimension 1.	True (ϕ a condition). The first keeps values of k . The second has values 0 and 1.
$k \gg m$ and $m // k$ have the same size.	False in general. <code>//</code> drops the input, so equal later values merge.
Pairwise independent FRPs are jointly independent.	False. Two fair flips and their parity are pairwise independent, but any two of them determine the third.
The likelihood of a model is the probability that the model is true.	False. It is the weight that model gives to the data we observed.
If $E(X) = E(Y)$ then X and Y have the same Kind.	False. Same Kind implies same expectation, not the other way round.
An observation about one component of an independent join changes the Kind of the other components.	False. That is what independence means: $(k \mid c) * j$ is $k * j$ given c on the first part.
Evolving a system for more steps always changes the Kind of the state.	False. Frozen terminal states stay put, and a system in equilibrium has stopped changing.

2 frplib essentials

In the playground everything below is already loaded. In a script or a plain Python session, start with `from frplib.playground import *`.

2.1 Making Kinds

Factory	What you get
<code>constant(3), constant(1, 2)</code>	One value with weight 1. Use it for a sure outcome inside a conditional Kind.
<code>choice(a, b), choice(a, b, r)</code>	Two values with weight 1 on a and r on b, so b is r times as likely as a. <code>choice(0, 1, 19)</code> puts 0.95 on 1 and <code>choice(0, 1, 1/999)</code> puts 0.001 on 1.
<code>either(a, b, r)</code>	The older, deprecated spelling, still used in <code>frplib.examples</code> and in copies of the text before v3.1, with the ratio the other way round: a is r times as likely as b. <code>either(0, 1, 2)</code> is <code>choice(0, 1, 1/2)</code> .
<code>binary(p)</code>	Values 0 and 1 with weight p on 1. <code>binary()</code> is a fair coin.
<code>uniform(1, 2, ..., 6)</code>	Equal weights. Also takes a list, set or generator: <code>uniform((i, j) for i in irange(1, 3) for j in irange(1, 3))</code> .
<code>weighted_as(1, 2, 3, weights=[5, 3, 2])</code>	The given weights (any positive numbers). Also <code>weighted_as({value: weight, ...})</code> and <code>weighted_as(list_of_values, weights=[...])</code> .
<code>weighted_by(1, 2, ..., 6, weight_by=f)</code>	Weight <code>f(v)</code> on each value <code>v</code> .
<code>weighted_pairs([(v1, w1), (v2, w2)])</code>	A list of (value, weight) pairs.
<code>geometric(0, 1, ..., 25, r=0.9)</code>	Weights 1, r, r ² , and so on down the list.
<code>linear(1, 2, ..., 6, first=1, increment=2)</code>	Weights 1, 3, 5, and so on.
<code>symmetric(1, 2, ..., 6, around=3.5)</code>	Weights symmetric around a center.
<code>without_replacement(2, [1, 2, 3, 4])</code>	Every 2-element subset as a sorted tuple, equal weights.
<code>ordered_samples(2, [1, 2, 3])</code>	Ordered draws without replacement, equal weights. (The v0.3.10 cheatsheet prints <code>ordered_sample</code> , which frplib 0.3.10 does not define.)
<code>permutations_of([1, 2, 3])</code>	All orderings, equal weights. It takes one collection.
<code>integers(1, 6), evenly_spaced(0, 1, num=5)</code>	Equal weights on 1, 2, 3, 4, 5 (the stop value is left out; <code>integers(0, 10, 2)</code> steps by 2), and on five evenly spaced values from 0 to 1.
<code>arbitrary(1, 2, 3, names=['a', 'b', 'c'])</code>	Symbolic weights a, b, c (see 3.13).
<code>k1 * k2, k ** n, independent_join([k1, k2, k3])</code>	Independent joins.
<code>Kind.empty</code>	The Kind with no components. It is the identity for <code>*</code> , so it is the right starting value when you build an independent join in a loop.

- `a, b, ..., c` continues the step from a to b up to and including c. The three dots are typed literally.
- Write exact fractions as strings, `'1/3'`. A float such as `1/3` works but may print as a decimal.
- Values can be numbers or tuples. `irange(1, 6)` is the inclusive range 1 to 6, and `irange(1, 52, exclude={7})` skips values.

2.2 Statistics

A statistic takes a value (a tuple) and returns a value. Built-ins applied to (3, 1, 4, 1, 5, 9, 2, 6):

Statistic	Result	Statistic	Result
Sum	31	Proj[2]	1 (components count from 1)
Count	8 (number of components)	Proj[1, 3]	(3, 4)
Min, Max	1 and 9	Proj[2:4]	(1, 4) (components 2 and 3)
Mean	3.875	Proj[3:], Proj[:3]	from 3 to the end; components 1 and 2
SumSq	173	Proj[-1]	6 (the last component)
Diff	(-2, 3, -3, 4, 4, -7, 4)	ArgMax, ArgMin	5 and 1 (positions counted from 0)
Ascending	(1, 1, 2, 3, 4, 5, 6, 9)	Distinct	0 (not all different)

Building new statistics from old ones

Tool	Examples
The hole <code>__</code> (two underscores)	<code>__ + 1, __ ** 2, __ % 2 == 0, 10 * __ ** 2 - 2</code>
Arithmetic and comparisons	<code>Proj[1] + 2 * Proj[2], Sum > 10, Max - Min, Abs(Proj[1] - Proj[2]) <= 5</code>
Composition	<code>Sqrt(Max)</code> or <code>Max ^ Sqrt</code> (left to right: Max, then Sqrt)
<code>Fork(s1, s2, ...)</code>	All results in one tuple: <code>Fork(Min, Max)</code> . Numbers become constants, so <code>Fork(__, 1)</code> appends a 1 and <code>Fork(__ + x, n + 1)</code> builds a next state.
<code>ForEach(s)</code>	Apply <code>s</code> to each component: <code>ForEach(__ == 4)</code> gives 0/1 flags, and <code>ForEach(__ == 4) ^ Sum</code> counts the 4s.
<code>IfThenElse(cond, a, b)</code>	<code>IfThenElse(__ == 0, -1, __)</code> : -1 if the input is 0, else the input.
<code>Cases({...}, default=d)</code>	A lookup table: <code>Cases({2: -1, 3: -1, 7: 1, 11: 1}, default=0)</code> .
<code>And, Or, Not, Xor</code>	<code>And(Proj[1] == 1, Proj[2] == 1)</code> . Any number of conditions.
<code>All(c), Any(c)</code>	Is <code>c</code> true for every component, or for some component: <code>All(__ > 0)</code> .
<code>ElementOf(2, 3, 12)</code>	Is the input one of these values. <code>Between(2, 5)</code> tests $2 \leq v < 5$.
<code>Append(v), Prepend(v)</code>	Add a component at the end or at the front.
<code>Permute(2, 1)</code>	Swap or reorder components.

More built-ins, most of them listed on the v0.3.10 cheatsheet, each entered on its own. Every result below was produced by frplib 0.3.10.

You type	It prints	You type	It prints
<code>Product(3, 1, 4)</code>	<12>	<code>Dim(3, 1, 4)</code>	<3>
<code>Descending(3, 1, 4)</code>	<4, 3, 1>	<code>Negate(3, -1)</code>	<-3, 1>
<code>Median(3, 1, 4, 1, 5)</code>	<3>	<code>Quartiles(3, 1, 4, 1, 5, 9, 2, 6)</code>	<2, 3.5, 5>
<code>IQR(3, 1, 4, 1, 5, 9, 2, 6)</code>	<3>	<code>Variance(3, 1, 4, 1, 5)</code>	<3.20>
<code>Round(2)(3.14159)</code>	<3.14>	<code>Dot(1, 2, 3)(4, 5, 6)</code>	<32>
<code>Diffs(2)(1, 4, 9, 16)</code>	<2, 2>	<code>Shift(1)(1, 2, 3, 4)</code>	<4, 1, 2, 3>
<code>IndexOf(4)(3, 1, 4, 1, 5)</code>	<2>	<code>Get([10, 20, 30])(1)</code>	<20>
<code>Constantly(7)(3, 1, 4)</code>	<7>	<code>Id(3, 1, 4)</code>	<3, 1, 4>
<code>Floor(2.7)</code>	<2>	<code>Ceil(2.1)</code>	<3>
<code>Binomial(5, 2)</code>	<10>	<code>Sqrt(16)</code>	<4>
<code>Chain(Sum, __ * 2)(1, 2, 3)</code>	<12>	<code>Compose(__ * 2, Sum)(1, 2, 3)</code>	<12>

You type	It prints	You type	It prints
ForEach(Sum, by=2)(1, 2, 3, 4)	<3, 7>	Between(2, 5)(5)	<0>
Freqs(3, 1, 4, 1, 5)	<2, 1, 1, 1, 5>	Keep(__ > 2)(3, 1, 4, 1, 5)	<3, 4, 5, 5, 5>

- `Id` is another name for the hole `__`. `Chain` applies its statistics left to right and `Compose` right to left. `Freqs` and `Keep` pad their result with empty slots, printed as `□`, to keep the dimension fixed.
- `Variance` and `StdDev` summarize the components of one value and divide by $n - 1$. `Var(k)` of a `Kind` is $E(X^2) - E(X)^2$.
- Also there, working as their names say: `Exp`, `Log` (natural), `Log2`, `Log10`, `Sin`, `Cos`, `Tan`, `NormalCDF`, `Gamma`, `ArgMin`, `ArgMax`, `Bag`, `Xor`, `top` (always true), `bottom` (always false).

Your own statistics

```
@scalar_statistic
def wins_before_loss(bets):          # one parameter: it receives the whole tuple
    n = 0
    for b in bets:
        if b == 0:
            break
        n += 1
    return n

@statistic(codim=2, dim=2)           # two parameters: one per component
def rotate90(x, y):
    return (-y, x)

@condition                           # returns 1 (true) or 0 (false)
def both_even(x, y):
    return x % 2 == 0 and y % 2 == 0

wins_before_loss(1, 1, 0, 1)
# <2>
rotate90(3, 4)
# <-4, 3>
both_even(2, 6)
# <1>
typeof(Fork(Count, Min, Mean, Max))
# '[1..) -> 4'
```

WATCH OUT `Proj` counts from 1. Python indexing, `ArgMax`, `IndexOf` and `index_of` count from 0. `Proj[2:4]` is components 2 and 3: the end is left out.

`Abs` of a tuple with several components is its Euclidean length. Apply it to a scalar expression, as in `Abs(Proj[1] - Proj[2])`. `Keep(cond)` pads the result back to the original dimension, so `Keep(cond) ^ Count` does not count matches. Count with `ForEach(cond) ^ Sum`.

`Contains(4, 5)` looks for a 4 immediately followed by a 5. For membership in a set of values use `ElementOf`.

There is no unary minus for statistics: `-__` and `Exp(-__)` fail with `bad operand type for unary -`. Write `-1 * __`, `0 - __` or `Negate`.

Four names on the v0.3.10 cheatsheet are not defined in `frplib 0.3.10`: `Ceiling` (use `Ceil`), `StDev` (use `StdDev`), `ordered_sample` (use `ordered_samples`) and `guard` (use `given`, **2.7**).

2.3 Operators and precedence

Write	Meaning
$k \wedge \text{psi}$ or $\text{psi}(k)$	Transform k with the statistic psi .
$k1 * k2$	Independent join.
$k ** n$	Independent join of n copies.
$k \gg m$	Join with the conditional Kind m . Values are k 's value followed by the target's.
$m // k$	Conditioning: $k \gg m$ with k 's components dropped. The later stage alone.
$k \mid (\text{cond})$	Update k with the observation that cond is true. Read $\mid$ as "given".
$\text{psi} @ k \mid (\text{cond})$	Transform that remembers k , so the condition can refer to k 's own values.

Python decides the grouping. From tightest to loosest:

```
[ ] ( ) then ** then * @ // / % then + - then >> then ^ then | then == < <= > >=
```

You type	Python reads	Verdict
$d6 ** 3 \wedge \text{Sum}$	$(d6 ** 3) \wedge \text{Sum}$	Fine.
$a \gg m \wedge \text{Proj}[2]$	$(a \gg m) \wedge \text{Proj}[2]$	Fine.
$m // k \wedge \text{psi}$	$(m // k) \wedge \text{psi}$	Fine.
$k \wedge \text{psi1} \wedge \text{psi2}$	left to right	Fine.
$k \mid \text{Proj}[2] == 1$	$(k \mid \text{Proj}[2]) == 1$	Wrong. Always wrap the condition: $k \mid (\text{Proj}[2] == 1)$.
$k \wedge \text{Sum} > 10$	$(k \wedge \text{Sum}) > 10$	Wrong. Write $k \wedge (\text{Sum} > 10)$.
$k \mid (\text{cond}) \wedge \text{psi}$	$k \mid ((\text{cond}) \wedge \text{psi})$	Wrong, and silently: psi is applied to the condition, not to the updated Kind. Write $(k \mid (\text{cond})) \wedge \text{psi}$ or $\text{psi} @ k \mid (\text{cond})$.
$(A + B ** 2) \% 3 == 0$	power, then +, then %, then ==	Fine with the parentheses. Without them % would bind before +.

WATCH OUT For an FRP X , $X * X$ joins X with itself, so both components show the same value. For independent copies write $X * \text{clone}(X)$ or $X ** 2$.

2.4 Looking inside a Kind

```
k = weighted_as(1, 2, 3, weights=[1, 2, 3])
k
#      ,---- 1/6 ---- 1
# <> -+---- 2/6 ---- 2
#      `---- 3/6 ---- 3
k.kernel(2), k.kernel(7)
# (0.3333333333333333, 0.0)
size(k), dim(k), typeof(k), values(k)
# (3, 1, '0 -> 1', {1, 2, 3})
E(k)
# 7/3
Var(k)
# 5/9
E(k ^ (__ >= 2))
# 5/6
as_float(E(k)), E(k) <= 2.5
# (2.3333333333333335, True)
Kind.equal(k, linear(1, 2, 3))
# True
```

```

show([E(k), Var(k)])          # a list of results, tidily
# [
#   7/3
#   5/9
# ]
for value, weight in k:       # a Kind can be looped over, branch by branch
    print(value, float(weight))
# <1> 0.16666666666666666
# <2> 0.3333333333333333
# <3> 0.5

```

- `k.kernel(v)` is the canonical weight on `v` as a float, and 0.0 if `v` is impossible. For a tuple value write `k.kernel(3, 4)` or `k.kernel((3, 4))`.
- `k.weights` is a dictionary from value tuples to exact weights (Decimals). Multiply a Decimal by a float only after `float(w)`.
- `E(k)` prints nicely but is not a float: use `as_float(E(k))`. Comparisons and arithmetic on it work directly. `E(k).raw` and `k.expectation` give the plain number or tuple, which is what `ChiSquare(...)` wants (**3.12**).
- `show(x)` prints a list or dictionary of Kinds, expectations and the like in frplib's form, one item per line, where the plain echo would be a row of `Decimal(...)`. The posted solutions use it: `show([E(Alice), E(Bob), E(Carlos)])`.
- `Kind.compare(k1, k2)` says in words how two Kinds differ, when `Kind.equal` is False.
- `frequencies(psi(v) for v in irange(1, 99))` counts how often each result occurs, which gives the kernel of a transformed uniform Kind without building it (**10.1**). Wrap it in `dict(...)` for a short echo.
- `clean(k)` drops branches whose weight is negligible (below $1e-16$ by default; `clean(k, 1e-9)` to choose) and renormalizes. `unfold(k)` shows the multi-level tree.
- `environment.set_max_denom(999)` makes the playground print fractions with larger denominators, for example 311/961 instead of 0.32362.
- `FRP.sample(10_000, k)` and `Market.demo(10_000, k)` tabulate a large demo.

2.5 Conditional Kinds

```

box = conditional_kind({      # a dictionary: input value -> Kind
    0: uniform(1, 2, 3),
    1: constant(8),
})
typeof(box)
# '1 -> 2'
box(0)
#   ,---- 1/3 ---- 1
# <> -+---- 1/3 ---- 2
#   `---- 1/3 ---- 3

@conditional_kind(codim=1)    # codim=1: the argument is a number
def bump(x):
    return choice(x, x + 1)

@conditional_kind(codim=2)    # one parameter per input component
def spread(x, n):
    return uniform(x - 1, x + 1) ^ Fork(__, n + 1)

@conditional_kind            # no codim: the argument is the whole input tuple
def last_plus(path):
    return choice(path[-1], path[-1] + 1)

f = E(box)                   # E of a conditional Kind is a function of the input
f(0)
# 2
f(1)
# 8
binary() >> box
#   ,---- 1/6 ---- <0, 1>
#   |---- 1/6 ---- <0, 2>
# <> -|
#   |---- 1/6 ---- <0, 3>
#   `---- 3/6 ---- <1, 8>

```

```
box // binary()
#      ,---- 1/6 ---- 1
#      |---- 1/6 ---- 2
# <> -|
#      |---- 1/6 ---- 3
#      `---- 3/6 ---- 8
```

- Every target must be a Kind (`constant(v)` for a sure value), and all targets must have the same dimension.
- The domain must contain every value the conditional Kind will be asked about. In `a >> m`, `m` receives the **whole** value of `a`, not only its last component.
- `m(u)`, or `m.target(u)`, is the target Kind for input `u`. `m.joined(u)` keeps the input in front. So a condition on `m(u)` counts components from the target's first one (`Proj[1]`), and on `m.joined(u)` from the input's.
- `m ^ psi` applies `psi` to the joined value (input and target together). To transform only the targets use `m.transform_targets(psi)`.
- `m.expectation` is a statistic, so `E(k ^ m.expectation)` equals `E(m // k)`.

2.6 FRPs, clones, samples

```
X = frp(uniform(1, 2, ..., 6))      # a fresh FRP of that Kind
Y = clone(X)                        # an independent FRP with the same Kind
R = X ** 3                          # three independent clones, joined
S = Sum(R)                          # a transformed FRP; kind(S) and E(S) work on it
R.value, S.value
# ((1, 6, 5), (12,))
FRP.sample(8, S, summary=False)
# [(8,), (11,), (11,), (12,), (11,), (9,), (8,), (8,)]
Coin = frp(binary())
Box = conditional_frp({0: frp(uniform(1, 2, 3)), 1: frp(constant(8))})
Coin >> Box
# An FRP with value <0, 3>
Box // Coin
# An FRP with value <3>
```

- Values differ from run to run. An FRP's value is fixed once you have looked at it; use `clone` for a new draw.
- **How results are echoed.** One result on its own prints in `frplib`'s form: `S.value` shows a value such as `<13>`, and `E(S)` shows a number or a fraction inside a box. Several results separated by commas make a Python tuple, and the playground echoes a tuple, list or dictionary in raw form: that value as `(13,)`, an expectation as a long `Decimal(...)`, Kinds as trees run together. For the tidy form ask for one thing per line, or write `print(a, b)` or `show([a, b])`.
- `FRP.sample(10_000, S)` without `summary=False` prints a table of counts and proportions.
- For repeated use in `evolve`, define the conditional FRP with `auto_clone=True` so each step gets fresh FRPs.
- `E` of an FRP whose Kind is huge is an approximation and `frplib` says so. Prefer working with the Kind when it is small enough to compute.

2.7 Procedures: @kind and @frp

The procedural form of the Big 3. `yield` binds a name to every value of a Kind (the weights are tracked for you), `yield given(...)` imposes an observation, and `return` gives the value of the new Kind.

```
has_disease = choice(0, 1, 1/999)
test = conditional_kind({0: choice(0, 1, 1/99), 1: choice(0, 1, 19)})

@kind
def status_given_positive():
    d = yield has_disease      # stage 1
    t = yield test(d)          # stage 2, contingent on d: a join
    yield given(t == 1)        # observation: keep only positive tests
    return d                   # transform: report the disease status

status_given_positive
#      ,---- 0.91316 ----- 0
# <> -|
#      `---- 0.086837 ---- 1
```

After the definition the name is a Kind, not a function. `@frp` works the same way with FRPs and conditional FRPs. Return a tuple, `return (d, t)`, to keep several pieces.

WATCH OUT The v0.3.10 cheatsheet writes the observation line as `yield guard(cond(v))`. `frplib` 0.3.10 has no `guard`, and typing it gives name `'guard' is not defined`. The function is given, as above and as in the text.

2.8 Factories and helper functions

A factory is a function that takes parameters and returns a Kind, a statistic or a condition built from them. The returned object remembers the parameters it was made with. The decorators below register the factory with `frplib` so that it documents itself; a plain Python function that returns the object works too.

```
@kind_factory
def sum_of_dice(n, sides=6):
    "Kind of the sum of n balanced dice"
    return fast_join_pow(Sum, uniform(1, 2, ..., sides), n)

@statistic_factory
def count_of(v):
    "a statistic that counts the components equal to v"
    return ForEach(__ == v) ^ Sum

@condition_factory
def repeats_before(x):
    "a condition: does the first component repeat before x appears?"
    @condition
    def check(first, *rest):
        for r in rest:
            if r == first:
                return True
            if r == x:
                return False
        return False
    return check

E(sum_of_dice(10))
# 35
count_of(4)(4, 1, 4, 6)
# <2>
repeats_before(7)(5, 2, 5, 7)
# <1>
```

- `@frp_factory` does the same for a function that returns an FRP.
- A conditional Kind that needs parameters is built the same way: an ordinary function that defines a `@conditional_kind` inside and returns it (the lecture's `gamble_with(wealth, goal)`).

Three helpers that come up in custom statistics and weights:

```
index_of(7, (5, 2, 7, 7)), index_of(9, (5, 2, 7), not_found=99) # first position, from 0
# (2, 99)
index_where(lambda r: r > 4, (1, 3, 6, 8)) # first position passing a test
# 2
pts = [(x, y) for x in (-1, 0, 1) for y in (-1, 0, 1)]
kw = weighted_by(pts, weight_by=sqrt(1 + SumSq)) # scalar_fn: statistic -> function
kw.kernel(0, 0), kw.kernel(1, 1)
# (0.047619047619047616, 0.14285714285714285)
```

2.9 Predict the output: a drill

Cover each teal line, say what the statement prints, then look. To save space a Kind is written here on one line as `value`: `weight` (the playground draws the tree), and `frplib` prints a set of weights over a common denominator, so `3/9` rather than `1/3`. The lines run in order in one session.

```
d = uniform(1, 2, 3)
c = weighted_as(0, 1, weights=[1, 3])
size(d ** 3), dim(d ** 3)
# (27, 3)
size(d * c), dim(d * c)
# (6, 2)
E(d * c)
# <2, 3/4>
d ** 2 ^ Sum
# 2: 1/9   3: 2/9   4: 3/9   5: 2/9   6: 1/9
d ^ (__ % 2)
# 0: 1/3   1: 2/3
uniform(-1, 0, 1) ^ Abs
# 0: 1/3   1: 2/3
d ** 2 ^ (Proj[1] == Proj[2])
# 0: 2/3   1: 1/3
d ** 2 ^ Max - Min
# 0: 3/9   1: 4/9   2: 2/9
d ^ Fork(__, __ + 1)
# <1, 2>: 1/3   <2, 3>: 1/3   <3, 4>: 1/3
weighted_as(1, 2, 3, weights=[2, 2, 4])
# 1: 1/4   2: 1/4   3: 2/4
d ^ (__ > 1)                                # ask whether it is true
# 0: 1/3   1: 2/3
d | (__ > 1)                                # take it as known
# 2: 1/2   3: 1/2
d ** 2 | (Sum == 4)
# <1, 3>: 1/3   <2, 2>: 1/3   <3, 1>: 1/3
Proj[1] @ d ** 2 | (Sum == 5)
# 2: 1/2   3: 1/2
d | (__ > 1) | (__ < 3)
# 2: 1
d | (__ > 5)
# the empty Kind (no branches)
E(d | (__ > 1))
# 5/2
E(d ^ (__ > 1))
# 2/3
```

```
m = conditional_kind({1: constant(0), 2: choice(0, 1), 3: constant(1)})
typeof(m), size(d >> m), dim(d >> m)
# ('1 -> 2', 4, 2)
d >> m
# <1, 0>: 2/6   <2, 0>: 1/6   <2, 1>: 1/6   <3, 1>: 2/6
m // d
# 0: 1/2   1: 1/2
Proj[1] @ (d >> m) | (Proj[2] == 1)          # Bayes: the early stage given the later one
# 2: 1/3   3: 2/3
E(m // d)
# 1/2
E(d >> m)
# <2, 1/2>
E(d)
# 2
E(d ^ (__ ** 2))
# 14/3
Var(d)
# 2/3
E(d ** 2 ^ Max)
# 22/9
```

```
E(d ** 2 ^ Sum) == 2 * E(d)
# True
E(d ** 2 ^ (Proj[1] * Proj[2]))          # independent, so E(d) times E(d)
# 4
```

Statistics applied to plain tuples, each entered on its own. The result is always a tuple, printed in angle brackets even when it has one component.

You type	It prints	You type	It prints
Sum(1, 2, 3)	<6>	Diff(1, 4, 9)	<3, 5>
Count(4, 4)	<2>	Max(1, 2, 3)	<3>
Proj[2](7, 8, 9)	<8>	Proj[2:](7, 8, 9)	<8, 9>
Proj[-1](7, 8, 9)	<9>	Proj[1, 3](7, 8, 9)	<7, 9>
(Sum ^ (__ * 2))(1, 2, 3)	<12>	Fork(Sum, Count)(4, 5, 6)	<15, 3>
ForEach(__ % 2)(1, 2, 3)	<1, 0, 1>	(ForEach(__ == 4) ^ Sum)(4, 1, 4)	<2>
IfThenElse(__ > 2, __ * 10, 0)(3)	<30>	Cases({1: 10, 2: 20}, default=0)(3)	<0>
And(__ > 0, __ < 5)(7)	<0>	Or(__ < 0, __ > 5)(7)	<1>
Not(__ > 5)(7)	<0>	(Max - Min)(3, 1, 4)	<3>

Lines that do not do what they look like. The first is fine. The next three run without any error and print something other than the Kind you wanted. The last three fail.

```
d ^ __ + 1          # fine: + binds tighter than ^
# 2: 1/3   3: 1/3   4: 1/3
d ^ __ % 2 == 0     # read as (d ^ (__ % 2)) == 0
# False
d ** 2 | Sum == 4    # read as (d ** 2 | Sum) == 4
# False
d ** 2 | (Sum == 4) ^ Proj[1]  # the projection went to the condition
# <1, 3>: 1/3   <2, 2>: 1/3   <3, 1>: 1/3
d ^ __ > 1
# TypeError: '>' not supported between instances of 'Kind' and 'int'
d + 1
# TypeError: unsupported operand type(s) for +: 'Kind' and 'int'
d ^ 2
# Invalid value transform or join provided: should be function or mapping dictionary
```

- A bare True or False where you expected a Kind means a comparison escaped its parentheses. Arithmetic inside a statistic needs none after ^. Comparisons always do.
- `d ** 2 | (Sum == 4) ^ Proj[1]` still shows both components. Write `(d ** 2 | (Sum == 4)) ^ Proj[1]` or `Proj[1] @ d ** 2 | (Sum == 4)`.
- A Kind is not a number: `d + 1` fails, and the transform `d ^ (__ + 1)` is what was meant. ^ wants a statistic on its right, and the power is **.
- `d | (__ > 5)` is the empty Kind, which the playground prints as `<> -+`. If you see that, the condition was impossible or was applied to the wrong components.

3 Pattern templates

Each template is a small complete example. Swap in the problem's Kinds and statistic. The blocks in this section run in order in one session, so later blocks reuse names such as `d6` defined earlier.

3.1 Independent trials, then ask a question

Build the Kind of all the trials with `**` (or `*` for different Kinds), then transform it with the statistic that expresses the question. A probability is `E` of a condition.

```
d6 = uniform(1, 2, ..., 6)
rolls = d6 ** 3                                # three independent rolls: the Data Kind
size(rolls), dim(rolls)
# (216, 3)
E(rolls ^ Sum)
# 21/2
E(rolls ^ (Sum > 10))                          # probability the sum exceeds 10
# 1/2
E(rolls ^ (ForEach(__ == 4) ^ Sum >= 2))        # probability of at least two 4s
# 2/27
mixed = d6 * binary('1/3') * constant(7)      # different Kinds: join with *
E(mixed)
# <7/2, 1/3, 7>
```

The size of `k ** n` is `size(k)n`. Past a few hundred thousand branches, switch to [3.2](#).

3.2 Large powers: the monoidal trick

For Sum, Min or Max, `fast_join_pow(stat, k, n)` computes `stat(k ** n)` without ever building `k ** n`. This is Alice and Bob's trick from Chapter 7: these statistics can be computed in pieces, $\psi(a :: b) = \psi(\psi(a) :: \psi(b))$, so the Kind for `n` trials is built from the Kind for `n/2` trials.

```
heads = fast_join_pow(Sum, binary(), 100)      # Sum(binary() ** 100), not 2**100 branches
size(heads)
# 101
E(heads)
# 50
E(heads ^ (__ >= 60))
# 0.0284439668204904
lo_hi = fast_join_pow(Fork(Min, Max), d6, 50)  # Kind of <min, max> over 50 rolls
E(lo_hi ^ (Proj[2] - Proj[1] == 5))            # probability both a 1 and a 6 appear
# 0.9997802319300942
mean30 = fast_join_pow(Sum, d6, 30) ^ (__ / 30) # Mean is not monoidal, but Sum is
E(mean30)
# 7/2
s2 = Sum(d6 ** 2)                              # the trick by hand: sum of 2, twice
Kind.equal(Sum(s2 * s2), Sum(d6 ** 4))
# True
```

WATCH OUT `fast_join_pow` applies the statistic to pairs of partial results, so it is right only when $\psi(a :: b) = \psi(\psi(a) :: \psi(b))$ holds as written. In `frplib` 0.3.10 that covers Sum, Min, Max and Fork(Min, Max). It does **not** cover Count (the count of two counts is 2), or a Fork that mixes Sum with Min or Max, although `frplib` and the text describe those as monoidal too. For them `fast_join_pow` returns a wrong Kind with no warning. To count matches, turn each trial into 0 or 1 and use Sum. For any other statistic, test a small power first with `Kind.equal`, or combine the pieces yourself as below.

```
Kind.equal(fast_join_pow(Count, d6, 3), Count(d6 ** 3))    # not what it promises
# False
Kind.equal(fast_join_pow(Fork(Sum, Max), d6, 3), d6 ** 3 ^ Fork(Sum, Max))
# False
fours = fast_join_pow(Sum, d6 ^ (__ == 4), 50)             # number of 4s in 50 rolls: Sum of 0/1 flags
E(fours)
# 25/3
```

```
def join_pow_by(merge, one, n):
    "Kind of a summary of n trials: `one` summarizes one trial, `merge` combines two summaries"
    if n == 1:
        return one
    half = join_pow_by(merge, one, n // 2)
    both = half * half ^ merge
    return both if n % 2 == 0 else one * both ^ merge

add_and_max = Fork(Proj[1] + Proj[3], Proj[2, 4] ^ Max)          # <s1, m1, s2, m2> -> <s1 + s2, max>
sum_max = join_pow_by(add_and_max, d6 ^ Fork(__, __), 5)      # <sum, max> of five rolls
Kind.equal(sum_max, d6 ** 5 ^ Fork(Sum, Max))
# True
```

- The text's own version of `fast_join_pow` is `monoid_trick` in `frplib.examples.alice_bob`, with the same limits.
- Lecture 13 prints the source of `fast_join_pow`. It finds the `Kind` for $n // 2$ trials, `kn2`, and returns `mstat(kn2 * kn2)`, with one more `mstat(k * ...)` around it when n is odd. The statistic is applied to a pair of earlier results, which is right for `Sum` and wrong for `Count`.
- If the statistic is not monoidal and the `Kind` is too big, sample instead: `FRP.sample(10_000, psi(frp(k) ** n))` gives approximate proportions.
- Another route is to carry a small state through the trials with `evolve` (3.6).

3.3 Updating with an observation

Update the Data `Kind` with the observation, `k | (cond)`, then transform. The `@` form lets the condition talk about the original values while you extract a feature.

```
two = d6 ** 2
two | (Sum == 10)
#      ,---- 1/3 ---- <4, 6>
# <> -+---- 1/3 ---- <5, 5>
#      `---- 1/3 ---- <6, 4>
Proj[1] @ two | (Sum >= 11)          # first die, given the sum is at least 11
#      ,---- 1/3 ---- 5
# <> -|
#      `---- 2/3 ---- 6
E(Max @ two | (Proj[1] == 2))        # predicted max, given the first die is 2
# 11/3
E(two ^ (Sum >= 11))                 # probability of the observation itself
# 1/12
```

Shortcut for powers. When an observation restricts every trial separately, update the base `Kind` first and then take the power. "The minimum is 3" means every draw is at least 3 and some draw equals 3:

```
base = weighted_as(1, 2, 3, 4, weights=[4, 3, 2, 1])
direct = base ** 4 | (Min == 3)
short = (base | (__ >= 3)) ** 4 | (Min == 3)    # restrict each draw, join, then finish
Kind.equal(direct, short), size(short)
# (True, 15)
```

- Conditional probability of A given C : $E(A @ k | C)$. It equals $E(k ^ \text{And}(A, C)) / E(k ^ C)$.
- Observations never move weight between surviving branches. If the survivors had equal weights before, they have equal weights after.
- Test a shortcut like this on a small power, as above, before using it on the real one.

3.4 Two stages: conditional Kinds and joins

Describe the later stage as a conditional Kind: for each value of the early stage, the Kind of what happens next. `>>` keeps both stages. `//` keeps only the later one.

```
coin = binary()                                # 0 tails, 1 heads
pick = conditional_kind({
  0: uniform(1, 2, ..., 10),                  # tails: any of ten boxes
  1: constant(8),                             # heads: always box 8
})
both = coin >> pick                             # Kind of <flip, box>
size(both), dim(both)
# (11, 2)
E(both ^ (Proj[2] == 8))                       # probability box 8 is chosen
# 0.55
E(pick // coin)                                # predicted box number
# 27/4
```

When the next stage depends on the earlier value by a rule, use a function:

```
first = uniform(1, 2, ..., 10)

@conditional_kind(codim=1)
def second(card):
    return uniform(irange(1, 10, exclude={card}))

@conditional_kind
def third(cards):
    return uniform(irange(1, 10, exclude=set(cards)))

draw = first >> second
E(draw ^ (Proj[2] == 2 * Proj[1]))              # second card is twice the first
# 1/18
Kind.equal(draw, ordered_samples(2, irange(1, 10)))
# True
three = first >> second >> third
size(three), dim(three)
# (720, 3)
```

- `unfold(both)` displays the two-level tree, which is the picture to draw if asked (5.5).
- Weights multiply along a path: the weight of $\{0, 3\}$ above is $1/2 \cdot 1/10$.
- For FRPs the same code works with `frp(...)` and `conditional_frp(...)`.

3.5 Bayes's Rule: learn the early stage from the later one

Join, update with the observed later value, project onto the early components.

```
has_disease = choice(0, 1, 1/999)               # 1 person in 1000
test = conditional_kind({
  0: choice(0, 1, 1/99),                       # healthy: 1% test positive
  1: choice(0, 1, 19),                         # sick: 95% test positive
})
joint = has_disease >> test                     # <status, result>
Proj[1] @ joint | (Proj[2] == 1)
# ,---- 0.91316 ---- 0
# <> -|
# `---- 0.086837 ---- 1
Kind.equal(bayes(1, has_disease, test), Proj[1] @ joint | (Proj[2] == 1))
# True
status_given_test = conditional_kind({r: bayes(r, has_disease, test) for r in (0, 1)})
```

By hand: multiply prior weight by the weight of the observation, then normalize. Healthy: $0.999 \cdot 0.01 = 0.00999$. Sick: $0.001 \cdot 0.95 = 0.00095$. So the probability of disease given a positive test is $0.00095 / (0.00999 + 0.00095) = 0.0868$.

A chain of three stages (supplier, then resin, then quality). The last stage depends only on the middle one, so collapse the middle with `//` before applying Bayes:

```
supplier = weighted_as(1, 2, 3, weights=['1/2', '1/3', '1/6'])
resin = conditional_kind({1: binary('1/10'), 2: binary('1/2'), 3: binary('9/10')})
quality = conditional_kind({0: binary('9/10'), 1: binary('1/5')}) # given the resin only
quality_by_supplier = conditional_kind({s: quality // resin(s) for s in (1, 2, 3)})
bayes(0, supplier, quality_by_supplier) # supplier, given a low-quality part
#      ,---- 0.23832 ---- 1
# <> -+---- 0.42056 ---- 2
#      `---- 0.34112 ---- 3

@conditional_kind(codim=2) # to keep the whole history instead
def quality_h(s, r):
    return quality(r)

full = supplier >> resin >> quality_h # <supplier, resin, quality>
Kind.equal(Proj[1] @ full | (Proj[3] == 0), bayes(0, supplier, quality_by_supplier))
# True
```

WATCH OUT `supplier >> resin >> quality` raises an error: in a join the conditional Kind is handed the whole value so far, here (supplier, resin), and `quality` is only defined on the resin. Either collapse with `//` or define it on the full history. The posted homework 5 solution does the second with a dictionary keyed by (supplier, resin) (9.1).

3.6 Evolving a state

The example is a random walk, the gambler's ruin: she bets a dollar at a time until she is down 3 or up 2. Those two states are absorbing.

Design the state first. It must hold what you need to decide what can happen next and what you need to answer the question. Then write three things: the Kind of the initial state, the conditional Kind of the next state given the current one, and a statistic that reads the answer off the final state. 7.2 lists state designs for more than a dozen systems.

```
bet = choice(-1, 1) # lose or win a dollar, equal weights

@conditional_kind(codim=1)
def next_state(s): # state: net winnings so far
    if s == -3 or s == 2: # ruined, or goal reached: frozen
        return constant(s)
    return bet ^ (__ + s)

after = evolve(constant(0), next_state, 200) # same as 200 times: state = next_state // state
clean(after)
#      ,---- 2/5 ---- -3
# <> -|
#      `---- 3/5 ---- 2
```

Add a clock when the question is about how long, or about a deadline:

```
@conditional_kind(codim=2)
def next_state_t(s, n): # state: <net winnings, bets made>
    if s == -3 or s == 2:
        return constant(s, n)
    return bet ^ Fork(__ + s, n + 1)

st = evolve(constant(0, 0), next_state_t, 200)
finished = Or(Proj[1] == -3, Proj[1] == 2)
as_float(1 - E(st ^ finished)) # weight still unresolved: the approximation error
# 4.571083928e-19
done = st | finished
E(done ^ Proj[2]) # predicted number of bets
# 6
E(done ^ (Proj[2] <= 6)) # probability it is over within 6 bets
# 0.671875
```

- **Freeze terminal (absorbing) states** by returning `constant(state)`. Then evolving for extra steps changes nothing, and "by step n" questions are answered by evolving n steps.
- **Keep the state a fixed dimension.** `frplib` cannot apply a conditional Kind (`>>`, `//`, `evolve`) to a Kind whose values have different dimensions. The ragged Kind itself is built without complaint and prints its short values padded with `□`, and the error arrives one step later. If a history has to stop growing, pad it (repeat the final entry), as in [4.4](#) and in the posted robot solution ([8.3](#)).
- **Run until resolved.** Instead of guessing a number of steps, loop until the unresolved weight is below a tolerance, as the posted practice solution does: `while (curr ^ Proj[2]).kernel(0) > 1e-7: curr = next_state // curr` ([4.5](#)).
- Build the next state from a simple Kind with a statistic: `bet ^ Fork(__ + s, n + 1)`.
- If the next step depends on the whole path, keep the history with repeated joins, `path = path >> step` in a loop, as in [8.3](#) and [7.1](#).
- `evolve(start, next_state, n, transform=clean)` prunes negligible branches at every step when the Kind grows large.
- A system built from FRPs works the same way with a conditional FRP defined with `auto_clone=True`.

3.7 A random number of random amounts

N things happen, each contributing an independent amount; find the total T. Given $N = n$ the total is $\text{Sum}(\text{amount} \times n)$. Make that a conditional Kind and condition on N.

```
kind_N = uniform(0, 1, ..., 4)                # how many customers
purchase = weighted_as(0, 1, 2, weights=['3/4', '1/6', '1/12'])

@conditional_kind(codim=1)
def total_given_n(n):
    return fast_join_pow(Sum, purchase, n)      # for n = 0 this is constant(0)

total = total_given_n // kind_N                # Kind of the total
as_float(E(total)), as_float(E(kind_N)) * as_float(E(purchase))
# (0.6666666666666666, 0.6666666666666666)
```

The FRP version, when the problem asks you to construct and simulate the FRPs:

```
N = frp(kind_N)

@conditional_frp(codim=1)
def Total_given_N(n):
    return Sum(frp(purchase) ** n)              # for n = 0 this is an FRP with value 0

T = N >> Total_given_N ^ Proj[2]                # the same as Total_given_N // N
N.value
# <1>
T.value
# <0>
FRP.sample(10, T, summary=False)              # ten fresh clones of T
# [(1,), (1,), (2,), (0,), (0,), (3,), (1,), (3,), (2,), (0,)]
```

- $E(T) = E(N) \cdot E(\text{one amount})$, because $E(T \mid N = n) = n \cdot E(\text{amount})$ by additivity.
- Updating on what has been seen so far depends on how you read the observation. [4.7](#) works three readings of one sentence, starting with the posted ones.

3.8 Fixed points: when the future looks like the beginning

For "how many flips until a pattern appears". List the short prefixes that either finish or put you back at the start. Write the Kind of the total as a function of the Kind k of what remains after a restart, then find the k that this function leaves unchanged by iterating. Example: waiting for two heads in a row.

```
def wait_for_HH(k, p=0.5):
    "k is the Kind of the flips still needed once we are back at the start"
    q = 1 - p
    prefix = weighted_as(0, 2, 3, weights=[q, p * q, p * p])    # T, H T, H H
    given_prefix = conditional_kind({
        0: k ^ (__ + 1),          # T: one flip used, start over
        2: k ^ (__ + 2),          # H T: two flips used, start over
        3: constant(2),          # H H: done in two flips
    })
    return given_prefix // prefix

hh = iterate(wait_for_HH, 60, constant(2))    # apply the function 60 times to a first guess
hh.kernel(2), hh.kernel(3), hh.kernel(4)
# (0.25, 0.125, 0.125)
as_float(E(hh))
# 5.999999872433748
```

- After n iterations the weights on values up to about n are exact. The expectation is slightly low because the tail is cut off, so iterate more or solve exactly.
- Exact expectation.** Replace each Kind by its expectation: $E = q(1 + E) + pq(2 + E) + p^2 \cdot 2$. With $p = q = 1/2$ this is $E = 1.5 + 0.75 E$, so $E = 6$.
- Exact weights.** Equate weights branch by branch: $t(n) = q \cdot t(n - 1) + pq \cdot t(n - 2)$ for $n > 2$, with $t(2) = p^2$.
- Pass other arguments through `iterate`: `iterate(wait_for_HH, 60, constant(2), p=0.75)`.
- The textbook versions are in `frplib.examples.waiting_for_heads`, `doubled_heads_and` and `recursive_rover`.
- Lecture 13's `wait_for_two_heads` is this function with a symbol q for the probability of tails. Its expectation is $E(N) = (1 + p) / p^2$, which is 6 for a fair coin and 8.75 for $p = 0.4$. For any other pattern, use the position along the pattern as the state (3.18).

3.9 First-step equations for exact answers

First-step analysis, the lecture's Dirichlet problem: hitting probabilities and expected stopping times. Condition on the first step. Let k_s be the Kind of the next state from state s . Both rows below are the one equation $f(s) = \alpha + \beta \cdot E(f(k_s))$, with $\alpha = 0$ for a probability and $\alpha = 1$ for a time, and $\beta = 1$.

Quantity $f(s)$, starting from state s	Boundary values	Equation at every other state
Probability of reaching set A before set B	$f = 1$ on A, $f = 0$ on B	$f(s) = E(f(k_s)) = \sum \text{weight} \cdot f(\text{next state})$
Predicted number of steps until the system stops	$f = 0$ on the stopping states	$f(s) = 1 + E(f(k_s))$

Small systems solve by hand. The recursive rover: $E = (1/3) \cdot 3 + (1/3)(5 + E) + (1/3)(7 + E)$, so $E = 15$. For the gambler's ruin, a fair ± 1 walk started at 0 that stops at $-w$ or $+b$: the probability of reaching $+b$ first is $w / (w + b)$, and the predicted duration is $w \cdot b$. With win to lose ratio $r \neq 1$ the probability is $(1 - r^{-w}) / (1 - r^{-(w+b)})$. More systems are set up and solved by hand in 5.6.

`frplib` ships a solver for larger systems (it needs `numpy` and `scipy`):

```
from frplib.examples.dirichlet import solve_dirichlet_sparse
w, b = 10, 15

@conditional_kind(codim=1, domain=irange(-w, b))
def step(s):
    if s == -w or s == b:
        return constant(s)
    return bet ^ (__ + s)

states = [tup(v) for v in irange(-w, b)]
```

```

reach = solve_dirichlet_sparse(step, states=states, fixed=[{tup(-w)}, {tup(b)}],
                              fixed_values=[0, 1])
reach(0)                                # probability of +15 before -10
# 0.39999999999999936
steps = solve_dirichlet_sparse(step, states=states, fixed=[{tup(-w)}, {tup(b)}],
                              fixed_values=[0], alpha=1)
steps(0)                                # predicted number of bets
# 149.99999999999991

```

3.10 Expectations and probabilities

```

k = weighted_as(-200, 0, 10, 100, weights=[1, 1, 5, 3])
E(k)
# 15
E(k ^ ( __ ** 2))
# 7050
Var(k)
# 6825
E(k ^ ( __ > 0))                                # probability of a positive payoff
# 4/5
E(k | ( __ > 0))                                # prediction given that it is positive
# 175/4
E(d6 ** 2)                                    # one number per component
# <7/2, 7/2>
E(pick)(0)                                    # E of a conditional Kind is a function
# 11/2
E(pick)(1)
# 8
mu = as_float(E(k))
E(k ^ Abs(__ - mu))                            # mean absolute deviation (MAD)
# 51
as_float(Var(k)) ** 0.5                        # standard deviation
# 82.61355820929153
Market.buy(20_000, [10, 15, 20], k)          # prints net payoff per unit at each price

```

- **By hand:** a weighted average. Here $(-200 \cdot 1 + 0 \cdot 1 + 10 \cdot 5 + 100 \cdot 3) / 10 = 15$.
- **Of a transform:** average $\psi(v)$, not ψ of the average. $E(k^2)$ above is not 15^2 .
- **Linear pieces:** $E(a + bX) = a + b \cdot E(X)$ and $E(X + Y) = E(X) + E(Y)$ always.
- **Counting:** write a count as a sum of 0/1 events. Its expectation is the sum of their probabilities. The number of 4s in three rolls has expectation $3 \cdot 1/6$.
- **Does it favor you:** compute E of your net payoff. Negative favors the house.
- **Through a join:** $E(\text{later}) = \sum \text{weight}(u) \cdot E(\text{later} \mid \text{early} = u)$, which is $E(m \text{ // } k)$ or $E(k \wedge m.\text{expectation})$.
- **Estimating a price by experiment:** in the `Market.buy` table, prices with a clearly positive net payoff per unit (10 here) are arbitrage prices, and the risk-neutral price is where the payoff per unit crosses zero (15 here). `Market.compare(n, k1, k2)` demos two Kinds side by side.
- **Variance, StdDev, Mean and Median** with capital letters are statistics: they summarize the components of one value. $\text{Var}(k)$ and $E(k)$ act on a Kind.

3.11 Likelihood and the maximum likelihood estimate

The likelihood of a candidate model is the canonical weight its Kind gives to what you observed. Compute it for each candidate and pick the largest.

```

fair = uniform(1, 2, ..., 6)
loaded = weighted_as(1, 2, ..., 6, weights=[1, 1, 1, 1, 1, 5])
fair.kernel(6), loaded.kernel(6)              # likelihoods of one observed 6
# (0.16666666666666666, 0.5)
data = (6, 3, 6, 6, 2, 6)                    # independent rolls of the unknown die
loglik = ijoin_log_likelihood(data)           # a function of the base Kind (natural log)
loglik(fair), loglik(loaded)                  # larger wins: the estimate is loaded
# (-10.75055681536833, -7.3777589082278725)
import math

```

```
sum(math.log(loader.kernel(x)) for x in data) # the same by hand: add up the logs of the weights
# -7.377758908227872
(loader ** 2).kernel(6, 3) # likelihood of a joined value, directly
# 0.05
```

WATCH OUT `ijoin_log_likelihood` takes the observed values as a tuple, as above, or as separate arguments. In `frplib` 0.3.10 a list fails with `as_quantity: could not convert [...] to a quantity`, although the posted practice solution types one (4.5). The result is a float. For data whose pieces are tuples, give the flattened values: the function splits them by the dimension of the Kind.

A continuous parameter. Write the likelihood as a formula and maximize it. With symbols `frplib` gives the formula:

```
p = symbol('p')
flip = weighted_as(0, 1, weights=[1 - p, p])
(flip ** 5).kernel(1, 0, 1, 1, 0) # likelihood of H T H H T as a function of p
# p^3 + -2 p^4 + p^5
```

- That polynomial is $p^3(1 - p)^2$. In general $L(p) = p^a(1 - p)^b$ is largest at $p = a / (a + b)$: take logs, $a \cdot \ln p + b \cdot \ln(1 - p)$, and set the derivative to zero.
- For a join, multiply the normalized weights along the observed path. A chain that switches state with probability u and was seen to switch 4 times and stay 12 times has $L(u) = (1/2) \cdot u^4 (1 - u)^{12}$, so the estimate is $4/16 = 1/4$.
- For a list of candidate Kinds: `{name: loglik(k) for name, k in candidates.items()}`.
- The by-hand line is homework 4's problem 5(a): for independent data the log-likelihood is the sum of the log weights (the two results differ only by rounding in the last digit). It also works on a `frplib` that is too old to have `ijoin_log_likelihood` (6.4).

3.12 Hypothesis test threshold

Find the Kind of the test statistic when the null hypothesis is true, then take the smallest threshold t whose exceedance probability is at most α .

```
null_stat = fast_join_pow(Sum, binary(), 20) # number of 1s in 20 fair bits, under the null
alpha = 0.05
t = 0
while E(null_stat ^ (__ > t)) > alpha: # raise t until P(stat > t) <= alpha
    t += 1

t, as_float(E(null_stat ^ (__ > t)))
# (14, 0.020694732666015625)
```

Then compute the statistic on the observed data and reject the null if it exceeds t , otherwise retain it. For two single models the statistic is the likelihood ratio, and on the log scale it is `loglik(alternative) - loglik(null)`.

A chi-square statistic: the runs test from homework 5. The null hypothesis is that n bits are independent and fair. Count the runs, compare the counts with their expectations under the null using `ChiSquare(expected)`, which computes $\sum (\text{observed} - \text{expected})^2 / \text{expected}$, and reject when that distance is large. (`hw5.py` supplies `runs3`, so `from hw5 import *` is all you need there. The one below is written out so this section stands alone, and it returns the same four counts for every sequence.)

```
@statistic
def runs3(bits):
    counts = [0, 0, 0, 0]
    m = 1
    for prev, cur in zip(bits, bits[1:]):
        if cur == prev:
            m += 1
        else:
            counts[0] += 1
            if m <= 3:
                counts[m] += 1
            m = 1
    counts[0] += 1
    if m <= 3:
        counts[m] += 1
    return counts

runs3(0,1,1,0,0,0,1,0,0,1,1,0,1,0,0,1,1,1,1)
# <10, 4, 4, 1>
n_bits = 10
null_runs = binary() ** n_bits ^ runs3
erc = E(null_runs).raw
erc
# <5.5000000000, 3.0000000000, 1.3750000000, 0.6250000000>
psi = runs3 ^ ChiSquare(erc)
null_psi = clean(binary() ** n_bits ^ psi)

def threshold(k, alpha):
    "smallest value t of the scalar Kind k with P(value > t) <= alpha"
    tail = 1
    for v, w in sorted(k.weights.items()):
        tail = tail - w
        if tail <= alpha:
            return v[0]

t_runs = threshold(null_psi, 0.075)
float(t_runs), as_float(E(null_psi ^ (__ > t_runs)))
# (6.015151515151515, 0.07421875)
thresh = 4.0
while (null_psi ^ (__ > thresh)).kernel(1) > 0.10:
    thresh += 0.05

round(thresh, 2), (null_psi ^ (__ > thresh)).kernel(1)
# (6.05, 0.07421875)
as_float(psi(0,1,0,1,0,1,0,1,0,1))
# 22.015151515151516
as_float(psi(1,1,0,1,0,0,0,1,1,0))
# 0.5545454545454546
```

The two thresholds are the same test. The Kind of the statistic under the null has no value between 6.0152 and 6.7182, so every threshold in that gap is exceeded with probability 0.0742. The posted solution reports 6.05 for ten bits at $\alpha = 0.10$, and expected counts (5.5, 3, 1.375, 0.625).

For long sequences `binary()` `** n` is too big, so carry a partial runs count through the bits one at a time: (total, runs of 1, of 2, of 3, length of the current run, its bit). The current run is only counted once it ends.

```
@conditional_kind(codim=6)
def next_bit(r, r1, r2, r3, m, b):
    same = (r, r1, r2, r3, m + 1, b)          # the next bit continues the run
    ended = [r + 1, r1, r2, r3]              # the next bit ends the run
    if m <= 3:
        ended[int(m)] += 1
    return uniform(same, tuple(ended) + (1, 1 - b))

@statistic(codim=6, dim=4)
def finalize(r, r1, r2, r3, m, b):           # no more bits: count the last run
    out = [r + 1, r1, r2, r3]
    if m <= 3:
        out[int(m)] += 1
    return out

start = binary() ^ Fork(0, 0, 0, 0, 1, __)    # the state after one bit
Kind.equal(evolve(start, next_bit, n_bits - 1) ^ finalize, null_runs)
# True
```

- The posted solution's pieces have the same shape under other names: `initial_partialrc = choice((0, 0, 0, 0, 1, 0), (0, 0, 0, 0, 1, 1))`, a `next_bit` that returns `choice(eq_b_last, neq_b_last)`, and `k50 = evolve(initial_partialrc, next_bit, 49) ^ finalize_counts`. Then `erc_50 = k50.expectation`, `cs_50 = k50 ^ ChiSquare(erc_50)`, and the same search with $\alpha = 0.075$ gives a threshold of 7.9 for 50 bits. The posted solution reports 7.9 again for 64 bits (9.5).
- That 49-step evolve is slow, 5 to 10 minutes: `k50` has 12,376 values, and the state `Kind` before `finalize_counts` reaches 245,176. Start it early if you need it. 9.5 has the results for the homework's sequences, and a formula for the expected counts that needs no evolve.
- If even the evolved `Kind` is too slow, find the threshold by simulation: compute the statistic on many data sets drawn from the null with `FRP.sample(n, X, summary=False)` and take the value that only a fraction α of them exceed (8.5).

3.13 Symbolic weights

```
heads3 = flip ** 3 ^ Sum                    # flip was defined with the symbol p above
heads3
#      ,---- 1 + -3 p + 3 p^2 + -1 p^3 ---- 0
#      |---- 3 p + -6 p^2 + 3 p^3 ----- 1
# <> -|
#      |---- 3 p^2 + -3 p^3 ----- 2
#      `---- p^3 ----- 3
E(heads3)
# 3 p
E(heads3 ^ (__ >= 2))
# 3 p^2 + -2 p^3
substitution(E(heads3 ^ (__ >= 2)), p='1/2')
# <0.5>
```

- `a, b = symbols('a b')` makes several symbols. `weighted_as(0, 1, weights=[1 - p, p])` is the symbolic version of `binary(p)`.
- Joins, observations, `bayes` and `E` all work with symbols. `unfold` does not.
- `substitution(x, p=0.3)` plugs in numbers, and works on a whole `Kind` as well. Follow it with `clean` if some weights become zero.

3.14 Checking independence

```
joint = d6 ** 2 ^ Fork(Sum, Diff)           # <sum, difference> of two rolls
Kind.equal(joint, Proj[1](joint) * Proj[2](joint))
# False
flips = binary() ** 2 ^ Fork(Proj[1], Sum % 2) # <first flip, parity of the two>
Kind.equal(flips, Proj[1](flips) * Proj[2](flips))
# True
Kind.equal(Proj[1] @ joint | (Proj[2] == 0), Proj[1](joint)) # observing changes things
# False
```

The joint Kind equals the independent join of its two marginals exactly when the components are independent. The last line is the other test: does an observation of one component change the Kind of the other? Check every possible observed value.

3.15 Entropy and mutual information

```
kx = weighted_as(1, 2, 3, weights=['1/2', '1/4', '1/4'])
kx.entropy
# Decimal('1.50')
cz = conditional_kind({1: choice(0, 1), 2: constant(0), 3: choice(0, 1, 3)})
as_float(cz.conditional_entropy(3))           # H(Y | X = 3)
# 0.8112781244591328
average_conditional_entropy(kx, cz)          # H(Y | X)
# Decimal('0.7028195311147832159774239480')
(cz // kx).entropy                           # H(Y)
# Decimal('0.9886994082884974988586821119')
mutual_information(kx, cz)                   # I(Y; X) = H(Y) - H(Y | X)
# Decimal('0.2858798771737142828812581639')
Kind.divergence(binary(0.5), binary(0.9))   # KL(first; second)
# 0.7369655941662062
```

- By hand: $H = -\sum p \cdot \lg p$. For weights 1/2, 1/4, 1/4: $(1/2)(1) + (1/4)(2) + (1/4)(2) = 1.5$ bits.
- $h(1/2) = 1$, $h(1/3) = 0.918$, $h(1/4) = 0.811$, $h(0.9) = 0.469$, $h(0.99) = 0.081$.
- $\lg 3 = 1.585$, $\lg 5 = 2.322$, $\lg 6 = 2.585$, $\lg 7 = 2.807$, $\lg 10 = 3.322$.
- `Kind.divergence` returns infinity whenever the two Kinds do not have exactly the same set of values.
- The entropy functions return exact decimals, which the playground echoes as `Decimal('...')`. Wrap one in `float(...)` for a short number.

3.16 A game that runs until it resolves: craps

The homework's Lazy C Casino rules. Roll two dice. A first roll of 7 or 11 wins and a 2, 3 or 12 loses. Otherwise keep rolling until the first roll comes up again (win) or a 7 or 11 appears (lose). State: (first roll, rolls so far, outcome) with outcome 1 for a win, -1 for a loss and 0 while the game is still going.

```
d6 = uniform(1, 2, ..., 6)
two_dice = Sum(d6 * d6)
first_outcome = Cases({2: -1, 3: -1, 12: -1, 7: 1, 11: 1}, default=0)
initial = two_dice ^ Fork(__, 1, first_outcome) # the state after the first roll

@conditional_kind(codim=3)
def next_roll(f, n, w):
    if w != 0: # game over: frozen
        return constant(f, n, w)
    result = Cases({f: 1, 7: -1, 11: -1}, default=0) # repeat f: win. 7 or 11: lose
    return two_dice ^ Fork(f, n + 1, result)

game = evolve(initial, next_roll, 60)
as_float(E(game ^ (Proj[3] == 0))) # weight still unresolved
# 5.912434741534293e-11
done = game | (Proj[3] != 0)
```

```

clean(done ^ Proj[3], 1e-9)
#      ,---- 0.55141 ---- -1
# <> -|
#      `---- 0.44859 ---- 1
as_float(E(done ^ Proj[3])), as_float(E(done ^ Proj[2]))    # predicted payoff and number of rolls
# (-0.10282310280317158, 2.981351977730113)

```

The predicted payoff of about -0.103 per dollar favors the casino, and it matches the posted homework 3 solution (win probability 0.448588). The posted code takes one step at a time with `state >> next_roll ^ Proj[4, 5, 6]` and stops when the unresolved weight is below 10^{-7} , which takes 41 rolls. The same three pieces serve any game of this shape: an initial state from the first move, a frozen outcome flag, and a transition that reads the next move with `Cases`.

3.17 Decision problems: which option is best

Some problems ask for a choice: where to stand in line, which die to take, when to stop, whether to switch. The text calls this optimizing a decision over a menu of strategies (Example 8.7). The pattern is always the same.

1. List the options: a position k , a threshold t , one of a few objects, a yes or no rule.
2. For one option, build the Kind of the outcome (the payoff, or the event that you win) with the earlier templates.
3. Wrap that in a Python function of the option and tabulate its expectation over every option.
4. Answer with the best option, its value, and a sentence on why its neighbors do worse.

A threshold rule. Roll a die. You may keep the roll, or roll once more and keep that instead. The strategies are "keep the first roll if it is at least t ":

```

d6 = uniform(1, 2, ..., 6)

def keep_if_at_least(t):
    "Kind of the final roll under the strategy with threshold t"
    @conditional_kind(codim=1)
    def final(first):
        return constant(first) if first >= t else d6    # keep it, or roll again
    return final // d6

{t: round(as_float(E(keep_if_at_least(t))), 4) for t in irange(1, 6)}
# {1: 3.5, 2: 3.9167, 3: 4.1667, 4: 4.25, 5: 4.1667, 6: 3.9167}
max(irange(1, 6), key=lambda t: as_float(E(keep_if_at_least(t))))    # the best threshold
# 4

```

Keep a 4, 5 or 6: the prediction is $17/4 = 4.25$. The reason in a sentence: rolling again is worth $7/2$, so keep exactly the rolls that beat it.

A position in line: Win on Repeat (lectures 11 and 13). Tickets 1 to 100, each pick is recorded and the ticket is put back. The winner is the first person to pick a number already picked. State: (distinct numbers seen, position of the winner or 0).

```
n_tickets = 100

@conditional_kind(codim=2)
def next_player(seen, winner):
    if winner != 0:
        return constant(seen, winner)
    return weighted_as((seen, seen + 1),
                      (seen + 1, 0),
                      weights=[seen, n_tickets - seen])

final = evolve(constant(1, 0), next_player, n_tickets)
pos = final ^ Proj[2]
as_float(E(final ^ (Proj[2] == 0)))
# 0.0
[(k, round(pos.kernel(k), 5)) for k in irange(9, 13)]
# [(9, 0.06002), (10, 0.06213), (11, 0.06282), (12, 0.06219), (13, 0.06038)]
max(irange(2, 101), key=pos.kernel)
# 11
```

- By hand: player k wins when the first $k - 1$ picks are all different and pick k repeats one of them, so $P(k) = (99/100)(98/100) \cdots ((102 - k)/100) \cdot (k - 1)/100$. The ratio $P(k + 1)/P(k) = (101 - k) \cdot k / (100 \cdot (k - 1))$ is above 1 exactly while $k^2 - k < 100$, that is up to $k = 10$, so the probability rises until position 11 and falls after it.
- **Lecture 13's solution** is the same reduction with the state (tickets drawn, won): because the choice is uniform, whether a ticket repeats depends only on how many have been drawn. Its transition is `constant(n + 1) * binary(n / PLAYERS)`, its answer is position 11 with probability 0.062816, and its closed form is $P(n) = 100^{-n} \cdot 100! / (101 - n)! \cdot (n - 1)$, the formula above written with factorials. The slide's `evolve(constant(0, 0), next_ticket_n)` takes one step, because `n_steps` defaults to 1. Give the number of steps: `evolve(constant(0, 0), next_ticket_n, 100)`.
- The lecture's first two attempts keep every ticket drawn: an independent join of all 100 picks, then a join that carries the history. Those Kinds are far too big. With the history kept, three players already give a million branches and took minutes here. Keep counts, not lists, whenever only the count matters.
- **When the other player moves first** (Funny Dice, 9.2): find your best reply to each of their choices, then average over their choice with `arbitrary` weights. If no symbol is left in the answer, how they choose does not matter.
- **Switch or stay, and when to stop looking.** The text's versions are importable:

```
from frplib.examples.monty_hall import switch_win, dont_switch_win
E(switch_win)
# 2/3
E(dont_switch_win)
# 1/3
from frplib.examples.assistant_assistance import assistant, best_k
[round(float(p), 4) for p in assistant(7)]
# [0.1429, 0.35, 0.4143, 0.4071, 0.3524, 0.2619, 0.1429]
best_k(7)
# <2, 0.4142857142857142857142857143>
```

- Assistant Assistance (Example 8.7): n applicants in random order, skip the first k , then take the first one better than all of those. `assistant(n)` gives the success probability of each k . The best k is near n/e and succeeds with probability near 0.37.
- A yes or no decision is the same pattern with two options: take the bet when E of the net payoff is positive (4.8), pay at most the risk-neutral price (3.10).
- An expectation ignores spread. If the question mentions risk, also compare `Var`, or the probability of a loss, across the options.

3.18 Waiting for a pattern: the position along it (lecture 13)

How many flips until a given pattern, such as H T T H T, first appears? Lecture 13's answer needs no cleverness and is the same for every pattern: build the system one flip at a time and keep the minimal state, the **position along the pattern**, meaning how many of its leading symbols the most recent flips match. From position s , the flip that matches the next symbol moves to $s + 1$. The other flip falls back to the longest beginning of the pattern that the recent flips still match, which is often 0 but not always. The last position is frozen.

```
def next_position(pattern, s, flip):
    "the position after `flip`, when the last s flips match the first s symbols of the pattern"
    seen = pattern[:s] + flip
    for m in range(min(len(pattern), len(seen)), -1, -1):
        if seen.endswith(pattern[:m]):
            # the longest beginning of the pattern still matched
            return m

def pattern_step(pattern, p=0.5):
    "conditional Kind of the next position, for flips that are heads with probability p"
    n = len(pattern)
    p = as_quantity(p)
    @conditional_kind(codim=1, domain=irange(0, n))
    def step(s):
        if s == n:
            # the pattern has appeared: frozen
            return constant(s)
        return weighted_as(next_position(pattern, s, 'T'), next_position(pattern, s, 'H'),
                           weights=[1 - p, p])
    return step

[[next_position('HTTHT', s, flip) for flip in 'TH'] for s in range(5)] # where T and H lead
# [[0, 1], [2, 1], [3, 1], [0, 4], [5, 1]]
evolve(constant(0), pattern_step('HHH'), 10) # the position after ten fair flips
#      ,---- 0.26758 ---- 0
#      |---- 0.14551 ---- 1
# <> -|
#      |---- 0.079102 ---- 2
#      `---- 0.50781 ---- 3
```

Pattern	From each position s : where a T leads, where an H leads
HHH	$s = 0$: T $\rightarrow$ 0, H $\rightarrow$ 1; $s = 1$: T $\rightarrow$ 0, H $\rightarrow$ 2; $s = 2$: T $\rightarrow$ 0, H $\rightarrow$ 3
HTTHT	$s = 0$: T $\rightarrow$ 0, H $\rightarrow$ 1; $s = 1$: T $\rightarrow$ 2, H $\rightarrow$ 1; $s = 2$: T $\rightarrow$ 3, H $\rightarrow$ 1; $s = 3$: T $\rightarrow$ 0, H $\rightarrow$ 4; $s = 4$: T $\rightarrow$ 5, H $\rightarrow$ 1
HHTTTH	$s = 0$: T $\rightarrow$ 0, H $\rightarrow$ 1; $s = 1$: T $\rightarrow$ 0, H $\rightarrow$ 2; $s = 2$: T $\rightarrow$ 3, H $\rightarrow$ 2; $s = 3$: T $\rightarrow$ 4, H $\rightarrow$ 1; $s = 4$: T $\rightarrow$ 5, H $\rightarrow$ 1; $s = 5$: T $\rightarrow$ 0, H $\rightarrow$ 6
THTT	$s = 0$: T $\rightarrow$ 1, H $\rightarrow$ 0; $s = 1$: T $\rightarrow$ 1, H $\rightarrow$ 2; $s = 2$: T $\rightarrow$ 1, H $\rightarrow$ 3; $s = 3$: T $\rightarrow$ 4, H $\rightarrow$ 0

These are the four diagrams drawn in lecture 13. For H H H every miss returns to 0. For H T T H T, an H at position 1, 2 or 4 keeps one symbol of progress, because that H can start a new attempt.

The expected number of flips is a first-step problem (3.9, 5.6). Let E_s be the expected number of further flips from position s . At the end $E_n = 0$. Elsewhere $E_s = 1 + (1 - p) \cdot E_t + p \cdot E_h$, where t and h are the positions that a T and an H lead to from s . That is n equations in n unknowns, and in the lecture's notation $f = (I - T)^{-1} \mathbf{1}$. For H H H this gives $E_0 = 1/p + 1/p^2 + 1/p^3$, which is 14 for a fair coin, and for H H it gives $(1 + p) / p^2$, which is 6. frplib's Dirichlet solver does the algebra, and adding up the weight that has not reached the end gives the same number with no solver.

```
from frplib.examples.dirichlet import solve_dirichlet_sparse

def expected_flips(pattern, p=0.5):
    "the expected number of flips still needed, as a function of the position"
    n = len(pattern)
    return solve_dirichlet_sparse(pattern_step(pattern, p), states=[tup(s) for s in irange(0, n)],
                                  fixed=[tup(n)], fixed_values=[0], alpha=1)

f = expected_flips('HTTHT')
[round(float(f(s)), 6) for s in range(6)] # from positions 0 to 5
# [36.0, 34.0, 32.0, 28.0, 18.0, 0.0]
```

```
{pat: round(float(expected_flips(pat)(0)), 6) for pat in ('HH', 'HT', 'HHH', 'HTH', 'THHT', 'HHTTTH')}\n# {'HH': 6.0, 'HT': 4.0, 'HHH': 14.0, 'HTH': 10.0, 'THHT': 18.0, 'HHTTTH': 66.0}\nround(float(expected_flips('HH', 0.4)(0)), 6), (1 + 0.4) / 0.4 ** 2\n# (8.75, 8.749999999999998)\n\ndef wait_summary(pattern, p=0.5, steps=3000, show=8):\n    \"E(N) as the sum over n of P(N > n), the weight still unresolved, and P(N = n) for small n\"\n    step, n = pattern_step(pattern, p), len(pattern)\n    k, total, first = constant(0), 0, {}\n    for flips in irange(1, steps):\n        before = k.kernel(n)\n        total += 1 - before\n        k = step // k\n        if flips <= show:\n            first[flips] = round(k.kernel(n) - before, 6)    # weight that arrives at flip number `flips`\n    return round(total, 6), 1 - k.kernel(n), first\n\nwait_summary('HH')\n# (6.0, 0.0, {1: 0.0, 2: 0.25, 3: 0.125, 4: 0.125, 5: 0.09375, 6: 0.078125, 7: 0.0625, 8: 0.050781})
```

- For H H the weights $P(N = n)$ follow the lecture's recursion $d_n = p^2 \{n = 2\} + (1 - p) d_{n-1} + p(1 - p) d_{n-2}$: $1/4, 1/8, 1/8, 3/32, \dots$ for a fair coin, the same numbers as the fixed point in **3.8**.
- For the whole Kind of N, put a clock in the state as in **3.6**, or use the fixed point of **3.8**. `wait_summary` gives the first branches without either.
- The middle number from `wait_summary`, and the third from `race` below, is the weight that has not resolved within `steps` flips. It should be 0 or negligible. If it is not (a long pattern, or a small p), the answers are too low: raise `steps` until it is.
- The solver needs numpy and scipy. If they are missing, `wait_summary` needs nothing but `frplib`, and three equations can be solved by hand.
- A pattern given by a rule instead of a string (an odd run of heads followed by a tail, practice 5) gets its own small state machine in the same spirit (**4.5**).

Pattern	Fair coin	General p, with $q = 1 - p$	Pattern	Fair coin	General p
H	2	$1/p$	H T H	10	$1/p + 1/(p^2q)$
H H	6	$1/p + 1/p^2$	H T T	8	$1/(pq^2)$
H T	4	$1/(pq)$	T H H T	18	$1/q + 1/(p^2q^2)$
H H H	14	$1/p + 1/p^2 + 1/p^3$	H T T H T	36	$1/(pq) + 1/(p^2q^3)$
H H T	8	$1/(p^2q)$	H H T T T H	66	$1/p + 1/(p^3q^3)$

A check on any of these, not from the course materials. Find every k for which the first k symbols of the pattern equal its last k symbols (the whole pattern always counts). $E(N)$ is the sum of $1 / P(\text{those } k \text{ symbols})$ over those k . H T T H T matches itself at $k = 5$ and at $k = 2$ (H T), so $E(N) = 1/(p^2q^3) + 1/(pq)$, which is $32 + 4 = 36$ for a fair coin. For a fair coin H H matches at $k = 2$ and $k = 1$, giving $4 + 2 = 6$, while H T matches only at $k = 2$, giving 4. Use it to check an answer, and show the first-step equations as your work.

Which of two patterns comes first. The same idea with one more piece of state: the position is the longest recent stretch of flips that is a beginning of either pattern, and both complete patterns are frozen.

```
def race(pat_a, pat_b, p=0.5, steps=400):
    "P(pat_a first), P(pat_b first), weight unresolved (neither pattern may contain the other)"
    p = as_quantity(p)
    prefixes = sorted({pat[:i] for pat in (pat_a, pat_b) for i in range(len(pat) + 1)},
                      key=lambda t: (len(t), t))
    index = {t: i for i, t in enumerate(prefixes)}          # the state is the index of a prefix
    def advance(t, flip):                                # longest prefix that ends t + flip
        return max((u for u in prefixes if (t + flip).endswith(u)), key=len)
    @conditional_kind(codim=1)
    def step(i):
        t = prefixes[int(i)]
        if t in (pat_a, pat_b):                          # one pattern is complete: frozen
            return constant(i)
        return weighted_as(index[advance(t, 'T')], index[advance(t, 'H')], weights=[1 - p, p])
    final = evolve(constant(index['']), step, steps)
    first_a, first_b = final.kernel(index[pat_a]), final.kernel(index[pat_b])
    return round(first_a, 6), round(first_b, 6), round(1 - first_a - first_b, 6)

race('HH', 'TH')
# (0.25, 0.75, 0.0)
race('HH', 'HT')
# (0.5, 0.5, 0.0)
race('HHT', 'HTT')
# (0.666667, 0.333333, 0.0)
```

H H loses to T H three times in four: unless the first two flips are both heads, a tail arrives before any H H, and from then on the first head completes T H. The expected waits do not settle a race. H T waits 4 flips on average and H H waits 6, yet each comes first half the time. H H T and H T T both wait 8, yet H H T comes first two times in three.

4 The Oct 3 practice set, worked

Answers to all nine practice problems, checked against the posted solutions (v1.0.1, 4 Oct). Where the posted solution and an earlier version of this guide took different routes, the posted route now comes first and the other is labeled. Hand arithmetic, closed forms and probabilities that a problem did not ask for are this guide's additions, and each was checked by code. The numbers in the green boxes come from the code shown, which ran in frplib 0.3.10 when this file was built.

4.1 Problem 1: short answers

```
psi = Fork(Proj[2] - Proj[1], Proj[2] + Proj[1])          # <x, y> -> <y - x, y + x>
phi = Fork((Proj[2] - Proj[1]) / 2, (Proj[2] + Proj[1]) / 2) # <u, v> -> <(v - u)/2, (v + u)/2>
phi(psi(3, 7))
# <3, 7>
psi(phi(3, 7))
# <3, 7>
uniform(-1, 1) ^ (__ ** 2)
# <> ----- 1 ---- 1
```

ANSWER (a) ψ sends (x, y) to $(u, v) = (y - x, y + x)$. Solving, $x = (v - u)/2$ and $y = (v + u)/2$, which is the `phi` on the second line of the code.

(b) X and Y are independent if observing the value of one does not change our predictions about the other: $\text{kind}(Y \mid X = u) = \text{kind}(Y)$ for every possible value u of X. Alternatively, we can analyze the whole system by combining separate analyses of the parts: $\text{kind}(X :: Y) = \text{kind}(X) * \text{kind}(Y)$.

(c) `constant(1)`, because squaring maps both -1 and 1 to 1 , which combines the two branches of the original Kind.

(d) False. It does not return a Kind: neither the value `c` nor the dictionary of Kinds is a Kind.

(e) The new FRP gets the value from the original FRP, applies the statistic, and the result is its value. It is activated when the original is activated, and vice versa. (On Kinds: transform every leaf, then merge leaves with equal values, adding weights.)

(f) The branches that are inconsistent with our observation: those whose value makes the condition false.

4.2 Problem 2: short answers

```
Fork(Count, Min, Mean, Max)(3, 1, 4, 1, 5)
# <5, 1, 2.8, 5>
(10 * __ ** 2 - 2)(1)
# <8>
f = 3 * And(__ > -10, __ < 2) + 2 * (__ > 0) - 10 * (__ < 1) - (__ == 4)
f(1)
# <5>
k = weighted_as((5, -2), (2, -3), (-1, -4), (8, -5), (1, 1),
                weights=[1/3, 1/6, 1/8, 1/4, 1/8])
phi = ((Proj[1] + Proj[2] ** 2) % 3 == 0)
k ^ phi
#      ,---- 7/24 ----- 0
# <> -|
#      `---- 17/24 ---- 1
k | phi
#      ,---- 3/17 ---- <-1,-4>
# <> -+---- 8/17 ---- <5,-2>
#      `---- 6/17 ---- <8,-5>
```

ANSWER (a) Transforming with Statistics, Building with Joins, Updating with Observations, Predicting with Expectations.

(Lectures 6 to 10 and the older text called the third one Constraining with Observations. The posted solution, the current text and the cheatsheet say Updating.)

(b) A statistic is a function that maps values to values. A conditional Kind maps values to Kinds. A conditional FRP maps values to FRPs.

(c) psi takes a value, computes the number, minimum, average and maximum of its components, and returns a 4-tuple whose components are those four numbers in that order.

(d) 8, then $\sin(\pi/6) = 1/2$, then $f(1) = 3 + 2 - 0 - 0 = 5$.

(e) The Data FRP represents all the relevant quantities that we collect or observe as we watch a random system evolve. We transform it in various ways to address our questions (Feature FRPs), and we build it from simpler pieces (Element FRPs) whose structure and interaction are defined in our model.

(f) $k \wedge \text{phi}$ is the transform of the Kind k by the compatible statistic phi : the Kind of an FRP produced by passing a value from an FRP with Kind k through the statistic. Because phi is a condition, the transformed FRP is an event. $k \mid \text{phi}$ updates k with an observation, embodied by the condition phi , erasing all branches of k that are inconsistent with it. We use it when we have observed that the condition holds, in reality or hypothetically.

(f), the numbers (not in the posted solution). ϕ is true on $\{5, -2\}$, $\{-1, -4\}$ and $\{8, -5\}$ (9, 15 and 33 are multiples of 3). $k \wedge \text{phi}$ has weight 7/24 on 0 and 17/24 on 1. $k \mid \text{phi}$ keeps those three values with weights 8/17, 3/17 and 6/17.

4.3 Problem 3: unbalanced rolls

The full Kind $\text{roll_kind} \mathrel{\mathbf{**}} 10$ has $6^{10} = 60,466,176$ branches, so reason about which branches survive each observation and build only those. **7.3** works through other observations on the same rolls.

```
roll_kind = weighted_as(1, 2, ..., 6, weights=['1/2', '1/4', '1/8', '1/16', '1/32', '1/32'])
n = 10
# (b) min == max: only the six branches <v, v, ..., v> survive, with weight kernel(v) ** n
same = weighted_as([tuple([v] * n) for v in irange(1, 6)],
                  weights=[roll_kind.kernel(v) ** n for v in irange(1, 6)])
same
#      ,---- 0.99902 ----- <1, 1, 1, 1, 1, 1, 1, 1, 1, 1>
#      |---- 0.00097561 ---- <2, 2, 2, 2, 2, 2, 2, 2, 2, 2>
#      |---- 9.5274E-7 ----- <3, 3, 3, 3, 3, 3, 3, 3, 3, 3>
# <> -|
#      |---- 9.3041E-10 ---- <4, 4, 4, 4, 4, 4, 4, 4, 4, 4>
#      |---- 9.0861E-13 ---- <5, 5, 5, 5, 5, 5, 5, 5, 5, 5>
#      `---- 9.0861E-13 ---- <6, 6, 6, 6, 6, 6, 6, 6, 6, 6>
Kind.equal(fast_join_pow(Fork(Min, Max), roll_kind, n) | (Proj[1] == Proj[2]),
          same ^ Fork(Min, Max))
# True
# (c) min == 5: every roll is a 5 or a 6, and at least one is a 5
min5 = (roll_kind | (__ >= 5)) ** n | (Min == 5)
```

```
size(min5), len(set(min5.weights.values())), min5.kernel((5,) * n), 1 / 1023
# (1023, 1, 0.0009775171065493646, 0.0009775171065493646)
# the shortcut agrees with the direct computation on a power small enough to run
Kind.equal(roll_kind ** 4 | (Min == 5), (roll_kind | (__ >= 5)) ** 4 | (Min == 5))
# True
```

ANSWER (a) `roll_kind ** 10`. (Write the expression. Do not try to print it.)

(b) `roll_kind ** 10 | (Min == Max)`. The observation means all ten rolls show the same value, so six branches remain, $(v, v, \dots, v)$ for $v = 1$ to 6. Unnormalized weights: $2^{-10}, 4^{-10}, 8^{-10}, 16^{-10}, 32^{-10}, 32^{-10}$. Canonical: each of those divided by their sum, that is $j^{-10} / (2^{-10} + 4^{-10} + 8^{-10} + 16^{-10} + 32^{-10} + 32^{-10})$ for $j = 2, 4, 8, 16, 32, 32$. In decimals: 0.99902, 0.00097561, 9.5274e-7, 9.3041e-10, 9.0861e-13, 9.0861e-13.

(c) `roll_kind ** 10 | (Min == 5)`. The observation means every roll is a 5 or a 6. The values are all the 10-tuples of 5s and 6s with at least one 5: $2^{10} - 1 = 1023$ of them. Every such value had weight 32^{-10} in the original Kind, so the surviving weights are all equal: $1/1023$ each in canonical form.

4.4 Problem 4: Dr. Jones

ANSWER (a) Keep track of whether the treasure has been found and which cells have been searched: $(t, x_1, y_1, x_2, y_2, \dots)$, where $t = 1$ if the treasure has been found and 0 otherwise, and $(x_1, y_1), (x_2, y_2), \dots$ are the cells searched, in order.

(b) The initial state is $(0, 2, 2)$, since $(2, 2)$ was searched unsuccessfully, and at most 8 more searches are needed: `state = evolve(constant(0, 2, 2), next_state, 8)`. The same thing as a loop: start with `state = constant(0, 2, 2)` and repeat `state = next_state // state` eight times. When the treasure is found early, `next_state` forwards that state along unchanged.

(c) The number of cells searched is the length of the tuple minus 1 (for the treasure flag), divided by 2. If that is 6 or more Belloc gets the jewel, otherwise Jones does: `museum = And(Proj[1] == 1, (Count - 1) // 2 < 6)`.

(d) The current state is $(0, 2, 2, 2, 1, 1, 1)$. From $(1, 1)$ the unsearched cells and their weights $1/\text{distance}^2$ are $(1, 2)$: 1, $(1, 3)$: $1/4$, $(2, 3)$: $1/5$, $(3, 1)$: $1/4$, $(3, 2)$: $1/5$, $(3, 3)$: $1/8$. They sum to $81/40$, so the canonical weights are $40/81, 10/81, 8/81, 10/81, 8/81, 5/81$. The Kind's values are the seven-component state followed by the new cell, with $t = 1$ only on the $(3, 1)$ branch, as printed below.

To check the numbers, here is a `next_state` in the posted layout. The problem only asks you to use `next_state`, not to write it.

```
JEWEL = (3, 1)
CELLS = [(i, j) for i in irange(1, 3) for j in irange(1, 3)]

@conditional_kind
def next_state(state):
    # <t, x1, y1, x2, y2, ...>
    found, rest = state[0], tuple(state[1:])
    if found == 1:
        # found: record the jewel cell again (see below)
        return constant((1,) + rest + JEWEL)
    searched = [(rest[m], rest[m + 1]) for m in range(0, len(rest), 2)]
    current = searched[-1]
    options = [c for c in CELLS if c not in searched]
    weights = [as_quantity(1) / ((c[0] - current[0]) ** 2 + (c[1] - current[1]) ** 2)
               for c in options]
    return weighted_as([(1 if c == JEWEL else 0,) + rest + c for c in options], weights=weights)

next_state(0, 2, 2, 2, 1, 1, 1) # (d)
# ,---- 0.49383 ---- <0, 2, 2, 2, 1, 1, 1, 2>
# |---- 0.12346 ---- <0, 2, 2, 2, 1, 1, 1, 3>
# |---- 0.098765 ---- <0, 2, 2, 2, 1, 1, 1, 2, 3>
# <> -|
# |---- 0.098765 ---- <0, 2, 2, 2, 1, 1, 1, 3, 2>
# |---- 0.061728 ---- <0, 2, 2, 2, 1, 1, 1, 3, 3>
# `---- 0.12346 ---- <1, 2, 2, 2, 1, 1, 1, 3, 1>
final = evolve(constant(0, 2, 2), next_state, 8) # (b)
dim(final), size(final)
# (19, 13700)
museum = And(Proj[10] == 3, Proj[11] == 1) # padded state: the fifth cell is the jewel cell
E(final ^ museum)
# 0.4456873103802542
```

```
@conditional_kind
def literal(state):
    # as the problem words it: unchanged once found
    return constant(state) if state[0] == 1 else next_state(state)

evolve(constant(0, 2, 2), literal, 8)
# Value set elements have different dimensions, {5, 7}.
```

WATCH OUT The last line is why the code pads. A `next_state` that leaves a found state unchanged gives states of different lengths after two steps: found states stop at dimension 5 while the others grow to 7. `frplib 0.3.10` builds that `Kind`, but `//` and `>>` will not take it as input, so the third step of the `evolve` fails. The padded version repeats the jewel cell so every state has the same length, and then the posted `(Count - 1) // 2 < 6` no longer tells the cases apart, so `museum` reads the fifth cell instead. On paper, give the posted answers in the box above.

Probability the jewel reaches a museum (not asked for, and not in the posted solution): 0.4457. That is the probability that the jewel is among the first five cells searched, counting (2, 2) as the first. If the sixth cell also counted it would be 0.5672.

4.5 Problem 5: odd heads run

The posted solution gives three approaches, and all three agree. The hint is the key to each: after a tail, or after an even run of heads, the future looks exactly like the beginning. How to find these states and prefixes yourself, and how to check the code, is in 4.11.

Approach 1: one flip at a time. The posted state is (n, h, d): flips so far, heads in the current run, and d = 1 when done. Only whether h is even or odd matters, so the version below keeps a phase instead: 0 for an even run (which includes the start and any moment just after a tail), 1 for an odd run, 2 for done. The smaller state keeps the `Kind` small.

```
def odd_run_kind(p, n_steps=200):
    flip = binary(p)
    @conditional_kind(codim=2)
    def next_state(phase, n):
        if phase == 2:
            return constant(phase, n)
        if phase == 0:
            return flip ^ Fork(__, n + 1)
        return flip ^ Fork(IfThenElse(__ == 1, 0, 2), n + 1)
    state = evolve(constant(0, 0), next_state, n_steps)
    return clean((state | (Proj[1] == 2)) ^ Proj[2], 1e-12)

    # 1 is heads, with weight p
    # done: frozen
    # H -> odd (1), T -> still even (0)
    # H -> even, T -> done
    # initial state <0, 0>
```

```
k_half, k_34 = odd_run_kind('1/2'), odd_run_kind(0.75)
[round(k_half.kernel(n), 6) for n in irange(2, 11)]
# [0.25, 0.125, 0.125, 0.09375, 0.078125, 0.0625, 0.050781, 0.041016, 0.033203, 0.026855]
[round(k_34.kernel(n), 6) for n in irange(2, 11)]
# [0.1875, 0.046875, 0.117188, 0.055664, 0.079834, 0.05127, 0.057724, 0.04327, 0.043287, 0.035161]
E(k_half)
# 5.999999999442845
E(k_34)
# 9.333333327660402
```

Approach 2: one prefix at a time, run until resolved. From the start, T costs one flip and resets, H H costs two flips and resets, and H T finishes in two. The state is just (n, d) . The posted run keeps stepping until the weight on "not done" is below a tolerance, which saves guessing a number of steps.

```
def odd_heads_system_alt(p=0.5):
    initial_state = constant(0, 0) # <flips so far, done>
    @conditional_kind(codim=2, target_dim=2)
    def next_state(state): # one parameter: it receives the whole state
        n_flips, done = state
        if done == 1:
            return constant(state)
        return weighted_as((n_flips + 1, 0), (n_flips + 2, 0), (n_flips + 2, 1),
                           weights=[1 - p, p * p, p * (1 - p)]) # T, H H, H T
    def run(tolerance=1e-7):
        curr = initial_state
        while (curr ^ Proj[2]).kernel(0) > tolerance: # weight still unresolved
            curr = next_state // curr
        return curr
    return (initial_state, next_state, run)

oha_init75, oha_next75, oha_run75 = odd_heads_system_alt(3/4)
final75 = oha_run75()
E(final75 ^ Proj[1]) # expected flips for p = 3/4
# 9.333332469838766
```

Approach 3: a fixed point (3.8), exact up to the size of the tree. The Kind of the total is unchanged by "take a prefix, then finish or start over":

```
def odd_heads_exact(remaining_flips, p=symbol('p')):
    p = as_quantity(p)
    # T, H H or H T, labeled 0, 3 and 2. The first two reset the state, the third is done.
    initial_flips = weighted_as(0, 3, 2, weights=[1 - p, p * p, p * (1 - p)])
    flips_given_initial = conditional_kind({
        0: remaining_flips ^ (__ + 1),
        2: constant(2),
        3: remaining_flips ^ (__ + 2),
    })
    return flips_given_initial // initial_flips

xk25 = iterate(odd_heads_exact, 100, constant(2), p='1/4')
xk50 = iterate(odd_heads_exact, 100, constant(2), p='1/2')
xk75 = iterate(odd_heads_exact, 100, constant(2), p='3/4')
E(xk50)
# 6
E(xk75)
# 9.33333326292268
```

(b) The friend's six counts are independent repetitions with one coin, so the likelihood of each candidate p is the product of that Kind's weights at the six observed values. Compare log-likelihoods:

```
observed_runs = (10, 8, 15, 9, 7, 12) # a tuple
log_like = ijoin_log_likelihood(observed_runs)
log_like(xk25)
# -19.955153961555197
log_like(xk50)
# -20.645822595495556
log_like(xk75)
# -19.317641877593893
ijoin_log_likelihood([10, 8, 15, 9, 7, 12]) # a list, as the posted solution types it
# as_quantity: could not convert [10, 8, 15, 9, 7, 12] to a quantity
```

WATCH OUT In frplib 0.3.10 `ijoin_log_likelihood` takes the observations as a tuple or as separate arguments. The posted solution passes a list, which fails with the message above, and it shows the results as `Decimal(...)` where 0.3.10 returns plain floats. The numbers are the same.

ANSWER (a) First ten branches (values 2 to 11). For $p = 1/2$: 0.2500, 0.1250, 0.1250, 0.09375, 0.07813, 0.06250, 0.05078, 0.04102, 0.03320, 0.02686. For $p = 0.75$: 0.1875, 0.04688, 0.1172, 0.05566, 0.07983, 0.05127, 0.05772, 0.04327, 0.04329, 0.03516.

Expectations: 6 for $p = 1/2$ and $28/3 \approx 9.333$ for $p = 0.75$, from all three approaches. A first-step equation, which the posted solution does not use, gives the same: $E = q(1 + E) + pq \cdot 2 + p^2(2 + E)$ with $q = 1 - p$, so $E = (1 + p) / (p(1 - p))$.

(b) The log-likelihoods are -19.955 for $p = 1/4$, -20.646 for $p = 1/2$ and -19.318 for $p = 3/4$, so the maximum likelihood estimate is $p = 3/4$. Both $1/4$ and $3/4$ favor long waits, so both beat $1/2$ on this rather extreme list of counts.

4.6 Problem 6: double or nothing

```
@conditional_kind
def bet_again(bets):
    if bets[-1] == 0:
        return constant(0)
    return choice(0, 1)

kind_N = choice(0, 1) >> bet_again >> bet_again ^ Sum
kind_N
#      ,---- 4/8 ---- 0
#      |---- 2/8 ---- 1
# <> -|
#      |---- 1/8 ---- 2
#      `---- 1/8 ---- 3
winnings = IfThenElse(__ < 3, -1000, 7000)
winnings(kind_N)
#      ,---- 7/8 ---- -1000
# <> -|
#      `---- 1/8 ---- 7000
E(winnings(kind_N))
# 0
```

ANSWER (a) The Kind of N is the transform of a join that represents the outcomes of all the bets (the code above). N is 0, 1, 2, 3 with weights $1/2, 1/4, 1/8, 1/8$: she stops at the first loss, and $N = 3$ means three wins. `frplib` prints the weights over a common denominator, $4/8, 2/8, 1/8, 1/8$.

(b) The statistic returns -1000 if she wins fewer than 3 bets, because she loses all her money, and otherwise $1000 \cdot (2^3 - 1) = 7000$: she doubles her money three times and we subtract her initial stake. `winnings = IfThenElse(__ < 3, -1000, 7000)`, or a `@statistic(dim=1)` function with the same two cases.

(c) $E(\text{winnings}(\text{kind_N}))$ is 0. As a weighted average: $-1000 \cdot 7/8 + 7000 \cdot 1/8 = 0$. The game is fair.

4.7 Problem 7: customer purchases

```
kind_N = geometric(0, 1, ..., 25, r=0.9)
purchase = weighted_as(0, 1, 2, weights=['3/4', '1/6', '1/12'])

@conditional_frp(codim=1)
def T_given_N(n):
    return Sum(frp(purchase) ** n)

N = frp(kind_N)
T = N >> T_given_N ^ Proj[2]
N.value
# <2>
T.value
# <0>
FRP.sample(10, T, summary=False)
# [(2,), (2,), (1,), (2,), (3,), (1,), (3,), (3,), (6,), (2,)]

@conditional_kind(codim=1)
def t_given_n(n):
    return fast_join_pow(Sum, purchase, n)

t = kind_N >> t_given_n ^ Proj[2]
```

```

size(t)
# 51
E(t)                                     # (c)
# 2.401361014528185
E(t | (__ >= 5))                         # (d): at least 5 items were bought
# 6.941050745742581
nt = kind_N >> t_given_n                 # the Kind of <N, T>
E(Proj[2](nt | And(Proj[1] >= 5, Proj[2] >= 5))) # (d): at least 5 customers and at least 5 items
# 6.953528174694529

```

ANSWER (a) The code above. `FRP.sample(10000, T)` prints a table of counts instead. Your values will differ from run to run.

(b) Given $N = n$, T has the Kind of `Sum(purchase ** n)`, which is `fast_join_pow(Sum, purchase, n)`. Then `t = kind_N >> t_given_n ^ Proj[2]`. It has 51 values, 0 through 50.

(c) `E(t)` gives $E(T) \approx 2.4$ (2.4014, which is $E(N) \cdot E(\text{purchase}) = 7.2041 \cdot 1/3$).

(d) The observation tells us at least 5 items will be bought today, since we have seen 5 purchases. Updating with that alone: `E(t | (__ >= 5))` gives $E(T | T \geq 5) \approx 6.94$. The five purchases came from five separate customers, so we also know $N \geq 5$. Updating with both gives $E(T | T \geq 5 \text{ and } N \geq 5) \approx 6.95$. The posted solution: the latter is more accurate, but either is acceptable.

A third reading, not in the posted solution. The update above uses "at least 5 customers and at least 5 items". Suppose instead that the five you watched are five particular customers, say the first five of the day, and that each bought exactly one item. Purchases are independent of N and of each other, so the factor $(1/6)^5$ for those five purchases is the same on every surviving branch and cancels. What is left: N given $N \geq 5$, and T is 5 plus the total over the other $N - 5$ customers.

```

n_obs = kind_N | (__ >= 5)

@conditional_kind(codim=1)
def t_given_n_obs(n):
    return fast_join_pow(Sum, purchase, n - 5) ^ (__ + 5)

E(n_obs)
# 11.41988797906104
E(t_given_n_obs // n_obs)
# 7.139962659687012

```

That gives 7.14, which is $5 + (11.42 - 5)/3$. Different readings of one sentence give different numbers, so on the exam write down which observation you are updating with, as the posted solution does, before you compute.

4.8 Problem 8: pricing the payoff

```

d6 = uniform(1, 2, ..., 6)
# (i) the three rolls, (ii) which rolls are 4s, (iii) -1 if there are none, else the number of 4s
kind_B_4 = d6 ** 3 ^ ForEach(__ == 4) ^ IfThenElse(Sum > 0, Sum, -1)
kind_B_4
#      ,---- 0.57870 ----- -1
#      |---- 0.34722 ----- 1
# <> -|
#      |---- 0.069444 ----- 2
#      `---- 0.0046296 ---- 3
E(kind_B_4)
# -0.0787037037037037
payoff = lambda n: ForEach(__ == n) ^ IfThenElse(Sum > 0, Sum, -1)
all(Kind.equal(d6 ** 3 ^ payoff(n), kind_B_4) for n in irange(1, 6)) # the same Kind for every n
# True
E(conditional_kind(lambda n: d6 ** 3 ^ payoff(n), codim=1) // arbitrary(1, 2, ..., 6))
# -0.0787037037037037
get4 = d6 ^ (__ == 4) # one roll: is it a 4?
E(get4 ** 3 ^ (Sum - Product(1 - __))) # the count of 4s, minus 1 when there are none
# -0.0787037037037037

```

ANSWER The number of 4s among three dice has weights $125/216$, $75/216$, $15/216$, $1/216$ on 0, 1, 2, 3. So $E(B_4) = (-125 + 75 + 30 + 3)/216 = -17/216 = -0.0787037$. The risk-neutral price is negative, so the bet favors the casino, by about 7.9 cents per dollar.

The expectation does not depend on n . Intuitively this follows from the symmetry of the balanced die: relabeling the faces leaves the Kind of the three rolls unchanged. The code checks it for each n , and then with n chosen by a Kind with arbitrary weights: no symbol is left in the answer, so it is the same however n is chosen.

`Product(1 - __)` of the three 0/1 flags is 1 exactly when no roll is a 4, so `Sum - Product(1 - __)` is the payoff in one statistic.

4.9 Problem 9: asymmetric binary channel

Two stages: the input bit is generated, then it is sent through the noisy channel. p_0 and p_1 are the probabilities that a 0 and a 1 are flipped.

```
def asym_binary_channel(r=symbol('r'), p0=symbol('p0'), p1=symbol('p1')):
    ibit = weighted_as(0, 1, weights=[1 - r, r])          # the input bit I
    @conditional_kind(codim=1)
    def transmission(input_bit):
        if input_bit == 0:
            return weighted_as(0, 1, weights=[1 - p0, p0]) # a 0 arrives as 1 with probability p0
        return weighted_as(0, 1, weights=[p1, 1 - p1])     # a 1 arrives as 0 with probability p1
    return ibit >> transmission                          # the Kind of <I, U>

i_u = asym_binary_channel()
kind_U = i_u ^ Proj[2]                                  # (a)
kind_U
# ,---- 1 + -1 p0 + p0 r + p1 r + -1 r ---- 0
# <> -|
# \---- p0 + -1 p0 r + -1 p1 r + r ----- 1
I_given_U = conditional_kind({                           # (b): update on U, project onto I
    0: (i_u | (Proj[2] == 0)) ^ Proj[1],
    1: (i_u | (Proj[2] == 1)) ^ Proj[1],
})
I_given_U
# A conditional Kind of type 1 -> 2 with wiring:
# ,---- (1 + -1 p0 + p0 r + -1 r)/(1 + -1 p0 + p0 r + p1 r + -1 r) ---- 0
# <0>: <> -|
# \---- p1 r/(1 + -1 p0 + p0 r + p1 r + -1 r) ----- 1
#
# ,---- (p0 + -1 p0 r)/(p0 + -1 p0 r + -1 p1 r + r) ---- 0
# <1>: <> -|
# \---- (-1 p1 r + r)/(p0 + -1 p0 r + -1 p1 r + r) ----- 1
substitution(I_given_U(1), r=0.5, p0=0.1, p1=0.2)        # a numeric spot check
# ,---- 1/9 ---- 0
# <> -|
# \---- 8/9 ---- 1
```

ANSWER (a) `kind_U = asym_binary_channel() ^ Proj[2]`. Simplified, U is 0 with weight $(1 - p_0) + (p_0 + p_1 - 1)r$ and 1 with weight $p_0 - (p_0 + p_1 - 1)r$. The same thing grouped by input: U is 1 with probability $(1 - r)p_0 + r(1 - p_1)$.

(b) The conditional Kind above. Given U = 1: I is 1 with probability $r(1 - p_1) / [(1 - r)p_0 + r(1 - p_1)]$. Given U = 0: I is 1 with probability $r p_1 / [(1 - r)(1 - p_0) + r p_1]$. Each target is also `bayes(u, ibit, transmission)` if you keep the two pieces separate.

4.10 The set in 90 minutes, and how to adapt each answer

The timed plan. Three minutes to skim all nine problems and write a search word next to each, 72 minutes of work in the order below, and 15 minutes in reserve. Each search word lands in the right section at its first hit. The practice set says it holds more problems than a real exam will, so this is a stress test, not the real pace.

No.	Problem	Min	Ctrl-F	Section	What to do
1	1. Short Answers 1	6	<code>uniform(-1, 1)</code>	4.1	Copy (a) to (f) from the green box.
2	2. Short Answers	7	<code>maps values</code>	4.2	Copy the box. (d) is 8, 1/2, 5.
3	6. Double or Nothing	6	<code>bet_again</code>	4.6	Paste the block. Write the Kind of N, the statistic, and $E(W) = -1000 \cdot 7/8 + 7000 \cdot 1/8 = 0$.
4	8. Pricing the Payoff	6	<code>kind_B_4</code>	4.8	Paste three lines. $-17/216 \approx -0.0787$, so it favors the casino, and it is the same for every n.
5	3. Unbalanced Rolls	7	<code>Min == Max</code>	4.3	Run nothing. Write the three expressions and the weights from the box.
6	9. Asymmetric Binary Channel	8	<code>I_given_U</code>	4.9	Paste the function and the two lines after it. Write the simplified weights from the box.
7	7. Customer Purchases	10	<code>T_given_N</code>	4.7	Paste both blocks. $E(T) \approx 2.4$, updated 6.94 or 6.95. Say which observation you update with.
8	4. Dr. Jones	9	<code>Belloc</code>	4.4	All four answers are in the box. Do (d) by hand: weights 1, 1/4, 1/5, 1/4, 1/5, 1/8 add to 81/40.
9	5. Odd Heads Run	13	<code>odd_heads_exact</code>	4.5	Paste the fixed-point block and the three <code>iterate</code> lines (about 4 seconds). First ten branches: <code>[round(xk50.kernel(n), 6) for n in irange(2, 11)]</code> , then the same with <code>xk75</code> . Then $E(xk50)$, $E(xk75)$ and the log-likelihood lines. The estimate is $p = 3/4$.

- **Order.** The first four are short and certain, so four problems are done 25 minutes in. The two longest go last, where an overrun cannot cost you the easy ones.
- **Hard cap.** When a problem's minutes are up, write the `frplib` expression you would run and one sentence of reasoning, then move on. On a real exam, share the working minutes out in proportion to the points printed on each problem.
- **Paste, do not retype.** Paste the block, then enter the one line whose result you want.
- **On paper.** The expression, the result, and one sentence of why. That is what the posted solutions contain. If the wording allows two readings, say which one you used.
- **Never print a huge Kind.** `roll_kind ** 10` has 60 million branches. Write it, do not run it.

When the problem is new. The search words above only find these nine problems. What carries over is the routine, with **7.1** for the long version:

1. Read the last sentence first and underline what is asked for: a Kind, an expectation or price, a probability, a statistic, an explanation, or an estimate.
2. Name the stages in order: what is random first, and what depends on it. One thing repeated independently is `**`. A step repeated on a state is `evolve`. Two different stages are a conditional Kind and `>>`. Something you are told happened is `|`.
3. Find the wording in the table on the first page and open that section, or search for a rare word from the problem.
4. Paste the template and change only its knobs: the Kinds, the number of steps, the statistic, the observation. The table below lists the knobs for each practice problem.
5. If the Kind could be large, run a small case first: fewer rolls, fewer steps.
6. Check before writing (**6.1**): the weights add to 1, the answer lies between the smallest and largest possible value, and one number agrees with a hand calculation.

Problem type	What a variant changes	What you edit	Check by hand
1, 2. Short answers (1.1 to 1.14 , 2.2)	The term to define, or the statistic or Kind to evaluate or describe.	Search the term in section 1. Type the expression into the playground, read the result, and say why in one sentence. An inverse: solve for the inputs by hand, then test <code>phi(psi(3, 7))</code> .	Recompute one value by hand.

Problem type	What a variant changes	What you edit	Check by hand
3. An observation on a big independent join (4.3 , 7.3 , 3.3)	The observation (the maximum is 2, the sum is 12, all rolls differ), the number of rolls, the weights.	Write <code>k ** n (condition)</code> and do not run it. Work out which values survive. Each keeps the product of its per-roll weights, divided by the total over the survivors. To verify, restrict each roll first or shrink <code>n</code> .	The number of survivors, and that their weights add to 1.
4. A search that carries a state (4.4 , 3.6)	The grid, the start, the target, the rule for the weights, the deadline.	<code>State</code> = a flag plus whatever the rule needs. <code>evolve(constant(start), next_state, most_steps_needed)</code> . A final statistic reads the flag and the count. One step by hand: list the options, their raw weights, the sum, then divide.	The weights of one step add to 1.
5. Waiting for a rule on coin flips, then an estimate of <code>p</code> (4.5 , 4.11 , 3.8 , 3.18 , 3.11)	The stopping rule, <code>p</code> , the observed counts, the candidate values of <code>p</code> .	A fixed string of flips: <code>expected_flips</code> in 3.18 . A rule with one way back to the start: the three prefix lines of <code>odd_heads_exact</code> (flips used, weight, start over or done). A rule that needs memory: phases, as in approach 1 of 4.5 . Then <code>ijoin_log_likelihood(tuple)</code> for each candidate, and the largest wins.	A first-step equation for <code>E</code> (5.6).
6. Bets in a row, stopping at a loss (4.6 , 3.4)	The number of bets, the chance of winning one, the stake rule.	The number of <code>>> bet_again</code> , the Kind of one bet, the two cases of <code>IfThenElse</code> . See <code>wins_kind</code> below.	The weighted average.
7. A random number of random amounts (4.7 , 3.7)	The Kind of the count, the Kind of one amount, the question about the total, the observation.	<code>kind_N</code> and <code>purchase</code> . Then <code>E(t), t.kernel(v)</code> or <code>E(t ^ condition)</code> . For an observation on the count and the total together, keep the join <code>nt</code> and update that.	$E(T) = E(N) \cdot E(\text{one amount})$.
8. The price of a bet (4.8 , 3.10)	The number of dice, what counts as a hit, the payoffs.	<code>d6 ** n</code> , the test inside <code>ForEach</code> , and the map from the count to money, with <code>IfThenElse</code> or <code>Cases</code> .	Binomial weights (5.2). The sign of the price says who is favored.
9. Two stages with letters for weights (4.9 , 3.5 , 3.13)	The Kind of the input, the noise, letters or numbers, which direction is asked.	The weights in <code>ibit</code> and <code>transmission</code> . The output is <code>joint ^ Proj[2]</code> . The backward question is <code>(joint (Proj[2] == u)) ^ Proj[1]</code> .	A Bayes table (5.4).

Five variants, worked. Each reuses names from the sections above and changes only the knobs:

```
# 3, a new observation: the maximum is 2. Every roll is a 1 or a 2, and at least one is a 2
max2 = (roll_kind | (__ <= 2)) ** 10 | (Max == 2)           # restrict each roll first, then update
size(max2), max2.kernel((2,) + (1,) * 9)                 # survivors, and the weight on <2, 1, 1, ..., 1>
# (1023, 0.00882378285221887)

# 6, any number of bets and any chance of winning each one
def wins_kind(n_bets, p='1/2'):
    @conditional_kind
    def bet_again(bets):
        return constant(0) if bets[-1] == 0 else binary(p)
    k = binary(p)
    for _ in range(n_bets - 1):
        k = k >> bet_again
    return k ^ Sum

wins_kind(3)                                             # the practice problem again
# 0: 4/8  1: 2/8  2: 1/8  3: 1/8
E(wins_kind(5, 0.6) ^ IfThenElse(__ < 5, -1000, 1000 * (2 ** 5 - 1))) # five bets, win chance 0.6
# 1488.32

# 7, other questions about the same total, and an observation on the customers alone
t.kernel(0), as_float(E(t ^ (__ >= 10)))                # nothing is sold; at least ten items are sold
# (0.32893376718636363, 0.02299821524330083)
E(t_given_n // (kind_N | (__ >= 5)))                    # the expected total, given at least five customers
# 3.806629326353679

# 8, other payoffs: count the hits first, then map the count to money
count4 = d6 ** 3 ^ ForEach(__ == 4) ^ Sum
```

```
E(count4 ^ Cases({0: -1, 1: 1, 2: 2, 3: 10})) # three 4s now pay 10
# -0.0462962962962963

# 9, numbers in place of the letters, and the backward question for one output
sent = asym_binary_channel(0.3, 0.1, 0.2) # r, p0, p1
(sent | (Proj[2] == 1)) ^ Proj[1] # the input bit, given that a 1 arrived
# 0: 7/31 1: 24/31
```

- **3.** The survivors are the sequences of 1s and 2s other than all 1s: $2^{10} - 1 = 1023$. After the restriction a 1 has weight $2/3$ and a 2 has weight $1/3$, so a value with a ones and b twos has canonical weight $(2/3)^a (1/3)^b / (1 - (2/3)^{10})$. This time the weights are not all equal.
- **6.** Stopping at the first loss, $N = j$ has weight $p^j(1 - p)$ for j below the number of bets n , and p^n for $j = n$. With a stake of 1000 that doubles, $E(W) = 1000 \cdot ((2p)^n - 1)$: zero for a fair coin, and $1000 \cdot (1.2^5 - 1) = 1488.32$ here.
- **7.** Given $N \geq 5$ the expected total is $E(N | N \geq 5) \cdot E(\text{purchase}) = 11.42/3$.
- **8.** $(-125 + 75 + 30 + 10)/216 = -10/216$. The same three weights as before, with a new payoff on the last one.
- **9.** $0.3 \cdot 0.8 / (0.7 \cdot 0.1 + 0.3 \cdot 0.8) = 24/31$.

4.11 Until problems: two routes, three checks, what each slip looks like

The signal is the word **until** with no fixed number of steps: flip until, roll until, keep going until. Worked here on the odd heads run of **4.5**: flip until a run of an odd number of heads is followed by a tail. Route 1 needs no insight and always works, so it is the default. Route 2 is the one the problem's hint points at. Both must give the same answer, which is a free check.

Route 1: one flip at a time. Ask what you must remember to know what the next flip does. It is not always the last flip: after H a tail ends the game, after H H it does not. Here it is whether the current run of heads is odd. Write the moves as a table with a row for every kind of state, done included, because the done row is what reminds you of the freeze:

State	Tail, weight $1 - p$	Head, weight p
even run, odd = 0 (the start, or just after a tail)	stays even	becomes odd
odd run, odd = 1	done	becomes even
done	unchanged	unchanged

Then type it in a fixed order: the decorator, the `def`, the freeze line, and one `weighted_as` per row with the tail tuple first, because `weights=[1 - p, p]` pairs the first tuple with the tail and the second with the head. Run the three checks with a p that is not $1/2$. At $p = 1/2$ heads and tails weigh the same, so a swap cannot show.

```
p = 3/4 # a lopsided p, so that a swap shows

@conditional_kind(codim=3)
def next_state(flips, odd, done): # flips so far, is the run of heads odd, is the game over
    if done == 1:
        return constant(flips, odd, done) # the freeze
    if odd == 0:
        return weighted_as((flips + 1, 0, 0), (flips + 1, 1, 0), weights=[1 - p, p]) # tail, head
    return weighted_as((flips + 1, 0, 1), (flips + 1, 0, 0), weights=[1 - p, p]) # tail, head

next_state(4, 0, 0) # check 1, an even state: the head (3/4) must make the run odd
# <5, 0, 0>: 1/4 <5, 1, 0>: 3/4
next_state(4, 1, 0) # an odd state: the tail (1/4) must end the game
# <5, 0, 0>: 3/4 <5, 0, 1>: 1/4
next_state(4, 0, 1) # a done state: must come back unchanged with weight 1
# <4, 0, 1>: 1
final = evolve(constant(0, 0, 0), next_state, 200)
E(final ^ (Proj[3] == 0)) # check 2, the weight not yet done: should be tiny
# 0
E(final ^ And(Proj[1] == 2, Proj[3] == 1)) # check 3, done at 2 flips is H T: p(1 - p) = 3/16
# 3/16
E(final ^ Proj[1]) # the expected number of flips: 28/3
# 9.333333333085183
```

- **Check 1, one state per row.** Call the function on one state from each row and read the result against the table. A head has weight $3/4$ and makes an even run odd, so `<5, 1, 0>` must carry the $3/4$. A done state must come back as itself with

weight 1.

- **Check 2, the leftover.** After `evolve`, the weight on not done should be tiny, and the expected count must not equal the number of steps. Small but not tiny means too few steps: at 100 steps it is $5.5e-06$ here and E reads 9.33329.
- **Check 3, the smallest outcome by hand.** The game can only end at 2 flips through H T, so that weight must be $p(1 - p) = 3/16$.

What each slip looks like. The first two give no error, which is why the checks exist:

Slip	What you see	Fix
The two tuples of a <code>weighted_as</code> are swapped (the even row written head first)	Nothing at $p = 1/2$: E is still 6. At $p = 3/4$ the even state shows 3/4 on <code><5, 0, 0></code> , done at 2 flips is 1/16 in place of 3/16, and E is 20 in place of 28/3.	Put the tail tuple first on every line, and end each line with the comment <code># tail, head</code> .
No freeze line	The done state comes back with two branches. After 100 steps E is exactly 100 and the weight on not done is 0.89.	The first line of the function: <code>if done == 1: then return constant(flips, odd, done)</code> .
<code>if odd = 0</code>	A <code>SyntaxError</code> as soon as the block is entered.	<code>==</code>
No <code>@conditional_kind(codim=3)</code> above the <code>def</code> , or a bare tuple as the start state	The error printed below, the same one in both cases.	Add the decorator. Start from <code>constant(0, 0, 0)</code> .
Too few steps	The weight on not done is small but not tiny.	More steps: 200 take about a second here.

```
evolve((0, 0, 0), next_state, 5)           # a bare tuple as the start state
# evolve requires either a Kind and Conditional Kind or an FRP and conditional FRP
```

Route 2: restart patterns and a fixed point. This is the question in the problem's hint: when does the future look like the beginning? Walk forward from the start, and after each flip call the position done, same as the start, or neither. Extend only the neither branches, and stop when none is left:

Opening	Position	Flips used	Weight	Total number of flips
T	same as the start	1	$1 - p$	<code>total ^ (__ + 1)</code>
H	neither: flip again			
H T	done	2	$p(1 - p)$	<code>constant(2)</code>
H H	same as the start: the run is even again	2	$p \cdot p$	<code>total ^ (__ + 2)</code>

The openings that are left are the values of a `weighted_as`. Their labels can be any different numbers, as long as they match the keys of the dictionary. Their probabilities are the weights, and must add to 1. Done gets `constant(flips used)`, same as the start gets `total ^ (__ + flips used)`, // mixes the cases, and `iterate` repeats the pass from a first guess. For the odd heads run that is `odd_heads_exact` in 4.5. For **flip until two heads in a row** the openings are the same, but H H is now done and H T starts over, so the two lines trade places:

```
def two_heads_exact(total, p=symbol('p')):           # flip until two heads in a row
    p = as_quantity(p)
    opening = weighted_as(0, 2, 3, weights=[1 - p, p * (1 - p), p * p]) # labels for T, H T, H H
    total_given_opening = conditional_kind({
        0: total ^ (__ + 1),           # T: one flip used, start over
        2: total ^ (__ + 2),           # H T: two flips used, start over
        3: constant(2),                # H H: done in two flips
    })
    return total_given_opening // opening

hh50 = iterate(two_heads_exact, 100, constant(2), p='1/2')
hh75 = iterate(two_heads_exact, 100, constant(2), p='3/4')
E(hh50)           # (1 + p) / p^2 = 6
# 6
E(hh75)           # 28/9
# 28/9
[round(hh75.kernel(n), 6) for n in irange(2, 7)]       # the first six branches for p = 3/4
# [0.5625, 0.140625, 0.140625, 0.061523, 0.041748, 0.021973]
```

By hand (5.6). The same three cases give an equation for the expectation. Odd heads run: $E = (1 - p)(1 + E) + p \cdot p(2 + E) + 2p(1 - p)$, so $E = (1 + p) / (p(1 - p))$, which is 6 at $p = 1/2$ and $28/3$ at $p = 3/4$. Two heads in a row: $E = (1 - p)(1 + E) + p(1 - p)(2 + E) + 2p \cdot p$, so $E = (1 + p) / p^2$, which is 6 and $28/9$.

4.12 A two-state Markov chain: the state only, or the whole path

A system that stays where it is with weight $3/4$ and switches with weight $1/4$. In the dictionary the key is where you are now, the values inside `weighted_as` are where you go next, and the weights follow the order of those values. `//` and `evolve` keep only the current state. `>>` keeps the whole path, and then the step is handed the whole path, so it must read the last value.

With the dictionary itself, `constant(1) >> move >> move` fails: it is not defined on a whole path (6.2).

```
move = conditional_kind({1: weighted_as(1, 2, weights=[3/4, 1/4]),      # from 1: stay 3/4, switch 1/4
                        2: weighted_as(1, 2, weights=[1/4, 3/4])})    # from 2: switch 1/4, stay 3/4
move // constant(1)           # one step, state only
# 1: 3/4   2: 1/4
evolve(constant(1), move, 2)   # two steps
# 1: 5/8   2: 3/8
evolve(constant(1), move, 3)   # three steps
# 1: 9/16  2: 7/16

@conditional_kind
def step(history):              # >> passes the whole history, so read the last value
    return move(history[-1])

constant(1) >> step >> step    # the whole path after two steps
#      ,---- 9/16 ---- <1, 1, 1>
#      |---- 3/16 ---- <1, 1, 2>
# <> -|
#      |---- 1/16 ---- <1, 2, 1>
#      |---- 3/16 ---- <1, 2, 2>
evolve(constant(2), move, 2)   # the Markov shortcut: told the state is 2, two steps remain
# 1: 3/8   2: 5/8
Kind.equal(move // uniform(1, 2), uniform(1, 2)) # the long-run Kind is unchanged by a step
# True
```

- **By hand.** The weight on a state after a step is (was there) · (stay) + (was on the other one) · (switch). After two steps: $3/4 \cdot 3/4 + 1/4 \cdot 1/4 = 5/8$ on 1. After three: $5/8 \cdot 3/4 + 3/8 \cdot 1/4 = 9/16$ on 1, and $7/16$ on 2. The $9/16$ is $36/64$, not $(3/4)^2$.
- **A path.** Multiply the weights along it. `<1, 1, 2>` is a start at 1 (weight 1), a stay ($3/4$) and a switch ($1/4$), so $3/16$. The four paths add to 1. This is the rule behind the path likelihood in 9.4.
- **Which one.** Where the system is after n steps: `evolve`. The weight of an observed sequence, or a question about the history: `>>`, once per step, or a loop with `k = k >> step`.
- **The Markov shortcut (7.1).** Told the whole current state, drop the history and evolve the remaining steps from `constant(state)`. Definitions are in 1.11.

5 By-hand math

5.1 Sums and counting

Fact	Formula
Finite geometric sum	$1 + r + r^2 + \dots + r^n = (1 - r^{n+1}) / (1 - r)$
Infinite geometric sum, $ r < 1$	$1 + r + r^2 + \dots = 1 / (1 - r)$, and $r^n + r^{n+1} + \dots = r^n / (1 - r)$
Weighted geometric sum	$r + 2r^2 + 3r^3 + \dots = r / (1 - r)^2$
Sums of integers	$1 + 2 + \dots + n = n(n+1)/2$, and $1 + 4 + \dots + n^2 = n(n+1)(2n+1)/6$
Orderings of n things	$n! = n(n-1) \dots 1$
Subsets of size k from n	$C(n, k) = n! / (k! (n-k)!)$
Ordered draws of k from n , no repeats	$n! / (n-k)!$
All subsets of n things; all 0/1 tuples of length n	2^n
Binomial theorem	$(x + y)^n = \sum C(n, k) x^k y^{n-k}$
Size of a join power	$\text{size}(k ** n) = \text{size}(k)^n$
Powers worth having	$2^{10} = 1024$, $2^{20} \approx 1.05e6$, $6^3 = 216$, $6^4 = 1296$, $6^5 = 7776$, $6^{10} = 60,466,176$

5.2 Kinds worth knowing by heart

Kind	Weights	Expectation
One balanced die	1/6 on each of 1 to 6	7/2 (variance 35/12)
Sum of two balanced dice	1, 2, 3, 4, 5, 6, 5, 4, 3, 2, 1 (out of 36) on 2 to 12	7
Uniform on 1 to n	1/n each	$(n+1)/2$
An event with probability p	$1-p$ on 0 and p on 1	p (variance $p(1-p)$)
Number of successes in n independent trials, each with probability p	$C(n, k) p^k (1-p)^{n-k}$ on $k = 0$ to n	np (variance $np(1-p)$)
Trials up to and including the first success	$(1-p)^{n-1} p$ on $n = 1, 2, \dots$; more than n trials has probability $(1-p)^n$	$1/p$
<code>geometric(0, 1, ..., m, r=r)</code>	$r^j (1-r) / (1-r^{m+1})$ on j	use $E(\dots)$

5.3 Probability rules, stated as expectations of events

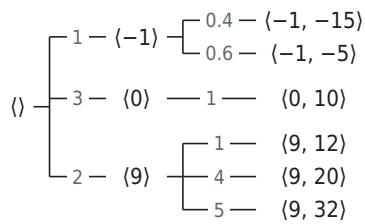
- Not A: $1 - E(A)$. At least one: $1 - (\text{probability of none})$.
- A or B: $E(A) + E(B) - E(AB)$. A and B: $E(A) \cdot E(B | A)$, which is $E(A) \cdot E(B)$ when independent.
- Given C: $E(A | C) = E(AC) / E(C)$.
- Law of total probability (averaging over the cases of an earlier stage): $E(B) = \sum E(A_i) \cdot E(B | A_i)$. The same with any FRP in place of B, the law of total expectation: $E(X) = \sum E(A_i) \cdot E(X | A_i)$.
- Bayes: $E(A_i | B) = E(A_i) \cdot E(B | A_i) / \sum_j E(A_j) \cdot E(B | A_j)$. Lay it out as a table: prior weight, weight of the observation given that case, product, normalize.
- Linearity of expectation is the Scaling and Additivity rules together: $E(aX + bY) = a \cdot E(X) + b \cdot E(Y)$, whether or not X and Y are independent.
- $\text{Var}(a + bX) = b^2 \cdot \text{Var}(X)$. For independent X and Y, $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$.
- For a count N of things each contributing an independent amount A: $E(\text{total}) = E(N) \cdot E(A)$.

5.4 Worked by hand

The arithmetic you may have to show on paper. Every number in this subsection is checked against frplib when the file is built.

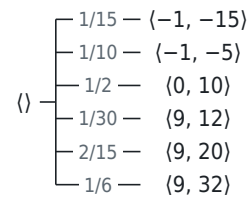
Canonical form of a tree

As given: two levels, any positive weights



→ **normalize** the weights
at each node, then
multiply along each path

Canonical form: one level



This is the two-level Kind pictured in Chapter 1 of the text. At the root, the weights 1, 3, 2 become $1/6$, $1/2$, $1/3$. Under (-1) , 0.4 and 0.6 already add to 1. Under (9) , the weights 1, 4, 5 become $1/10$, $4/10$, $5/10$. Products along the paths: $1/6 \cdot 0.4 = 1/15$, $1/6 \cdot 0.6 = 1/10$, $1/2 \cdot 1 = 1/2$, $1/3 \cdot 1/10 = 1/30$, $1/3 \cdot 4/10 = 2/15$, $1/3 \cdot 5/10 = 1/6$. Check that they add to 1: in thirtieths, $2 + 3 + 15 + 1 + 4 + 5 = 30$. A one-level Kind only needs dividing by the total: weights 3, 1, 2 become $1/2$, $1/6$, $1/3$.

Transform, update, expectation, variance

Take the Kind with weights 1, 3, 2, 2 on the values $-1, 0, 1, 2$. The total weight is 8.

- **Transform by x^2 .** The values become 1, 0, 1, 4. The two branches that land on 1 merge and their weights add: $3/8$ on 0, $1/8 + 2/8 = 3/8$ on 1, $2/8$ on 4.
- **Update with $x \neq 0$.** Erase the 0 branch. The others keep their weights 1, 2, 2, now out of 5: $1/5$, $2/5$, $2/5$ on $-1, 1, 2$.
- **Probability of an event.** Add the canonical weights of the branches where it is true: $P(x > 0) = (2 + 2)/8 = 1/2$.
- **Expectation.** $(-1 \cdot 1 + 0 \cdot 3 + 1 \cdot 2 + 2 \cdot 2)/8 = 5/8$. After the observation: $(-1 \cdot 1 + 1 \cdot 2 + 2 \cdot 2)/5 = 1$.
- **Expectation of a transform.** Average $\psi(v)$ with the original weights: $E(x^2) = (1 \cdot 1 + 0 \cdot 3 + 1 \cdot 2 + 4 \cdot 2)/8 = 11/8$.
- **Variance.** $E(x^2) - E(x)^2 = 11/8 - 25/64 = 63/64$.

A join, conditioning and Bayes in one table

Balls and Boxes from lecture 6. Box i holds $i + 1$ red balls and 1 blue ball, for $i = 1, 2, 3$. Pick a box uniformly, then a ball uniformly from that box. The ball is blue. Which box was it?

Box	Weight of the box	Weight of blue given the box	Product: the weight of (box, blue)	Divided by the total
1	$1/3$	$1/3$	$1/9 = 20/180$	$20/47$
2	$1/3$	$1/4$	$1/12 = 15/180$	$15/47$
3	$1/3$	$1/5$	$1/15 = 12/180$	$12/47$
Total	1		$47/180$	1

- The product column is the **join**: weights multiply along a path of the tree.
- Its total, $47/180$, is the probability of blue. That weighted average of the middle column is **conditioning, blue // box**.
- Dividing by the total is the **observation** (erase the red-ball branches, renormalize), and reading off the box is the projection. Together: Bayes's Rule (3.5). The tree is drawn in 5.5.
- An expectation from the same table: $E(\text{box given blue}) = (1 \cdot 20 + 2 \cdot 15 + 3 \cdot 12)/47 = 86/47$.

Iverson braces

Replace each brace by 1 when its condition is true and by 0 when it is false. For the practice set's $f(x) = 3 \cdot \{-10 < x < 2\} + 2 \cdot \{x > 0\} - 10 \cdot \{x < 1\} - \{x = 4\}$: $f(1) = 3 + 2 - 0 - 0 = 5$, $f(4) = 0 + 2 - 0 - 1 = 1$, and $f(0) = 3 + 0 - 10 - 0 = -7$. On an FRP: $\{X > 0\}$ is an event, $E\{X > 0\}$ is its probability, and $X \cdot \{X > 0\}$ equals X when X is positive and 0 otherwise.

Independence from a table of weights

Compare each joint weight with the product of its two marginal weights. One mismatch is enough to say dependent. Agreement everywhere means independent. Example: (x, y) has weight 0.4 on $(0, 0)$, 0.2 on $(0, 1)$, 0.1 on $(1, 0)$ and 0.3 on $(1, 1)$. The marginal weights are 0.6 and 0.4 for x , and 0.5 and 0.5 for y . Since $0.6 \cdot 0.5 = 0.3$ is not 0.4, x and y are dependent.

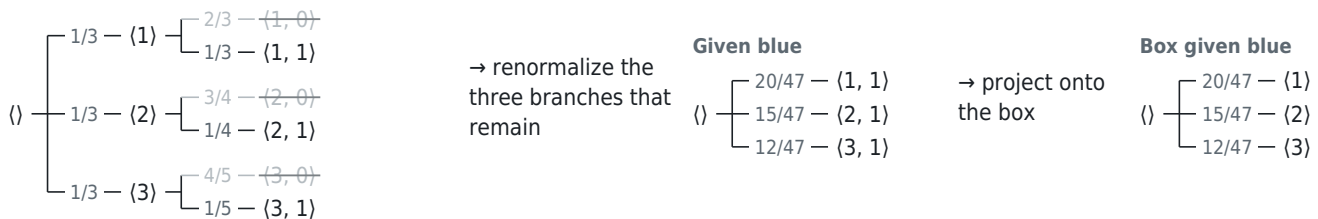
A likelihood by hand is a product of weights along the observed path (3.11, 9.4). Entropy by hand is in 3.15. A kernel and a distribution operator by hand are in 10.1.

5.5 Drawing Kind trees and wiring diagrams

A Kind tree by hand.

1. Put the root $()$ at the left. (frplib prints it as $\langle \rangle$.)
2. Draw one branch for each possible value of the first stage. Write the weight on the branch and the value, in angle brackets, at its end.
3. From each of those nodes, draw one branch for each value the next stage can take given that node, with its conditional weight. Label the new node with the whole history, (first, second): every node extends the node before it (the Prefix Rule).
4. Every path from the root must have the same number of branches (the Completeness Rule). The leaves are the possible values.
5. If canonical form is asked for: normalize at each node, multiply along each path, sort the values and redraw with one level (5.4).
6. For an observation, cross out the branches where the condition is false and renormalize what is left.

Join, with the red-ball branches erased



Balls and Boxes (5.4) as trees. The values of the join are (box, 1 if blue). Join, update, project. `unfold(k)` prints the multi-level tree of a Kind, which is a quick check on a hand drawing.

Wiring diagrams. Each of the Big 3 has one. A box is an FRP, a line is a wire carrying a value from left to right, and the hardware on the wire says what happens to the value. Replacing every FRP by its Kind turns the same diagram into the computation.

Wiring diagram	Algebra, and what happens to the Kind
	Transform. $X \hat{=} \psi \cdot i$. The wire passes through an adapter that computes the statistic. Kind: apply ψ to every leaf and merge equal values.
	Join. $X \gg C$. X's value enters the conditional FRP, whose switch picks the FRP to run next. Out comes X's value followed by the new one. Kind: attach the target Kind for u at each leaf u .
	Update. $X \mid \xi \cdot i$. An observation processor for the condition ξ only produces values for which ξ is true. Kind: erase the other branches.
	Conditioning. $C \parallel X$: a join followed by an adapter that drops X's components.
	Bayes's Rule. Join, update with the observed later value, project onto the early part: <code>bayes(v, a, t)</code> .

- To draw one: a square for each FRP with its name inside, the result labeled with its expression; a small ψ -shaped plug on the wire for a statistic, with the statistic's name under it; a box with a port on each side for a conditional FRP; a thin bar across the wire for an observation processor, with the condition under it. (Lectures 6 and 7 and the older text call it a constraint processor.)

- Longer systems chain these left to right in the order things happen, exactly as the algebraic expression reads.

5.6 First-step equations by hand

For an exact expected waiting time or hitting probability (3.9). The recipe:

1. Name one unknown for each state the system can be in before it stops: E_s for the expected number of further steps from s , or f_s for the probability of ending the way you want.
2. Stopping states have known values: 0 for a time, 1 or 0 for a probability.
3. Write one equation per state by looking one step ahead: $E_s = 1 + \sum \text{weight} \cdot E_{\text{next}}$, or $f_s = \sum \text{weight} \cdot f_{\text{next}}$.
4. Solve, starting with the state closest to stopping.
5. Check against *evolve* with a clock (3.6) or *iterate* (3.8).

System	Equations	Solution
Flips until the first head, $P(\text{heads}) = p$	$E = 1 + (1 - p) \cdot E$	$E = 1/p$
Flips until H H, fair coin. E_0 : no progress. E_1 : the last flip was H	$E_0 = 1 + \frac{1}{2} E_1 + \frac{1}{2} E_0$ and $E_1 = 1 + \frac{1}{2} \cdot 0 + \frac{1}{2} E_0$	$E_1 = 4$ and $E_0 = 6$
Flips until H T, fair coin	$E_0 = 1 + \frac{1}{2} E_1 + \frac{1}{2} E_0$ and $E_1 = 1 + \frac{1}{2} E_1 + \frac{1}{2} \cdot 0$	$E_1 = 2$ and $E_0 = 4$
Flips until H H H, $P(\text{heads}) = p$, $q = 1 - p$. E_s : the last s flips were heads	$E_0 = 1 + q E_0 + p E_1$, $E_1 = 1 + q E_0 + p E_2$ and $E_2 = 1 + q E_0 + p \cdot 0$	$E_0 = 1/p + 1/p^2 + 1/p^3$, which is 14 for a fair coin
Flips until an odd run of heads is followed by a tail (practice 5), $q = 1 - p$. E_0 : even run. E_1 : odd run	$E_0 = 1 + p E_1 + q E_0$ and $E_1 = 1 + p E_0 + q \cdot 0$	$E_0 = (1 + p) / (p(1 - p))$
Fair ± 1 walk from 0 that stops at -3 or $+2$: probability f of reaching $+2$	$f(s) = \frac{1}{2} f(s - 1) + \frac{1}{2} f(s + 1)$, with $f(-3) = 0$ and $f(2) = 1$	a straight line: $f(s) = (s + 3)/5$, so $f(0) = 3/5$
The same walk: expected number of steps g	$g(s) = 1 + \frac{1}{2} g(s - 1) + \frac{1}{2} g(s + 1)$, with $g(-3) = 0$ and $g(2) = 0$	$g(s) = (s + 3)(2 - s)$, so $g(0) = 6$
Repeat a trial until A or B happens, with probabilities a and b on each trial: probability f that A comes first	$f = a + (1 - a - b) \cdot f$	$f = a / (a + b)$, and the expected number of trials is $1 / (a + b)$

- **Why H T is quicker than H H.** After an H, a second H keeps your progress toward H T, but a T throws away your progress toward H H.
- **The race rule in use.** In the homework's craps game (3.16), a point of 4 has 3 chances in 36 on each roll against 8 in 36 for a 7 or 11, so the point comes first with probability $3/(3 + 8) = 3/11$. Doing that for every point gives the whole game by hand: $8/36 + (2/36) \cdot (9/11 + 16/12 + 25/13) = 0.4486$.
- **Solving two equations.** Substitute the simpler one into the other. For H H: $E_0 = 1 + \frac{1}{2} (1 + \frac{1}{2} E_0) + \frac{1}{2} E_0 = 1.5 + 0.75 E_0$, so $E_0 = 6$.
- **Any pattern.** The unknowns are the positions along the pattern, and 3.18 builds the equations, solves them, and lists the answers for ten patterns.

6 Checks and fixes

6.1 Ten-second checks before you write an answer down

- `size(k)` matches your own count of the possible values, and `dim(k)` matches the number of components you expect. For a conditional Kind, look at `typeof(m)`.
- Probabilities lie between 0 and 1. An expectation lies between the smallest and the largest value.
- Run the same code on a tiny case you can do by hand (2 rolls instead of 10).
- For anything evolved for a finite number of steps, print the unresolved weight, then run more steps and confirm the answer does not move.
- Check a shortcut against the direct computation on a small case with `Kind.equal`.
- By symmetry, should the answer depend on the label you picked? Try another label.
- Re-read what is asked for: a Kind, an expectation, a probability, an FRP's value, or code. "Show how you would find" wants the expression. "Find" wants the result too.
- If the problem asks for weights in unnormalized and canonical form, give both.

6.2 Error messages and what they mean

In the playground, `frplib`'s own errors (`KindError`, `MismatchedDomain` and the like) print only their message, without the error's name. Other errors end a short traceback with a line such as `TypeError: ... explain_error()` shows the full traceback of the most recent error, and a script run with plain Python always shows the name.

What you see	Cause	Fix
<code>MismatchedDomain: Value set elements have different dimensions</code>	A conditional Kind was applied (<code>//</code> , <code>>></code> , <code>evolve</code>) to a Kind whose values have different dimensions, usually a frozen short state mixed with longer ones. Such a Kind prints its short values padded with <code>□</code> .	Keep the state a fixed size: pad it, or carry a counter instead of a growing history.
<code>KindError: A conditional Kind is not defined on all the values requested of it, including <...></code>	The Kind on the left of <code>>></code> or <code>//</code> can produce a value outside the conditional Kind's domain. In a <code>>> m</code> , <code>m</code> is given all of <code>a</code> 's value.	Add the missing case, or define <code>m</code> as a function of the full value (3.5).
<code>I could not create a Kind from ...</code>	The conditional Kind returned a number, a tuple or a dictionary.	Return a Kind: <code>constant(v)</code> .
<code>evolve</code> requires either a Kind and Conditional Kind or an FRP and conditional FRP	The start state is a bare tuple, or the step function has no <code>@conditional_kind</code> above its <code>def</code> .	<code>evolve(constant(0, 0, 0), next_state, n)</code> , and add the decorator (4.11).
<code>float()</code> argument must be ... not 'Expectation'	<code>float(E(k))</code> .	<code>as_float(E(k))</code> .
<code>unsupported operand type(s) for +: 'decimal.Decimal' and 'float' from a ChiSquare(expected) statistic</code>	A plain integer among the expected values.	Write it with a decimal point, <code>13.0</code> , or pass <code>E(k).raw</code> (9.5).
<code>unsupported operand type(s) for *: 'decimal.Decimal' and 'float'</code>	Mixing exact weights with floats.	<code>float(w)</code> , or <code>as_quantity(x)</code> on the float.
The computation hangs	The Kind is too large: a big <code>k ** n</code> , or a long history kept in the state.	<code>fast_join_pow</code> ; a smaller state; <code>evolve(..., transform=clean)</code> ; sample FRPs; or reason about which branches survive (7.3).
Weights like <code>1E-30</code> on many branches	Negligible branches or round-off.	<code>clean(k)</code> .
Decimals where you wanted fractions	Float weights, or denominators above the display limit.	Give weights as strings like <code>'1/3'</code> . <code>environment.set_max_denom(999)</code> .
An updated Kind that was not transformed	<code>k (cond) ^ psi</code> applies <code>psi</code> to the condition.	<code>(k (cond)) ^ psi</code> or <code>psi @ k (cond)</code> .

What you see	Cause	Fix
name 'guard' is not defined, and the same for Ceiling, StDev, ordered_sample	The v0.3.10 cheatsheet prints these four names. frplib 0.3.10 does not define them.	given, Ceil, StdDev, ordered_samples.
as_quantity: could not convert [10, 8, 15] to a quantity	A list was passed to <code>ijoin_log_likelihood</code> , as in the posted practice solution. ChiSquare with a list fails the same way.	A tuple, <code>ijoin_log_likelihood((10, 8, 15))</code> , or separate arguments.
TypeError: bad operand type for unary -: 'Statistic'	<code>-__</code> or <code>Exp(-__)</code> . Statistics have no unary minus.	<code>-1 * __</code> , <code>0 - __</code> or <code>Negate</code> .
A function (probably a Statistic or conditional Kind/FRP) expects input of dimension at least 2 dimension 1 was given, or Statistic Proj[2] appears incompatible with this Kind	A projection past the end of the value. A common cause: <code>m(u)</code> is only the target, without the input in front.	Count components on what you really have: <code>dim(k)</code> . Use <code>m.joined(u)</code> to keep the input.
<code>evolve(start, next_state)</code> returns a Kind that has barely moved	The number of steps was left out. It defaults to 1.	<code>evolve(start, next_state, n)</code> .
Both components of <code>X * X</code> are equal	The same FRP was joined with itself.	<code>X * clone(X)</code> or <code>X ** 2</code> .
name '...' is not defined in a script	The playground names were not imported.	<code>from frplib.playground import *</code>
name 'ijoin_log_likelihood' is not defined in the playground, and the same for Negate, Shift, Between, source	The frplib on your machine is older than 0.3.10.	Update it while you have a network, or use the replacements in 6.4 .

6.3 Finding things during the exam

- Start the playground with `frp playground`. Load your own file from the working directory with `from myfile import *`.
- `info()` opens the interactive help, `info('object index')` lists every object, and `info(weighted_as)` or `help(weighted_as)` documents one. `help(weighted_as, True)` shows Python's own help, which includes the exact argument list. (The cookbook writes `info('object-index')`, which frplib 0.3.10 answers with "does not match any topics".)
- Input can span several lines, and a blank line at the end enters it. The arrow keys (or Control-p and Control-n) walk through the history, Control-r searches it, F3 shows all of it and F2 opens the settings menu.
- `source(obj)` prints the code behind a function, statistic or conditional Kind. `explain_error()` explains the most recent error.
- `textbook()`, `cookbook()` and `cheatsheet()` open the PDFs that ship inside frplib.

The examples from the text and lectures are importable, for instance `from frplib.examples.tournament import *`:

Module in frplib.examples	What it contains
tournament	Elimination tournament: <code>first_round</code> , <code>next_round</code> , <code>winner</code> .
mouse_escape	Both versions of the mouse on the steps: <code>move</code> , <code>move_alt</code> , <code>escaped</code> .
drunken_sailor	A walk with two boundaries, as a path and as a state with a clock.
dividing_the_pizza	Repeating a trial until it resolves, with <code>evolve</code> .
doubled_cards	Two cards drawn without replacement, joined.
hunters_success	Independent events with symbolic weights and <code>independent_join</code> .
rig_at_risk	A random number of random amounts (3.7).
elevator_stops	A random number of passengers, with the state kept as a set of floors.
buckets_and_balls, disease_testing_redux, a_kid_named, horse_race, monty_hall	Observations and Bayes's Rule, including symbolic versions.
waiting_for_heads, doubled_heads_and, recursive_rover	Fixed points (3.8).
dirichlet	<code>solve_dirichlet_sparse</code> and the lava room (3.9).
alice_bob	<code>monoid_trick</code> and the card-drawing conditional Kinds.

Module in <code>frplib.examples</code>	What it contains
<code>traffic_light</code> , <code>labyrinth</code>	State machines evolved step by step.
<code>six_of_one</code>	<code>without_replacement</code> with derived statistics.

6.4 Before the exam: an offline dry run

From lecture 13's announcement: Thursday, 7 to 8:30, in Porter Hall 100. Bring your computer. Open book, notes, your course work and `frplib`. No network or third party software. The practice set's cover sheet adds that all work goes in the spaces provided on the paper. So everything has to work with the wifi off, and the answers are written by hand.

While you still have a network. Lecture 13 lists the current versions as ProbExp 3.1.1, `frplib` 0.3.10 and Cheatsheet 0.3.10, and recommends updating `frplib`. Run `frp --version`. If it prints anything older, update now, following the course's instructions (with `pip` that is `pip install --upgrade frplib`, run in the same environment that `frp` starts from). `frplib` 0.3.10 carries the text v3.1.1 (3 Oct), the cheatsheet v0.3.10 (4 Oct) and the cookbook v1.0.6 (5 Oct) inside it, so updating `frplib` also updates all three. This guide was built and checked on 0.3.10, and its references to the text follow v3.1.1.

If you cannot update. An earlier build of this guide (7 Oct, before section 12 was added) was run in full on `frplib` 0.3.2, and apart from random samples it gave the same results there, with these exceptions. `ijoin_log_likelihood` does not exist, which stops the likelihood lines in 3.11, 4.5 and 8.5: add up `math.log(k.kernel(x))` over the observed values instead, as 3.11 shows. `Negate`, `Shift`, `Between` and `ForEach(..., by=2)` from the table in 2.2 do not exist. `source()` does not exist. Three `frplib.examples` modules are missing (`drunken_sailor`, `dividing_the_pizza`, `alice_bob_chatting`). And the PDFs inside it are older: text v3.0.1, cheatsheet v0.2.14, cookbook v1.0.5. Versions between 0.3.2 and 0.3.10 were not tried, and neither were the changes made after that run, chiefly sections 12 and 4.11.

Dry run, about ten minutes.

1. Turn the wifi off and leave it off for the rest of this list.
2. In a terminal, `frp --version` prints `frplib, version 0.3.10`, the version lecture 13 lists, and `frp playground` starts with a welcome line that names the same version.
3. In the playground, `uniform(1, 2, 3) ** 2 ^ Sum` prints a Kind, `from frplib.examples.tournament import *` loads, and `info()` opens the built-in help.
4. `textbook()`, `cookbook()` and `cheatsheet()` open in your PDF viewer. The three PDFs are stored inside `frplib`, so they need no network. `textbook(to='exam')` copies the text into a folder named exam.
5. Gather in one folder, as files and not as browser tabs: the lecture slides through lecture 13, the homework handouts and posted solutions through homework 6, your own homework, the course's Python files (`hw2.py`, `hw3.py`, `hw4.py`, `hw4_tchoo.py`, `hw5.py`, `hw6.py`, `lecture5.py`, `lecture10.py`), the practice set with its posted solutions, and this guide. Start the playground from that folder so that `from hw5 import *` works.
6. Search this PDF for a word (Ctrl-F or Cmd-F), and click a section number to confirm that the links work in your viewer.
7. Charge the laptop and pack the charger.

What the cover sheet allows. The practice set says its instructions are accurate for the exam.

Allowed	Not allowed
The text, the <code>frplib</code> Cheatsheet and Cookbook, and all course materials posted this semester	Third party materials or software, including LLM tools
Your notes and your previous work in the course	Communicating with anyone except the instructor and the TAs
<code>frplib</code> , Python, and a program to view or edit files	Network activity or browsing once the exam begins. Wifi off.
Asking the instructor or a TA any question you have, including about <code>frplib</code>	

This guide was assembled with an LLM from those course materials. If you have not already asked, check with the instructor or a TA before the exam whether it counts as your notes.

During the exam.

- The cover sheet's own advice: plan the approach and write down what you want to find before computing. Long, tedious calculations are not intended, so if you are in one, stop and look for the shortcut (7.3). Use spare time to check (6.1).
- Show the work. For a computed answer, write the `frplib` expression you ran and its result. For a system, the essential pieces are the initial state, the conditional Kind of the next state and the statistic that reads off the answer.

- Copy digits carefully. The practice set and the homework ask for four significant figures.
- Short answers are quick points. Do them first and keep each to a sentence or two (**1.14**).
- The practice set notes that it has more problems than an actual exam. With 90 minutes, decide early how long each problem deserves and move on when its time is up.

A timed pass, once, before Thursday. Wifi off, 90 minutes, paper and pen. Work practice problems 1, 2, 4, 5 and 7, or any five, to full written answers using only what the rules allow, then compare with the posted solutions and with **4.1** to **4.9**. Note every lookup that was slow and fix it: a bookmark, a file moved into the folder, a section number you now know.

7 From words to a model

The steps between reading a word problem and typing the first line. Chapter 8 of the text calls this the pattern language: describe the system with FRPs, and the same expression written with Kinds is the analysis.

7.1 The translation, step by step

1. **Write down what is asked.** A Kind, an expectation, a probability (the expectation of an event), a best choice, or code. Name the FRP whose value would answer it.
2. **List the sources of randomness.** Each roll, flip, draw, arrival or choice is an Element FRP. Give each one a Kind from the wording (table below).
3. **Decide how they combine.** If they do not affect each other: * or **. If a later one depends on an earlier one: write the later stage as a conditional Kind and join with >>. Put the stages in the order they happen.
4. **Decide what to keep.** If the data are the whole history, keep it with joins. If only the current situation matters, keep a state and use // or evolve (**7.2**).
5. **Turn "given", "you observe" and "suppose" into a condition** and update with | at the point in the system where the observation happens. Write down in words which observation you are using (**4.7**).
6. **Turn the question into a statistic.** "Is it true that" is a condition, "how many" is a count, "how much" is a sum or a payoff function.
7. **Write the same expression with Kinds,** compute it, and check it (**6.1**).

The seven steps applied to three of the practice problems (solutions in **4.8**, **4.7** and **4.4**):

Step	Pricing the payoff (practice 8)	Customer purchases (practice 7)	Dr. Jones (practice 4)
1. Asked	$E(B_4)$, and whether the bet favors you	the Kind of the total T, and $E(T)$	the Kind of the final state, and whether the jewel reaches a museum
2. Randomness	three balanced dice	the number of customers N, and each customer's purchase	which cell he searches next
3. Combine	independent: $d6 ** 3$	T depends on N: a conditional Kind for T given $N = n$	the next cell depends on where he is and where he has been: a conditional Kind
4. Keep	all three rolls	only T: join, then project onto it	a state: a found flag, then the cells searched so far, in order
5. Observations	none	part (d): five purchases seen, so $T \geq 5$, and five customers, so $N \geq 5$	part (d) describes one particular state; evaluate <code>next_state</code> there
6. Statistic	which rolls are 4s, then the payoff from their count	none needed, T is the answer	<code>museum</code> : found, and fewer than six cells searched
7. Expression	$E(d6 ** 3 \wedge \text{ForEach}(_ == 4) \wedge \text{IfThenElse}(\text{Sum} > 0, \text{Sum}, -1))$	$E(\text{kind_N} >> \text{t_given_n} \wedge \text{Proj}[2])$	<code>evolve(constant(0, 2, 2), next_state, 8) ^ museum on paper; 4.4</code> on running it

The words	The piece of the model
equally likely, at random, fair, balanced, well shuffled or well mixed	<code>uniform(...)</code>
twice as likely, weights proportional to, odds of r to 1	<code>weighted_as(..., weights=[...])</code> , or <code>choice(a, b, r)</code> with weight r on b for each 1 on a

The words	The piece of the model
with probability p	<code>binary(p)</code> for values 0 and 1, otherwise <code>weighted_as(a, b, weights=[1 - p, p])</code>
independently, unaffected by the others, returned to the box and mixed	<code>*</code> and <code>**</code>
without replacement, in succession from a deck, among those not yet searched	a conditional Kind that leaves out what is used up, or <code>ordered_samples, without_replacement</code>
depends on, if ... then ... otherwise, according to which	a conditional Kind and <code>>></code>
a random number of things, each with its own amount	the total given N as a conditional Kind, then <code>// kind_N (3.7)</code>
until, the first time, stops when	a state with frozen terminal values (3.6), or a fixed point (3.8)
by the n -th attempt, within n steps, before the sixth	evolve for exactly n steps, or put a clock in the state
given, you observe, suppose we learn, it turns out that	<code> </code> with a condition
what does that tell us about the earlier stage	Bayes's Rule (3.5)
the number of, how many	<code>ForEach(cond) ^ Sum</code> , or a counter in the state
the chance that, how likely	<code>E(k ^ condition)</code>
on average, expected, predict, a fair price	<code>E(...)</code>
which is best, should she	an expectation for each option (3.17)
estimate p from what was seen	likelihood (3.11)

The parallel expressions. A system runs for n stages, we observe something about it, and it runs for m more. Write the description with FRPs, replace every FRP by its Kind, and you have the computation. Lined up:

The system, with FRPs	$\text{Data} = ((X_0 \triangleright T_1 \triangleright \dots \triangleright T_n) \mid \zeta) \triangleright T_{n+1} \triangleright \dots \triangleright T_{n+m}$
The analysis, with Kinds	$\text{kind}(\text{Data}) = ((k_0 \triangleright t_1 \triangleright \dots \triangleright t_n) \mid \zeta) \triangleright t_{n+1} \triangleright \dots \triangleright t_{n+m}$
A question	the Feature FRP is $\psi(\text{Data} \mid \xi)$, and its Kind is $\psi(\text{kind}(\text{Data}) \mid \xi)$
If only the state is kept	every $\triangleright$ becomes conditioning, $S_{i+1} = T_{i+1} // S_i$; evolve n steps, update, evolve m more

The Markov shortcut. If the observation tells you the whole current state, the history before it no longer matters. Restart from `constant(observed state)` and evolve only the remaining steps (the robot of Example 8.1 in the text). A walk on 0 to 4 that bounces off the ends, observed at 2 after six steps:

```
t = conditional_kind({0: constant(1), 1: uniform(0, 2), 2: uniform(1, 3),
                    3: uniform(2, 4), 4: constant(3)})

@conditional_kind
def step(path):
    return t(path[-1])

path = constant(0)
for _ in range(6):
    path = path >> step

seen = path | (Proj[7] == 2)
later = seen >> step >> step
# the last component has the same Kind as two steps of a system restarted at 2
Kind.equal(later ^ Proj[9], evolve(constant(2), t, 2))
# True
```

7.2 Choosing the state

A state has two jobs: it must let you write the Kind of the next step using nothing else, and it must let you read the answer off at the end. Start with the smallest thing that does the first job and add components until it does the second.

- **Can I write the next step's Kind from the state alone?** If not, something is missing: the cells already searched, the first roll, the length of the current run.
- **Can I answer the question from the final state alone?** If not, add a clock, an outcome flag, a running total or a maximum so far.
- **Is every component used?** Drop what is not. Fewer distinct states means a smaller Kind and a faster *evolve*. Keep counts instead of lists when the order does not matter.
- Terminal states return `constant(state)`, and all the states possible at one step have the same dimension (**3.6**).
- Number of steps: the deadline if there is one, otherwise enough that the unresolved weight is negligible.

System	State	Start	One step	Read off
Gambler's ruin (3.6)	net winnings s ; add the number of bets n to ask how long	<code>constant(0)</code>	frozen at -3 and at 2 , otherwise $\text{bet} \wedge (_ + s)$	$_ == 2$, or <code>Proj[2]</code> for the clock
Mouse on the steps (lecture 11, 7.4)	the step, 0 to 3	<code>constant(0)</code>	up with weight 1, down with weight 2 (at step 0, stay); step 3 frozen	after 16 steps, $_ == 3$
Tournament (lecture 11, 7.4)	the players still in, in bracket order	<code>constant(1, 8, 4, 5, 2, 7, 3, 6)</code>	independent join of the winners of the adjacent pairs; the dimension halves	after three rounds the state is the winner
Win on Repeat (3.17)	(distinct tickets seen, winner's position or 0); lecture 13: (tickets drawn, won)	<code>constant(1, 0)</code>	a repeat with weight seen, a new ticket with weight 100 – seen; frozen once won	<code>Proj[2]</code>
Dr. Jones (practice 4, 4.4)	(found flag, then the cells searched so far, in order)	<code>constant(0, 2, 2)</code>	weight $1/\text{distance}^2$ on each unsearched cell; unchanged once found (on paper), or the jewel cell repeated (to run it)	found, and fewer than six cells searched; padded: the fifth cell is the jewel cell
Odd run of heads, then a tail (practice 5, 4.5)	posted: (flips, heads in the run, done), or (flips, done) stepping by T, H H, H T; here also (phase, flips)	<code>constant(0, 0, 0)</code> or <code>constant(0, 0)</code>	H lengthens the run; T after an odd run finishes; T after an even run starts over	the flips on the finished branches
Double or nothing (practice 6, 4.6)	the bets so far, kept with joins	<code>choice(0, 1)</code>	<code>bet_again</code> : after a loss the next bet is 0, otherwise a fair bet	Sum, then winnings
Customer purchases (practice 7, 4.7)	two stages rather than a state: N , then the total given N	<code>kind_N</code>	<code>fast_join_pow(Sum, purchase, n)</code> as a conditional Kind	<code>kind_N >> t_given_n ^ Proj[2]</code>
Waiting for a pattern (lecture 13, 3.18)	the position along the pattern	<code>constant(0)</code>	forward on a match, otherwise back to the longest beginning still matched; the end is frozen	the weight at the end position
Misplaced boarding pass (homework 6, 10.3)	(passenger now choosing, 1 if seat 1 is taken)	<code>constant(1, 0)</code>	uniform over seat 1 and the seats of later passengers; once seat 1 is taken everyone sits correctly	<code>Proj[2]</code> after 100 steps
Dicey wait (homework 6, 10.4)	(rolls so far, dice not yet six)	<code>constant(0, d)</code>	the number still not six, <code>fast_join_pow(Sum, choice(0, 1, 5), u)</code> ; frozen at 0	<code>Proj[1]</code> (posted), or $r + 1$ when dice remain; <code>Proj[2] == 0</code> for resolved
Urn game (homework 4, 8.4)	(reds, greens)	<code>constant(r, g)</code>	one round: a run of one color is removed	which count is 0 when the last round starts
Immortals' battle (homework 4, 8.6)	every fighter's power, Connor's team first and then Duncan's	<code>constant(initial_teams)</code>	frozen once a team has no power; otherwise the strategies pick two fighters and the winner takes the loser's power	<code>TeamPowers</code> , the two team totals

System	State	Start	One step	Read off
Robot's path (homework 4, 8.3)	the whole path, kept with joins	<code>constant(1)</code>	a neighbor not yet visited, by edge weight; <code>constant(22)</code> once at the end	<code>Proj[-1] == 22, Contains(11)</code>
Craps (3.16)	(first roll, rolls so far, outcome flag)	the state after the first roll	Cases on the next roll; frozen once the flag is not 0	<code>Proj[3]</code>
Runs test (homework 5, 3.12)	partial runs count (r, r1, r2, r3, m, b)	the state after one bit	the next bit continues the current run or ends it	<code>finalize</code>
Markov path (homework 5, 9.4)	the current symbol; the data are the whole path, so keep the history with joins	<code>choice(1, 2)</code>	switch with weight u, stay with weight 1 – u	the weight of the observed path
Draws without replacement	how many of each color are left	the starting counts	a color is drawn with weight equal to its count, and that count drops by 1	whatever is asked
Robot walk (text, Example 8.1)	the position (x, y)	uniform over the starting points	uniform over the moves that stay in range	the position; after observing it, restart from <code>constant(x, y)</code>

7.3 When the Kind is too big: which branches survive

Practice problem 3 is the model. `roll_kind ** 10` has 6^{10} branches and cannot be printed, yet every part has a short answer. Reason about the branches instead of computing them.

- Write the expression anyway.** "Show how you would find" is answered by `roll_kind ** 10 | (Min == Max)`, even though you will not run it.
- Describe the survivors.** Say which tuples the observation keeps, and count them.
- Weights.** A surviving branch keeps its original weight, the product of the base weights of its components. The survivors are equally weighted exactly when those products are all equal: for instance when only equally weighted faces survive, or when the survivors are rearrangements of one another.
- The normalizing constant is the probability of the observation.** Get it from a complement, or from a monoidal statistic: `E(fast_join_pow(Fork(Min, Max), roll_kind, 10) ^ cond)`.
- Restrict, then join.** If the observation limits every roll separately, update one roll and then take the power: `(k | c) ** n` is `k ** n | All(c)`.
- Check the shortcut on a small power** with `Kind.equal`.

Ten rolls with weights 1/2, 1/4, 1/8, 1/16, 1/32, 1/32 on the faces 1 to 6:

Observation	Branches that survive	Their canonical weights	Probability of the observation
<code>Min == Max</code> (practice 3b)	6: all ten rolls equal	proportional to (weight of the face) ¹⁰	$\sum w^{10} = 0.000977517$
<code>Min == 5</code> (practice 3c)	$2^{10} - 1 = 1023$: 5s and 6s with at least one 5	all equal, 1/1023, because 5 and 6 have the same weight	$(1/16)^{10} - (1/32)^{10}$
<code>Min == 2</code>	$5^{10} - 4^{10} = 8,717,049$: no 1s and at least one 2	not equal: the product of the base weights, divided by the probability at right	$(1/2)^{10} - (1/4)^{10} = 0.00097561$
<code>Max == 3</code>	$3^{10} - 2^{10} = 58,025$: only 1, 2, 3 with at least one 3	product of the base weights, divided by the probability at right	$(7/8)^{10} - (3/4)^{10} = 0.20676$
every roll is even	$3^{10} = 59,049$: tuples of 2s, 4s and 6s	<code>(roll_kind (__ % 2 == 0)) ** 10</code> , with weights 8/11, 2/11, 1/11 on each roll	$(11/32)^{10} = 0.000023037$
<code>Sum == 11</code>	10: nine 1s and one 2, in any position	all equal, 1/10: the survivors are rearrangements of each other	$10 \cdot (1/2)^9 \cdot (1/4) = 0.0048828$
<code>Sum == 10</code>	1: all ten rolls are 1	<code>constant(1, 1, ..., 1)</code>	$(1/2)^{10}$
the first roll is 6	6^9 : the other nine rolls are free	<code>constant(6) * roll_kind ** 9</code> : the other rolls are unchanged	1/32

Observation	Branches that survive	Their canonical weights	Probability of the observation
exactly one 6	$10 \cdot 5^9 = 19,531,250$	product of the base weights, divided by the probability at right	$10 \cdot (1/32) \cdot (31/32)^9 = 0.23483$
all ten rolls different	none: there are only six faces	the empty Kind	0

```
roll_kind = weighted_as(1, 2, ..., 6, weights=['1/2', '1/4', '1/8', '1/16', '1/32', '1/32'])
lo_hi = fast_join_pow(Fork(Min, Max), roll_kind, 10)      # Kind of <min, max> of the ten rolls
size(lo_hi)
# 21
as_float(E(lo_hi ^ (Proj[1] == 2))), 0.5 ** 10 - 0.25 ** 10      # P(min is 2), two ways
# (0.0009756088256835938, 0.0009756088256835938)
as_float(E(lo_hi ^ (Proj[2] == 3))), (7 / 8) ** 10 - (3 / 4) ** 10      # P(max is 3)
# (0.20676206145435572, 0.20676206145435572)
evens = roll_kind | (__ % 2 == 0)                                # one roll, given that it is even
evens
# 2: 8/11    4: 2/11    6: 1/11
# each shortcut agrees with the direct computation on a power small enough to build
Kind.equal(roll_kind ** 4 | (Min == 2), (roll_kind | (__ >= 2)) ** 4 | (Min == 2))
# True
Kind.equal(roll_kind ** 4 | All(__ % 2 == 0), evens ** 4)
# True
Kind.equal(roll_kind ** 4 | (Proj[1] == 6), constant(6) * roll_kind ** 3)
# True
roll_kind ** 4 | (Sum == 5)                                # four rolls: three 1s and one 2
# <1, 1, 1, 2>: 1/4    <1, 1, 2, 1>: 1/4    <1, 2, 1, 1>: 1/4    <2, 1, 1, 1>: 1/4
```

Other shortcuts that replace a long computation. The cover sheet says long, tedious calculations are not intended, so one of these is usually available.

- **Additivity.** The expectation of a sum is the sum of the expectations, with no Kind needed. A count is a sum of 0/1 events, so its expectation is the sum of their probabilities.
- **Symmetry.** FRPs with the same Kind have the same expectation (the aces, 48/5 each). Relabeling the faces of balanced dice changes nothing (practice 8).
- **Complement.** At least one is 1 minus none. $\text{Min} == m$ is "all at least m " minus "all above m ".
- **A race.** When trials repeat until A or B happens, only those two matter: $a / (a + b)$ (5.6).
- **A random number of amounts.** $E(\text{total}) = E(N) \cdot E(\text{one amount})$ (3.7).
- **Markov restart** after observing the whole state (7.1), and the **monoidal trick** for the Sum, Min or Max of many independent trials (3.2).
- **A Kind you already know.** The number of successes in n independent trials, or the trials until the first success (5.2).
- **Independent parts.** An observation about one component of an independent join leaves the Kind of the other components alone.

7.4 The lecture 11 workshop problems, reduced to their models

Lecture 11's exam-prep examples. The text's own versions are importable from `frplib.examples` (`doubled_cards`, `tournament`, `mouse_escape`, `hunters_success`). The current text writes these with `choice`. The modules in `frplib.examples` still use the older `either(a, b, r)`, which is `choice(a, b, 1/r)`. Lecture 13 gives the solutions to Hunter's Success and to the fifth example, Win on Repeat (3.17).

Doubled Cards. Two cards in succession from a deck numbered 1 to 100. D: the second is twice the first. T: either is twice the other. Stages: first card, then a second card that cannot equal it.

```
first_card = uniform(1, 2, ..., 100)

@conditional_kind(codim=1)
def second_card(first):
    # any card except the first one
    return uniform(irange(1, 100, exclude={first}))

draw = first_card >> second_card      # the Data Kind: <first, second>
is_doubled = Proj[2] == 2 * Proj[1]
either_doubled = Or(is_doubled, Proj[1] == 2 * Proj[2])
```

```
size(draw)
# 9900
E(draw ^ is_doubled)          # 1/198
# 0.005050505050505051
E(draw ^ either_doubled)      # 1/99
# 0.0101010101010101
```

By counting instead: $100 \cdot 99 = 9900$ equally weighted ordered pairs, of which 50 have the second card double the first, so $E(D) = 50/9900 = 1/198$ and $E(T) = 100/9900 = 1/99$.

Tournament. Eight ranked players: 1 plays 8, 4 plays 5, 2 plays 7, 3 plays 6, and the winners of neighboring matches meet in the next round. When seeds $r_1 < r_2$ meet, the better seed r_1 is 1.15^d times as likely to win, where $d = r_2 - r_1$. State: the players still in, in bracket order.

```
first_round = constant(1, 8, 4, 5, 2, 7, 3, 6)

@conditional_kind
def next_round(players):          # works for 8, 4 or 2 players
    k = Kind.empty
    for i in range(0, len(players), 2):
        r1, r2 = players[i], players[i + 1]
        k = k * choice(r1, r2, as_quantity('1.15') ** (r1 - r2)) # weight 1 on r1
    return k

winner = evolve(first_round, next_round, 3) # three rounds: 8 to 4 to 2 to 1
as_float(E(winner)), as_float(E(winner ^ (__ > 4))), winner.kernel(1), winner.kernel(8)
# (3.151837905582001, 0.24561554132156538, 0.2652034184779578, 0.036741574915144244)
```

Mouse Escapes. Three steps to climb, failure twice as likely as success on each attempt, and the cat arrives after the 16th attempt. State: the step.

```
move = conditional_kind({
    0: choice(0, 1, '1/2'),          # stay on the floor, or climb to step 1
    1: choice(0, 2, '1/2'),          # fall back, or climb
    2: choice(1, 3, '1/2'),          # escaped: frozen
    3: constant(3),
})
state = evolve(constant(0), move, 16)
state
# 0: 0.39326  1: 0.17758  2: 0.061183  3: 0.36797
as_float(E(state ^ (__ == 3)))      # probability the mouse escapes, about 0.368
# 0.36797378829388655
```

Hunter's Success. Eight hunters shoot independently, and hunter i hits with probability h_i . H is the number of hits. The Kind of H comes from a running sum, one hunter at a time. Its expectation needs no Kind at all: by additivity it is $h_0 + \dots + h_7$.

```
skill = [0.1, 0.1, 0.1, 0.1, 0.1, 0.7, 0.8, 0.9]
hits = constant(0)
for h in skill:
    hits = hits * binary(h) ^ Sum    # add one hunter to the running count

as_float(E(hits)), sum(skill)
# (2.9, 2.9)
[round(as_float(E(hits ^ (__ > c))), 4) for c in (0, 2, 4)] # P(H > 0), P(H > 2), P(H > 4)
# [0.9965, 0.6745, 0.0445]
number_of_hits = Sum(independent_join([binary(h) for h in skill])) # lecture 13: one join, then Sum
Kind.equal(number_of_hits, hits)
# True
hs = symbols('h_0 ... h_2')        # three hunters with symbolic weights
E(independent_join([weighted_as(0, 1, weights=[1 - w, w]) for w in hs]) ^ Sum)
# h_0 + h_1 + h_2
```

8 Homework 4

The six problems, in line with the posted solutions. Problems 2, 3, 5 and 6 use data and helpers from `hw4.py` and `hw4_tcboo.py`, so start the playground in the folder that holds them. The code below ran on those files when this guide was built, and wherever the posted solution prints the same quantity the two agree digit for digit. One computation takes about half an hour, the null simulation in **8.5**, so it was run once and its result is recorded there. Arithmetic by hand, closed forms and remarks that go beyond the posted solutions are this guide's additions, and their numbers are checked by code here.

8.1 Problem 1: Which Coin?

A bucket holds 9550 balanced coins, 442 coins weighted to show heads with probability $2/3$, and 8 two-headed coins. One coin is picked at random and flipped 10 times. Two stages, then Bayes's Rule: join, update with the observed number of heads, project onto the coin type.

```
selected_type = weighted_as(0, 1, 2, weights=[9550, 442, 8])      # 0 balanced, 1 weighted, 2 two-headed
heads_in_ten_flips = conditional_kind({
    0: choice(0, 1) ** 10 ^ Sum,                                # or fast_join_pow(Sum, choice(0, 1), 10)
    1: weighted_as(0, 1, weights=['1/3', '2/3']) ** 10 ^ Sum,
    2: constant(10),
})
coin_and_heads = selected_type >> heads_in_ten_flips            # the Data Kind: <coin type, heads>
CoinType = Proj[1]
NumHeads = Proj[2]
CoinType(coin_and_heads | (NumHeads == 10))                      # (a)
#      ,---- 0.37318 ---- 0
# <>  -+---- 0.30671 ---- 1
#      `---- 0.32011 ---- 2
Kind.equal(bayes(10, selected_type, heads_in_ten_flips), CoinType(coin_and_heads | (NumHeads == 10)))
# True
CoinType(coin_and_heads | (NumHeads == 7))                      # (b)
# 0: 0.90684    1: 0.093163
CoinType(coin_and_heads | (NumHeads == 9))
# 0: 0.70875    1: 0.29125
```

ANSWER (a) Given 10 heads the coin is balanced, weighted or two-headed with probability 0.37318, 0.30671, 0.32011. By hand, each count times the probability of 10 heads: $9550 \cdot (1/2)^{10} = 9.326$, $442 \cdot (2/3)^{10} = 7.665$, $8 \cdot 1 = 8$, then divide by their sum.

(b) Given 7 heads: 0.90684 balanced and 0.093163 weighted. The two-headed coin is ruled out, and the base rate of balanced coins overwhelms the higher than typical number of heads. Given 9 heads the evidence is stronger but still cannot overcome the base rate: 0.70875 and 0.29125.

Naming the projections, `CoinType = Proj[1]`, lets the expression read like the sentence it answers.

8.2 Problem 2: Obscured Words

A five-letter word is chosen with weight equal to its frequency. Each letter is then kept with probability 0.7 and otherwise replaced by an underscore, independently. We see the obscured word and want the Kind of the intended word. This is a noisy channel: stage 1 chooses the word, stage 2 is a conditional Kind for what we observe given the word. Words are tuples of character codes. The dictionary `five_letters` and the two converters come from `hw4.py`, and the rest is the posted code.

```
from hw4 import *                                              # five_letters, word_to_tuple, tuple_to_word and the rest
words_kind = weighted_pairs((word_to_tuple(word), freq) for word, freq in five_letters.items())
size(words_kind)                                              # the number of words in the dictionary
# 3321
words_kind.kernel(word_to_tuple('fresh'))                    # (a) the weight on one word
# 0.0007796800617809579
```

```

@statistic_factory
def obscure(word):
    obscured = ord('_')
    @statistic(codim=5, dim=5)
    def obscure_word(noise):
        return [c if noise[k] == 1 else obscured for k, c in enumerate(word)]
    return obscure_word

@conditional_kind(codim=5, target_dim=5)
def noisy_channel(intended_word):
    noise_pattern = binary(0.7) ** 5
    return noise_pattern ^ obscure(intended_word)

OriginalWord = Proj[:6]
ObscuredWord = Proj[6:]

def observe_obscured(obscured_word):
    obscured = word_to_tuple(obscured_word)
    return OriginalWord((words_kind >> noisy_channel) | (ObscuredWord == obscured))

k_are = observe_obscured('__are')
k_are ^ tuple_to_word
#      ,---- 0.24618 ----- aware
#      |---- 0.0020891 ----- blare
#      |---- 0.000047842 ----- chare
#      |---- 0.018834 ----- flare
#      |---- 0.026919 ----- glare
# <> +---- 0.00019137 ----- quare
#      |---- 0.036934 ----- scare
#      |---- 0.47350 ----- share
#      |---- 0.0040665 ----- snare
#      |---- 0.061588 ----- spare
#      `---- 0.12965 ----- stare
k_a_e = observe_obscured('_a_e_')
size(k_a_e)
# 59
sorted(((round(float(w), 5), tuple_to_word(v)) for v, w in k_a_e), reverse=True)[:4]
# [(0.41309, 'water'), (0.24693, 'paper'), (0.05869, 'label'), (0.05631, 'panel')]

```

- **(a)** The posted weights are 0.00077968 for fresh, 0.0063832 for point and 0.0058953 for state. `k.kernel(value)` gives the weight on one value after the weights are scaled to add up to 1.
- **(b)** `noisy_channel(word_to_tuple('chain')) ^ tuple_to_word` has 32 branches, one for each choice of letters to hide. A pattern that keeps j letters has weight $0.7^j \cdot 0.3^{5-j}$: 0.16807 for `chain` itself, 0.07203 with one letter hidden, down to 0.00243 for ____.
- **(c)** The observation eliminates every word that cannot produce the pattern we saw. Each surviving word needs the same noise pattern, so that factor is common to all of them and cancels: the remaining words keep their relative frequencies. For `__are`, `share` is about twice as likely as `aware` because its count is about twice as large.
- These are the posted results, digit for digit. For `_a_e_` the posted tree has the same 59 words, led by `water` 0.41309, `paper` 0.24693, `label` 0.058687 and `panel` 0.056309.
- **Speed.** The join `words_kind >> noisy_channel` has $3321 \cdot 32 = 106,272$ branches, and `observe_obscured` rebuilds it on every call. If you need several observations, store `joined = words_kind >> noisy_channel` once and update that. `bayes(word_to_tuple('__are'), words_kind, noisy_channel)` gives the same Kind in one call and takes about as long.
- `k ^ tuple_to_word` shows the words even though `tuple_to_word` is not a statistic. Use it for display only.

8.3 Problem 3: The Robot's Journey

A robot starts at node 1 of a graph and moves along edges, chosen by edge weight, never returning to a node it has visited, until it reaches the end node 22. The next move depends on everything visited so far, so the whole path is kept with joins. Once at 22 the step returns `constant(22)`, which repeats 22 and keeps every path the same length. `robot_graph` and `robot_weights` come from `hw4.py` (24 nodes, loaded by the import in 8.2). A move takes exactly 10 minutes, and the handout's observation is that the robot "arrives at node 22 exactly 2 hours after it started", so there are 12 joins.

```
ROBOT_END = 22
robot_start = constant(1)

@conditional_kind
def make_step(path_so_far):
    current = path_so_far[-1]
    if current == ROBOT_END:
        return constant(ROBOT_END) # the robot stops
    possible = robot_graph[current]
    seen = set(path_so_far)
    allowed = [node for node in possible if node not in seen]
    weights = [robot_weights[current, node] for node in allowed]
    return weighted_as(allowed, weights=weights)

robot_paths = robot_start
for _ in range(12): # about 15 seconds in all
    robot_paths = robot_paths >> make_step

size(robot_paths) # every path of 12 moves
# 28211
robot_paths = robot_paths | (Proj[-1] == ROBOT_END) # at the end node after the last move
size(robot_paths), dim(robot_paths)
# (5533, 13)
robot_thresh = sorted(robot_paths.weights.values())[-3] # the third largest weight
best_paths = [k for k in robot_paths.weights if robot_paths.weights[k] >= robot_thresh]
for path in best_paths:
    print(path)
# <1, 2, 4, 8, 16, 20, 22, 22, 22, 22, 22, 22, 22>
# <1, 2, 4, 8, 23, 22, 22, 22, 22, 22, 22, 22, 22>
# <1, 3, 7, 15, 24, 22, 22, 22, 22, 22, 22, 22, 22>

robot_paths ^ Contains(11) # did the robot pass through node 11?
# 0: 0.71773 1: 0.28227

@condition_factory
def move_from_a_to_b(a, b):
    "tests whether a path transitions from a to b"
    @condition
    def ab_move(path):
        for ind, node in enumerate(path):
            if node == a and ind + 1 < len(path) and path[ind + 1] == b:
                return True
        return False
    return ab_move

robot_paths ^ move_from_a_to_b(4, 10)
# 0: 0.96588 1: 0.034118

from frplib.exceptions import StatisticError # the playground does not load this name

@statistic
def penultimate_node(path): # the node just before the first 22
    end_ind = index_of(22, path) # -1 if 22 is not on the path
    if end_ind == -1:
        raise StatisticError('end node 22 never reached by robot')
    return path[end_ind - 1]

robot_paths ^ penultimate_node
# 20: 0.31466 21: 0.19248 23: 0.26933 24: 0.22353
```

```

exactly_12 = robot_paths | (Proj[12] != ROBOT_END)      # a narrower reading: not there after 11 moves
size(exactly_12)
# 2577
exactly_12 ^ Contains(11)
# 0: 0.49970    1: 0.50030
exactly_12 ^ Contains(4, 10)
# 0: 0.94576    1: 0.054237
exactly_12 ^ penultimate_node
# 20: 0.25886   21: 0.22960   23: 0.38468   24: 0.12685

```

- Down to `robot_paths ^ penultimate_node` these are the posted results: the three heaviest paths (1, 2, 4, 8, 16, 20, 22), (1, 2, 4, 8, 23, 22) and (1, 3, 7, 15, 24, 22) (the trailing 22s are the padding), 0.28227 for node 11, 0.034118 for the move from 4 to 10, and 0.31466, 0.19248, 0.26933, 0.22353 for the node before the end.
- `Contains(4, 10)` is the built-in for "a 4 immediately followed by a 10" and gives the same Kind as the hand-written condition factory.
- A path whose neighbors have all been visited gets the empty Kind from `weighted_as([], weights=[])`, and `>>` drops that path and rescales the others together. Nothing computed after the update changes. Your homework version kept such a path where it was instead, which gives 29,244 paths before the update where this code has 28,211, and the same answers after it.
- The guard in `penultimate_node` matters. Without it a path that never reaches 22 would raise no error: `index_of` returns `-1`, and `path[-2]` is the second to last node, a wrong answer that looks fine. After the update every path contains 22, so the guard never fires here.
- **Two readings of the observation.** The handout says the robot "arrives at node 22 exactly 2 hours after it started", and the posted text reads that as a path that is 12 steps long. The posted code does something wider. Finished paths are padded with 22s, so `Proj[-1] == ROBOT_END` after 12 joins also keeps every path that got to 22 sooner and waited: its own three heaviest paths take 5 and 6 moves, which is 50 or 60 minutes. So the posted numbers answer "at node 22 within 2 hours". Arriving at exactly 2 hours means first reaching 22 on move 12, the narrower condition `And(Proj[13] == 22, Proj[12] != 22)`, the last four lines above. Then 2577 paths remain, the robot passed through node 11 with probability 0.5003 and moved from 4 to 10 with probability 0.054237, and the node before the end is 20, 21, 23, 24 with weights 0.25886, 0.22960, 0.38468, 0.12685. Your own homework 4 answer used this narrower reading, which is the one the handout's sentence describes. If the difference matters to you, ask the instructor which was intended. On the exam, write the observation as a condition in words before you compute.

8.4 Problem 4: Urn Game

An urn holds r red and g green balls. In a round, balls are drawn one at a time until a ball of a different color from the round's first ball appears, and that ball goes back. A round that starts with a single color takes every ball, and the game ends. You win if the last ball drawn is green, and the question is how many balls of each color to start with. The state is (reds, greens). The round is itself a small system that is evolved inside the conditional Kind. (The posted solution names it `round`, which hides Python's own `round` for the rest of the session, so it is `urn_round` here.)

```

r, g = symbols('r g')
draw_one(r, g)                                     # (c) one draw, with symbols; draw_one is in hw4.py
#      ,---- r/(g + r) ---- <-1 + r, g>
# <> -|
#      `---- g/(g + r) ---- <r, -1 + g>

```

```

@conditional_kind(codim=2)
def urn_round(state):
    red, green = state
    if red == 0 and green == 0:
        return constant(state)
    if red == 0 or green == 0:
        return constant(0, 0)
    @conditional_kind(codim=3)
    def in_round_draw(delta):
        delta_red, delta_green, done = delta
        if done == 1:
            return constant(delta)
        if delta_red == 0:
            return weighted_as((delta_red, delta_green, 1), (delta_red, delta_green - 1, 0),
                               weights=[red + delta_red, green + delta_green])
        return weighted_as((delta_red - 1, delta_green, 0), (delta_red, delta_green, 1),
                           weights=[red + delta_red, green + delta_green])
    next_state = weighted_as((-1, 0, 0), (0, -1, 0), weights=[red, green])
    for _ in range(max(red + 1, green + 1)):
        next_state = in_round_draw // next_state
    return next_state ^ Proj[1, 2] ^ (__ + state)

urn_round(2, 3)
# <0, 3>: 0.1 <1, 3>: 0.3 <2, 0>: 0.100000 <2, 1>: 0.2 <2, 2>: 0.3
urn_round(10, 20) | Or(Proj[1] == 0, Proj[2] == 0)
# ,---- 1/2 ---- <0, 20>
# <> -|
# ,---- 1/2 ---- <10, 0>
urn_round.joined(32, 11) | Or(Proj[3] == 0, Proj[4] == 0)
# ,---- 1/2 ---- <32, 11, 0, 11>
# <> -|
# ,---- 1/2 ---- <32, 11, 32, 0>

```

ANSWER (a) At the start of the final round the balls all have the same color. Otherwise the round could end with a ball going back, and it would not be final.

(b) From (r, g) with both positive, a round leads to one of $(r - 1, g)$, ..., $(0, g)$, $(r, g - 1)$, ..., $(r, 0)$. With $r = 0$ or $g = 0$ it leads to $(0, 0)$.

(c) One draw: a red with weight $r/(r + g)$ and a green with weight $g/(r + g)$, since each of the $r + g$ balls is equally likely. `draw_one` is the conditional Kind of the urn after one draw given the state (r, g) : `weighted_as((r - 1, g), (r, g - 1), weights=[r, g])`.

(d) `urn_round` above. A round has its own contingencies, so the conditional Kind defines a small system inside itself, with state (change in reds, change in greens, done), and evolves it on every call. It then keeps the two changes and adds them to the state the round started from.

(e) Observing that a round is the penultimate one erases every branch except $(r, 0)$ and $(0, g)$, and those two have equal weight whatever r and g are. Drawing all the reds and then a green has probability $r/(r + g) \cdot (r - 1)/(r - 1 + g) \cdots 1/(1 + g) = r! g! / (r + g)!$, and drawing all the greens and then a red has the same value.

(f) Every game with both colors present has a penultimate round, so the final round is all red or all green with probability $1/2$ each. You win with probability $1/2$ whatever the starting numbers, as long as there is at least one ball of each color.

WATCH OUT The posted solution writes the observation in (e) as `round(10, 20) | Or(Proj[3] == 0, Proj[4] == 0)` and prints four-component values. In `frplib` 0.3.10 calling a conditional Kind gives the target alone, with two components here, so that line fails with a dimension error. Use `Proj[1]` and `Proj[2]` on the target, or `urn_round.joined(...)` to keep the input in front, as above.

8.5 Problem 5: Hypothesis Testing 1

(a) For data $(x_1, \dots, x_n)$ from $k \times n$, the path from root to leaf multiplies the weights $e^{\ell(x_1)}, \dots, e^{\ell(x_n)}$, so the log-likelihood is the sum $\ell(x_1) + \dots + \ell(x_n)$. (b) The log of the likelihood ratio is the sum over the data of (log-likelihood under the alternative – log-likelihood under the null). (c) When the Kind of that statistic is out of reach, simulate it: generate many data sets from the null, compute the statistic for each, sort, and use as the threshold the smallest of the top 10% of the simulated values. (d) Reject the null if the observed statistic is above the threshold, otherwise retain it.

The null model is a random graph on 64 nodes with every edge present with probability 0.4. The alternative has two blocks of 32 nodes, with edge probability 0.6 inside a block and 0.2 between blocks. The posted functions, with the helpers and the 128 observed graphs of `hw4.py` (each graph is a tuple of 2016 zeros and ones, one for every possible edge):

```
from frplib.examples.random_graphs import random_graph      # the FRP factory for the null model

def log_likelihood_ratio(graphs):                            # (b)
    ll = 0
    for graph in graphs:
        ll = random_block_graph_log_likelihood(graph, 32, 32, p_in=0.6, p_out=0.2)
        l0 = random_graph_log_likelihood(graph, p=0.4)
        ll += ll - l0
    return ll

def simulate_nulls(iterations, n=128):                      # (c)
    base = random_graph(64, p=0.4)
    lls = []
    for _ in range(iterations):
        data = [clone(base).value for i in range(n)]
        lls.append(log_likelihood_ratio(data))
    lls.sort()
    k = len(lls) // 10
    return (lls[-k], k, lls)

len(rgraph_data), len(rgraph_data[0])
# (128, 2016)
ll_ratio = log_likelihood_ratio(rgraph_data)               # (d) the observed statistic
float(ll_ratio)
# 22169.646528760768
```

ANSWER (c) `thresh, _, _ = simulate_nulls(1000)` is slow, because every simulated data set is 128 graphs and takes more than a second. One run for this guide took 28 minutes on a busy machine and gave a threshold of $-23,736.8$. Its 1000 simulated values ran from $-24,900$ to $-23,403$. The number changes a little from run to run.

(d) The observed statistic is 22,169.6, far above any threshold of that size, so the posted test prints `Reject null`. The posted solution shows the code and not the numbers. Your own homework 4 run had the same 22,169.6.

For scale (this guide's addition). When the null is true the statistic has expectation $-24,013.5$ and standard deviation 224.3, by additivity over the $128 \cdot 2016$ independent edges. The data sit about 206 standard deviations above that. The reason shows in the data: 0.600 of the possible edges inside the blocks are present and 0.201 of those between the blocks, against 0.4 everywhere under the null.

The same four steps on an example small enough to run in seconds, a fair die against a die that shows 6 half the time:

```
null, alt = uniform(1, 2, ..., 6), weighted_as(1, 2, ..., 6, weights=[1, 1, 1, 1, 1, 5])

def llr_rolls(data):                                       # data: a tuple of rolls
    ll = ijoin_log_likelihood(data)
    return ll(alt) - ll(null)

def simulate_null_rolls(iterations, n=30):
    base = frp(null) ** n                                  # an FRP for n rolls when the null is true
    lls = sorted(llr_rolls(x) for x in FRP.sample(iterations, base, summary=False))
    k = len(lls) // 10
    return lls[-k]                                         # the smallest of the top 10% of the values

thresh_rolls = simulate_null_rolls(1000)
thresh_rolls
# -2.4492654135069145
```

```
data = FRP.sample(1, frp(alt) ** 30, summary=False)[0]          # one data set, from the alternative
llr_rolls(data), llr_rolls(data) > thresh_rolls # True means reject the null
# (13.645113710834089, True)
```

- Both numbers in the dice example can change from run to run, because the threshold and the data come from simulation.
- `FRP.sample(n, X, summary=False)` gives the `n` values as a list, which does the job of the posted `[clone(base).value for i in range(n)]`.
- When the Kind of the statistic can be computed, use it instead (3.12).

8.6 Problem 6: There Can Be Only One

Two teams of immortals, each fighter with a power. In a fight between powers c and d the first wins with probability $c/(c + d)$, and the winner takes the loser's power. The state is the tuple of powers, Connor's t fighters first and then Duncan's t , and a fighter is named by its position in that tuple, counting from 0. Below is the template from `hw4_tcboo.py` with the posted lines, marked (b) to (f), filled in. Team, `has_winner` and the strategies `randomly`, `lead_strongest` and `lead_weakest` come from `hw4.py`.

```
def quickening(teams, cfighter, dfighter, winner):
    teams_prime = list(teams)
    total_pow = teams[cfighter] + teams[dfighter]          # (b) the winner takes the loser's power
    if winner == Team.CONNOR:
        teams_prime[cfighter] = total_pow
        teams_prime[dfighter] = 0
    else:
        teams_prime[cfighter] = 0                          # winner == Team.DUNCAN
        teams_prime[dfighter] = total_pow
    return as_vec_tuple(teams_prime)

def battle(initial_teams, connor_strategy, duncan_strategy):
    num_immortals = len(initial_teams)
    t = num_immortals // 2
    strategies = connor_strategy * duncan_strategy          # the two fighters, given the teams
    initial_state = constant(initial_teams)                 # (c)
    TeamPowers = Fork(Sum(Proj[(t + 1):]), Sum(Proj[(t + 1):])) # (d)
    @conditional_kind(codim=num_immortals + 2, target_dim=num_immortals)
    def fight(teams_with_matchup):
        teams = teams_with_matchup[:num_immortals]
        cfighter, dfighter = teams_with_matchup[-2], teams_with_matchup[-1]
        cpow, dpow = teams[cfighter], teams[dfighter]
        return weighted_as(                                  # (e)
            quickening(teams, cfighter, dfighter, Team.CONNOR),
            quickening(teams, cfighter, dfighter, Team.DUNCAN),
            weights=[cpow, dpow]
        )
    @conditional_kind(codim=num_immortals, target_dim=num_immortals)
    def one_round(teams):
        if has_winner(teams):                                # (f)
            return constant(teams)
        the_match = constant(teams) >> strategies
        return fight // the_match
    return (initial_state, one_round, TeamPowers)

teams = (21, 0, 0, 7, 14, 7)                               # Connor's side has 21, 0, 0 and Duncan's has 7, 14, 7
init, next_round, TeamPowers = battle(teams, randomly(Team.CONNOR, 6), lead_strongest(Team.DUNCAN, 6))
TeamPowers(teams)
# <21, 28>
next_round(teams)                                           # one round: 21 fights 14
# <0, 0, 0, 7, 35, 7>: 2/5   <35, 0, 0, 7, 0, 7>: 3/5
n = dim(init)
delta = TeamPowers(Proj[(n + 1):]) - TeamPowers(Proj[:n + 1])) # (g) the change over one round
E(next_round.joined(teams) ^ delta)                        # (g) predicted
# <0, 0>
evolve(init, next_round, dim(init) - 1) ^ TeamPowers       # (h) at most 2t - 1 = 5 rounds
# <0, 49>: 4/7   <49, 0>: 3/7
teams8 = (4, 14, 2, 1, 2, 7, 8, 4)                         # 21 against 21
```

```
init8, round8, Powers8 = battle(teams8, lead_weakest(Team.CONNOR, 8), lead_weakest(Team.DUNCAN, 8))
evolve(init8, round8, 7) ^ Powers8
# <0, 42>: 1/2    <42, 0>: 1/2
```

ANSWER (a) With t fighters a side there are at most $2t - 1$ rounds, because one immortal is eliminated in each fight. If the teams are full and the winners alternate teams, only one immortal remains.

(b) The update moves the loser's power to the winner and sets the loser's power to 0. **(c)** The initial state is `constant(initial_teams)`. **(d)** `TeamPowers` adds up the powers on each side. **(e)** `fight` knows the two fighters, so their powers are the weights on the two possible quickening results. **(f)** If there is a winner the state stays as it is. Otherwise `constant(teams) >> strategies` is the Kind of (teams, the two fighters), which is what `fight` takes, and `//` gives the Kind of the next state.

(g) In a fight of c against d , the first team gains d with probability $c/(c + d)$ and loses c with probability $d/(c + d)$, so the expected change is 0 for both teams: `predicted` is $(0, 0)$.

(h) At the end the team powers are $(c + d, 0)$ or $(0, c + d)$, where c and d are the teams' total powers at the start. Their expectation is still (c, d) , so if p is the probability that the second team wins, $d = p(c + d)$ and $p = d/(c + d)$. The strategies do not matter. Above, 21 against 28 ends at $(0, 49)$ with probability $28/49 = 4/7$, and 21 against 21 is even. Both match your homework run.

The single fight of (g) with symbols, for any c and d :

```
c, d = symbols('c d')
one_fight = weighted_as((c + d, 0), (0, c + d), weights=[c, d])    # the two fighters' powers afterwards
one_fight
#      , ---- c/(c + d) ---- <c + d, 0>
# <> -|
#      ---- d/(c + d) ---- <0, c + d>
E(one_fight)                                                       # the same as before the fight: <c, d>
# <c, d>
```

- `strategies = connor_strategy * duncan_strategy` is in the template: the independent mixture of two conditional Kinds is the conditional Kind of both choices, of type $2t \rightarrow 2t + 2$.
- **A remark of this guide, not in the posted solution.** When each step leaves an expectation unchanged and the system must end in one of two known states, that expectation gives the probability of each ending. The fair gambler's ruin is the same argument (3.9): the expected winnings stay 0, so the probability of reaching $+b$ before $-w$ is $w/(w + b)$.

9 Homework 5

All five problems, in line with the posted solutions. Problems 3, 4 and 5 use objects from `hw5.py`, so start the playground in the folder that holds it. Their numbers were computed on that file for this guide and agree with the posted solutions. Two of those computations take minutes, the Fruit Pattern Kind in 9.3 and the long runs tests in 9.5, so they were run once and their results are recorded here. Arithmetic by hand, closed forms and remarks that go beyond the posted solutions are this guide's additions, and their numbers are checked by code here.

9.1 Problem 1: Bayes practice (WidgetCorp)

Three stages: supplier, then resin given the supplier, then quality given the resin. The posted solution builds the Data Kind with a join of all three, and then gets each conditional Kind by updating with an observation and projecting. The third table has to be upgraded to take (supplier, resin), even though the supplier does not change its targets. (Once the resin is known, quality no longer depends on the supplier.)

```
supplier = weighted_as(1, 2, 3, 4, weights=['1/6', '1/4', '1/3', '1/4'])
resin = conditional_kind({
    1: weighted_as(1, 2, 3, weights=['1/6', '1/3', '1/2']),
    2: weighted_as(1, 2, 3, weights=['3/5', '1/5', '1/5']),
    3: weighted_as(1, 2, 3, weights=['1/10', '8/10', '1/10']),
    4: weighted_as(1, 2, 3, weights=['1/4', '1/2', '1/4']),
})
```

```

quality = conditional_kind({
    # given the resin: 1 low, 2 medium, 3 high
    1: weighted_as(1, 2, 3, weights=['1/6', '1/2', '1/3']),
    2: weighted_as(1, 2, 3, weights=['2/15', '4/15', '3/5']),
    3: weighted_as(1, 2, 3, weights=['1/50', '1/5', '39/50']),
})
quality_sr = conditional_kind(({sup, res}): quality.target(res) # the third table, upgraded
                             for sup in irange(1, 4) for res in irange(1, 3)})
data_kind = supplier >> resin >> quality_sr # the Data Kind: <supplier, resin, quality>
size(data_kind), dim(data_kind)
# (36, 3)

@conditional_kind(codim=1)
def quality_given_supplier(sup):
    return Proj[3] @ data_kind | (Proj[1] == sup)

@conditional_kind(codim=1)
def supplier_given_quality(qual):
    # Bayes's Rule: the same two steps, other components
    return Proj[1] @ data_kind | (Proj[3] == qual)

supplier_given_quality.target(3) # given high quality
# 1: 0.18933 2: 0.20940 3: 0.34685 4: 0.25442
quality_given_supplier.target(2)
# 1: 0.13067 2: 0.39333 3: 0.476
[[round(quality_given_supplier(sup).kernel(q), 4) for q in irange(1, 3)] for sup in irange(1, 4)]
# [[0.0822, 0.2722, 0.6456], [0.1307, 0.3933, 0.476], [0.1253, 0.2833, 0.5913], [0.1133, 0.3083, 0.5783]]
[[round(supplier_given_quality(q).kernel(sup), 4) for sup in irange(1, 4)] for q in irange(1, 3)]
# [[0.1176, 0.2804, 0.3587, 0.2432], [0.1439, 0.3119, 0.2996, 0.2445], [0.1893, 0.2094, 0.3469, 0.2544]]

```

Quality given supplier	Low	Medium	High	Supplier given quality	1	2	3	4
Supplier 1	0.0822	0.2722	0.6456	Share of batches	0.1667	0.2500	0.3333	0.2500
Supplier 2	0.1307	0.3933	0.4760	Given low	0.1176	0.2804	0.3587	0.2432
Supplier 3	0.1253	0.2833	0.5913	Given medium	0.1439	0.3119	0.2996	0.2445
Supplier 4	0.1133	0.3083	0.5783	Given high	0.1893	0.2094	0.3469	0.2544

ANSWER **Quality given supplier** is the left table, `quality_given_supplier`. One entry by hand, low quality from supplier 2: average the quality Kinds over that supplier's resins, $3/5 \cdot 1/6 + 1/5 \cdot 2/15 + 1/5 \cdot 1/50 = 49/375 = 0.1307$.

Supplier given quality is the right table, `supplier_given_quality`: Bayes's Rule for each observed quality. Given low, the exact weights are $2/17$, $882/3145$, $1128/3145$, $9/37$. One entry by hand, supplier 2 given low: $(1/4 \cdot 49/375)$ divided by the overall probability of low, $629/5400$, which is 0.2804 .

The posted conclusion. Supplier 1 is the most likely to produce a high-quality widget and the least likely to produce a low-quality one. But WidgetCorp gets a smaller share of its widgets from supplier 1, so a high-quality widget is still most likely to have come from supplier 3, the main supplier. This suggests getting more widgets from supplier 1 and fewer from supplier 2.

Why, from the tables (this guide's reading). Supplier 2 provides 25% of the batches but 28% of the low-quality and 31% of the medium-quality widgets, because it uses the most green resin (3/5), the resin with the worst record. Supplier 1 is the reverse, because half of its resin is red.

- The posted solution names the categories with numbers, GREEN, BROWN, RED = 10, 20, 30 and LOW, MEDIUM, HIGH = 100, 200, 300, or with symbols, `symbols('green brown red')`. Either avoids the limitations of strings. With symbols the trees print the names, sorted alphabetically (high, low, medium). Here they are 1, 2, 3.
- The symmetry is the point: both conditional Kinds are an update with an observation followed by a projection. The second one is what we call Bayes's Rule.

A shorter route to the same tables. Quality depends on the supplier only through the resin, so average the resin out with `//`, then apply `bayes` once for each quality:

```
q_given_s = conditional_kind({sup: quality // resin(sup) for sup in irange(1, 4)})
s_given_q = conditional_kind({q: bayes(q, supplier, q_given_s) for q in irange(1, 3)})
all(Kind.equal(q_given_s(sup), quality_given_supplier(sup)) for sup in irange(1, 4))
# True
all(Kind.equal(s_given_q(q), supplier_given_quality(q)) for q in irange(1, 3))
# True
```

9.2 Problem 2: Funny Dice

Alice's die has faces 2, 2, 4, 4, 9, 9, Bob's has 1, 1, 6, 6, 8, 8 and Carlos's has 3, 3, 5, 5, 7, 7. The higher roll wins a match, and there are no ties. Repeated values in `uniform` merge, so each die is uniform on its three values.

```
Alice = uniform(2, 2, 4, 4, 9, 9)
Bob = uniform(1, 1, 6, 6, 8, 8)
Carlos = uniform(3, 3, 5, 5, 7, 7)
show([E(Alice), E(Bob), E(Carlos)])           # (a)
# [
#   5
#   5
#   5
# ]
player1_wins = (Proj[1] > Proj[2])
E(Alice * Bob ^ player1_wins)                 # (b)
# 5/9
E(Bob * Carlos ^ player1_wins)
# 5/9
E(Carlos * Alice ^ player1_wins)
# 5/9
bob_chooses = arbitrary(1, 2, 3, names=['b1', 'b2', 'b3']) # (c): however Bob chooses

@conditional_kind(codim=1)
def danielle(bobs_choice):
    if bobs_choice == 1:
        return Alice * Carlos
    if bobs_choice == 2:
        return Bob * Alice
    return Carlos * Bob
# the Kind of <Bob's roll, Danielle's roll>
# Bob takes Alice's die, Danielle answers with Carlos's
# Bob takes Bob's die, Danielle answers with Alice's
# Bob takes Carlos's die, Danielle answers with Bob's

danielle_wins = (Proj[3] > Proj[2])
E(bob_chooses >> danielle ^ danielle_wins)
# 5/9
# the join is <choice, Bob's roll, Danielle's roll>
```

ANSWER (a) All three expectations are 5: $(2 + 4 + 9)/3 = (1 + 6 + 8)/3 = (3 + 5 + 7)/3$. The dice have the same risk-neutral price, so on that basis there is no reason to prefer one.

(b) A roll of two dice is an independent join, and a statistic checks who has the bigger roll. Each probability is $5/9$. By hand, count the winning pairs out of 9. Alice over Bob: $2 > 1$, $4 > 1$, $9 > 1$, $9 > 6$, $9 > 8$. Bob over Carlos: $6 > 3$, $6 > 5$, $8 > 3$, $8 > 5$, $8 > 7$. Carlos over Alice: $3 > 2$, $5 > 2$, $5 > 4$, $7 > 2$, $7 > 4$. Alice tends to beat Bob, Bob tends to beat Carlos, and Carlos tends to beat Alice: no die is best.

(c) Going second is the advantage. Whatever die Bob takes, Danielle takes the die that beats it. Bob's choice is given arbitrary weights b_1 , b_2 , b_3 , so that if the result depended on how he chooses it would show in the answer. It does not: her probability of winning is $5/9$ with no symbol left in it.

9.3 Problem 3: Fruit Pattern

A long chain described with the pattern language: apricots and mangoes fall (independently), squirrels arrive (their number depends on the fruit fallen), some fruit rots, and the squirrels eat some of the fresh fruit. The child gathers what is left, fresh or rotten. Fresh mangoes sell for 70 cents and fresh apricots for 20, and the whole batch is rejected if more than 1/4 of it is rotten. The pieces come from `hw5.py`: the Kinds `apricot_count` and `mangoes_count` and the conditional Kinds `squirrels_count`, `rot` and `not_eaten`. (The handout's table calls the third one `squirrel_count`. The file and the posted solution say `squirrels_count`.) The lines below are the posted solution's.

```
from hw5 import *
data_kind = (apricot_count * mangoes_count) >> squirrels_count >> rot >> not_eaten      # (b)
fruits_gathered = Sum(Proj[4, 5, 6, 7])      # rotten and fresh, both fruits
fresh_fruits_gathered = Sum(Proj[6, 7])
too_many_rotted = Sum(Proj[4, 5]) > fruits_gathered / 4
money_earned = IfThenElse(too_many_rotted, 0, 20 * Proj[6] + 70 * Proj[7])
money_c = money_earned(data_kind | (fruits_gathered == 12))      # (c)
fresh_d = fresh_fruits_gathered(data_kind | And(money_earned > 300, money_earned <= 400)) # (d)
```

Recorded result	Kind, as value: weight	Expectation
(c) <code>money_c</code> , cents earned given 12 fruits gathered	0: 0.39374, 200: 0.020280, 240: 0.0024633, 300: 0.12456, 340: 0.019506, 400: 0.21878, 440: 0.037269, 500: 0.10243, 540: 0.028748, 600: 0.038302, 640: 0.0077854, 700: 0.0035360, 740: 0.0024593, 840: 0.00013899	251.675
(d) <code>fresh_d</code> , fresh fruit gathered given earnings above 300 and at most 400	6: 0.35579, 8: 0.26004, 10: 0.26438, 12: 0.10670, 14: 0.013095	8.323

These lines were run once on your `hw5.py` for this guide and the results recorded, because they are slow: the join stages have 144, 720, 40,500 and 232,860 branches, the Data Kind took about 5 minutes to build on a busy machine, and the two updates took another 90 seconds between them. Budget for that before you start a Kind of this size in the exam.

ANSWER (a) The Data FRP has dimension 7: (apricots fallen, mangoes fallen, squirrels, rotten apricots, rotten mangoes, fresh apricots left uneaten, fresh mangoes left uneaten). Its Element FRPs are the two fruit counts and the FRPs behind each conditional stage.

(b) The first line, a join of the stages in the order they happen. The FRP expression has the same shape, with each Kind replaced by its FRP: `(Apricot_count * Mangoes_count) >> Squirrels_count >> Rot >> Not_eaten`. Aligned, the two expressions are the parallel in [7.1](#).

(c) Update with the observation that 12 fruits were gathered, then transform to the money earned: the expectation is about 251.675 cents, and the Kind puts weight 0.39374 on earning nothing.

(d) The same pattern with the roles changed: update with the observation on the money, transform to the fresh fruit. The expectation is about 8.323.

Both Kinds and both expectations match the posted solution digit for digit.

What to copy from it. A statistic can be named once and then used both in an observation and as a transform. A condition can compare two statistics, and `m > n / 4` is written so that nothing is divided by a count that might be 0.

9.4 Problem 4: Markov likelihood

The system starts at 1 or 2 with equal weight. At each step it switches to the other symbol with probability u and stays with probability $1 - u$. We keep the whole history for 16 steps and observe (1, 1, 2, 2, 2, 1, 1, 1, 1, 2, 2, 2, 2, 1, 1, 1).

```
path = (1, 1, 2, 2, 2, 1, 1, 1, 1, 2, 2, 2, 2, 1, 1, 1)
switches = sum(a != b for a, b in zip(path, path[1:]))
stays = len(path) - 1 - switches
switches, stays
# (4, 12)
```

```
def likelihood(u):
    "weight of the observed path: the start, then one factor for each transition"
    start = choice(1, 2)
    move = conditional_kind({1: weighted_as(1, 2, weights=[1 - u, u]),
                               2: weighted_as(1, 2, weights=[u, 1 - u])})
    L = start.kernel(path[0])
    for prev, cur in zip(path, path[1:]):
        L = L * move(prev).kernel(cur)
    return L

likelihood(0.25), 0.5 * 0.25 ** 4 * 0.75 ** 12          # the code and the formula agree
# (6.186787504702806e-05, 6.186787504702806e-05)
max((i / 100 for i in range(1, 100)), key=likelihood)  # the best u on a grid
# 0.25
```

ANSWER (a) From either state the system changes state with probability u and otherwise stays. The next symbol depends only on the current one (the conditional Kind reads only the last component), which is the Markov property. The possible values are the tuples of 1s and 2s of dimension equal to the number of steps plus 1, for the initial state.

(b) `kind(Data)` is a repeated join, and compacting it multiplies the normalized weights along each path from root to leaf: $1/2$ for the start, a factor u at each step where the state changes and $1 - u$ at each step where it stays. The observed tuple has 4 changes in 16 steps, so $L(u) = (1/2) \cdot u^4 \cdot (1 - u)^{12}$ and $\ell(u) = \log(1/2) + 4 \log u + 12 \log(1 - u)$.

(c) Setting $\ell'(u) = 4/u - 12/(1 - u)$ to zero gives $(1 - u)/u = 3$, so $\hat{u} = 1/4$.

In general, for an observed sequence of length $n + 1$ with c state changes: $L(u) = (1/2) \cdot u^c \cdot (1 - u)^{n-c}$, $\ell(u) = c \log u - (n - c) \log(1 - u)$, and $\hat{u} = c/n$, the fraction of steps on which the state changed.

`hw5.py` has the system as FRPs: `Init, Move = markov_system(u)` gives the starting FRP and the conditional FRP for one step, and the handout's `Data = repeated_joins(Init, Move, 16)` joins the step 16 times (always 16, whatever its third argument says). Each join doubles the work, so building `Data` takes a minute or more, and the problem does not need it.

WATCH OUT Do not build `kind(Data)` to answer this. Sixteen joins give $2^{17} = 131,072$ branches and take more than a minute to compute, and the problem says you need not. Multiply the weights along the one observed path.

9.5 Problem 5: the runs test

The method is worked in **3.12** on ten bits, including the posted threshold search. Here are the results for the homework's data, with `runs3`, `seq50s` and `opt64s` from `hw5.py`. The statistic compares a sequence's runs counts (total runs, runs of length 1, of 2, of 3) with their expectations for fair bits.

```
from hw5 import *
erc_50 = (25.5, 13.0, 6.375, 3.125)          # runs3, seq50s, opt64s
psi50 = runs3 ^ ChiSquare(erc_50)           # k50.expectation: expected runs counts for 50 fair bits
for seq in seq50s:                          # the test statistic
    print(runs3(seq), psi50(seq))           # (g) the runs counts and the statistic
# <25, 12, 9, 3> <1.172609351432880844645550528>
# <50, 50, 0, 0> <138.3469079939668174962292609>
# <23, 9, 8, 3> <1.895082956259426847662141780>
# <22, 1, 14, 7> <25.48241327300150829562594269>
# <26, 10, 11, 3> <4.062503770739064856711915535>
# <29, 20, 1, 6> <11.42648567119155354449472096>
# <30, 16, 10, 2> <3.952699849170437405731523378>

erc_64 = (32.5, 16.5, 8.125, 4.0)
print(runs3(opt64s), (runs3 ^ ChiSquare(erc_64))(opt64s)) # (h)
# <50, 36, 14, 0> <40.71660839160839160839160839>
```

ANSWER (a) For ten fair bits the expected runs counts are (5.5, 3, 1.375, 0.625) and the threshold is 6.05. (The posted search uses $\alpha = 0.10$ there. With $\alpha = 0.075$ it stops at the same 6.05.) With that threshold the test rejects with probability 0.0742 when the null is true, and with probability 0.1926 for the posted alternative, `(binary(0.7) * binary(0.3)) ** 5`. The posted solution estimates those two rates from 10,000 samples each and reports 7.38% and 19.44%.

(e), (f) `k50 = evolve(initial_partialrc, next_bit, 49) ^ finalize_counts` has 12,376 values, `erc_50 = k50.expectation` is (25.5, 13, 6.375, 3.125), and the search with $\alpha = 0.075$ stops at 7.9. That evolve is slow, 5 to 10 minutes, because the state Kind grows to 245,176 values. It was run once for this guide and recorded.

(g) The statistics are 1.17, 138.35, 1.90, 25.48, 4.06, 11.43 and 3.95, as posted. Those above 7.9 reject the null hypothesis: the second, fourth and sixth sequences. The rest retain it.

(h) For 64 bits the expected runs counts are (32.5, 16.5, 8.125, 4) and the threshold is again 7.9. The statistic for `opt64s` is 40.7166, so the null is rejected. The 63-step evolve is slower still (the handout says about 8 minutes on the instructor's computer), and the state Kind reaches 817,000 values.

The expected counts without the evolve (this guide's addition). For n fair bits the expected number of runs is $(n + 1)/2$, and the expected number of runs of length exactly m is $(n - m + 3)/2^{m+1}$ when $m < n$. A run of length m in the middle needs m equal bits with a different bit on each side, probability $2^{-(m+1)}$, at $n - m - 1$ places. A run at either end needs one different neighbor, probability 2^{-m} , at 2 places. The evolve confirms the formula for 10 and for 50 bits, and it is how the code block gets by without `k50`.

WATCH OUT Write whole numbers in `ChiSquare(expected)` with a decimal point, as in 13.0 and 4.0 above. A plain integer among the expected values turns one term of the sum into a float, and the statistic then fails with `unsupported operand type(s) for +: 'decimal.Decimal' and 'float'`. The tuple from `k.expectation` or `E(k).raw`, which the posted solution uses, never has this problem.

10 Homework 6

The five problems, in line with the posted solutions. `hw6.py` holds only two templates to fill in, `dicey_wait_system` and `balance_outcome`, so everything here runs without it. Arithmetic by hand, closed forms and remarks that go beyond the posted solutions are this guide's additions, and their numbers are checked by code here.

10.1 Problem 1: kernel and distribution workout

(a) k is `uniform(1, 2, ..., 99)` and $\psi(x)$ adds up those of 3, 7, 11, 13 that divide x . The kernel of k is read off its tree: $k(n) = (1/99) \cdot \{n \in [1 \dots 99]\}$. For $\psi(k)$, count how many of 1, ..., 99 give each value, without building the Kind:

```
psi = 3 * (__ % 3 == 0) + 7 * (__ % 7 == 0) + 11 * (__ % 11 == 0) + 13 * (__ % 13 == 0)
dict(frequencies(psi(n) for n in irange(1, 99)))          # value of psi -> how many n give it
# {(0,): 47, (3,): 24, (7,): 8, (11,): 5, (13,): 4, (10,): 4, (14,): 3, (16,): 2, (18,): 1, (20,): 1}
psi(uniform(1, 2, ..., 99))                             # the check: each weight is its count over 99
# ,---- 0.47475 ---- 0
# |---- 0.24242 ---- 3
# |---- 0.080808 ---- 7
# |---- 0.040404 ---- 10
# |---- 0.050505 ---- 11
# <- -|
# |---- 0.040404 ---- 13
# |---- 0.030303 ---- 14
# |---- 0.020202 ---- 16
# |---- 0.010101 ---- 18
# `---- 0.010101 ---- 20
```

ANSWER The kernel of $\psi(k)$ is $(47/99) \{n = 0\} + (24/99) \{n = 3\} + (8/99) \{n = 7\} + (5/99) \{n = 11\} + (4/99) \{n = 10 \text{ or } n = 13\} + (3/99) \{n = 14\} + (2/99) \{n = 16\} + (1/99) \{n = 18 \text{ or } n = 20\}$.

By hand: there are 33 multiples of 3 up to 99, 14 of 7, 9 of 11 and 7 of 13. Numbers with two of the divisors: 21, 42, 63, 84 give 10; 33, 66, 99 give 14; 39, 78 give 16; 77 gives 18; 91 gives 20. So 3 alone occurs $33 - 4 - 3 - 2 = 24$ times, 7 alone $14 - 4 - 1 - 1 = 8$, 11 alone $9 - 3 - 1 = 5$, 13 alone $7 - 2 - 1 = 4$, and 0 occurs for the other 47.

(b) $D_X(\psi) = (1/16)\psi(-2) + (1/8)\psi(-1) + (3/16)\psi(0) + (1/4)\psi(4) + (1/8)\psi(8) + (1/4)\psi(16)$ says that X has values $-2, -1, 0, 4, 8, 16$ with those weights. For each transformed FRP, name the statistic and plug it in.

```
X = weighted_as(-2, -1, 0, 4, 8, 16, weights=['1/16', '1/8', '3/16', '1/4', '1/8', '1/4'])
D_(X)(__) # E(X)
# 23/4
D_(X)(__ * (__ % 4 == 0)) # E(X {X divisible by 4})
# 6
D_(X)(__ ** 2 % 5) # E(X^2 mod 5)
# 11/8
D_(X)(Exp(-1 * __) * 4 / (4 + __)) # E(e^(-X) 4/(4 + X)); write -1 * __, since -__ fails
# 1.566482421873519
```

Transformed FRP	Statistic ψ	Expectation $E(\psi(X)) = D_X(\psi)$
X	the identity	$-2/16 - 1/8 + 0 + 4/4 + 8/8 + 16/4 = 5.75$
$X \cdot \{X \text{ divisible by } 4\}$	$x \cdot \{x \bmod 4 = 0\}$	$0 + 0 + 0 + 4/4 + 8/8 + 16/4 = 6$
$X^2 \bmod 5$	$x^2 \bmod 5$	$4/16 + 1/8 + 0 + 1/4 + 4/8 + 1/4 = 11/8$
$e^{-X} \cdot 4/(4 + X)$	$\langle t \rangle \mapsto e^{-t} \cdot 4/(4 + t)$	$(1/16)e^2 \cdot 2 + (1/8)e \cdot (4/3) + 3/16 + (1/4)e^{-4} \cdot (1/2) + (1/8)e^{-8} \cdot (1/3) + (1/4)e^{-16} \cdot (1/5) \approx 1.566$

10.2 Problem 2: intuition and evidence

```
heads = Sum(choice(0, 1) ** 10)
heads | (__ >= 9) # (a): told "Yes, at least nine heads"
# ,---- 10/11 ---- 9
# <> -|
# \---- 1/11 ----- 10

def two_headed(m=10_000, n=10): # (b): m coins, one two-headed; we see n heads in n flips
    which_coin = choice(2, 1, m - 1) # 2 means two-headed, 1 means regular
    flips = conditional_kind({
        1: fast_join_pow(Sum, choice(0, 1), n), # the Kind of the number of heads in n flips
        2: constant(n),
    })
    outcome = which_coin >> flips # <which coin, number of heads>
    observed = outcome | (Proj[2] == n) # we saw n heads: treat that as a fact
    return observed[1] # the Kind of the coin (the same as observed ^ Proj[1])

two_headed(n=10)
# 1: 0.90710 2: 0.092897
two_headed(n=13)
# 1: 0.54967 2: 0.45033
two_headed(n=15)
# 1: 0.23380 2: 0.76620
```

ANSWER (a) Given "Yes", ten heads has probability $1/11$, far below $1/2$. If I had said the *first nine* flips were heads, the tenth flip would decide and the answer would be $1/2$. But a single tail can be on any of the 10 flips, so nine heads is 10 times as likely as ten.

(b) For $m = 10,000$ and $n = 10$ the probability that the coin is the two-headed one is 0.092897 . It is 0.45033 at $n = 13$ and 0.7662 at $n = 15$. By hand (a closed form added by this guide): $(1/m) \cdot 1$ against $((m - 1)/m) \cdot 2^{-n}$, which gives $2^n / (2^n + m - 1)$.

The likelihood ratio in favor of the two-headed coin is 2^n , which grows fast, but the regular coins start with a base rate 9999 times higher. The two balance just past $n = 13$ ($2^{13} = 8192$ and $2^{14} = 16384$). Bayes's Rule weighs the likelihood against the base rate.

10.3 Problem 3: the misplaced boarding pass

100 passengers board in order. Passenger 1 picks a seat uniformly at random. Each later passenger takes their own seat if it is free and otherwise picks uniformly among the free seats. Will passenger 100 get seat 100? If passenger 1 takes seat i , passengers 2 to $i - 1$ sit correctly and passenger i is the next to face a choice, among seat 1 and the seats after i . If someone picks seat 1, everyone afterwards sits correctly. So the state only needs (the passenger now choosing, 1 if seat 1 is taken).

```
start = constant(1, 0)

@conditional_kind
def passenger_seating(state):
    passenger, one_filled = state
    if passenger == 100:
        return constant(state) # everyone is seated: freeze
    if one_filled == 1:
        return constant(passenger + 1, one_filled) # seat 1 is taken: this passenger sits correctly
    next_passenger = uniform(1, irange(passenger + 1, 100)) # seat 1, or a later passenger's seat
    return next_passenger ^ IfThenElse(__ == 1, (passenger + 1, 1), Fork(Id, 0))

match100 = evolve(start, passenger_seating, 100)
match100
#      ,---- 1/2 ---- <100, 0>
# <> -|
#     `---- 1/2 ---- <100, 1>
```

ANSWER The final state is $\langle 100, 1 \rangle$ (seat 1 was taken, so seat 100 is free for passenger 100) or $\langle 100, 0 \rangle$, each with probability $1/2$. The last passenger gets the right seat with probability $1/2$.

Intuitively: when passenger 100 boards, one seat is left, and it is seat 1 or seat 100. Every displaced passenger chooses between those two seats on equal terms, so by symmetry each is the last one left with probability $1/2$.

10.4 Problem 4: a dicey wait

Roll d dice, set aside the sixes, roll the rest again, and so on, for at most r rolls. N is the number of rolls used, or $r + 1$ if some dice are still not six. We do not need the individual dice, only how many are left: the state is (rolls so far, dice not yet six).

```
def dicey_wait_system(d, r):
    initial = constant(0, d)
    @conditional_kind(codim=2, target_dim=2)
    def next_state(current_state):
        num_rolls, remaining = current_state
        if remaining == 0:
            return constant(current_state) # all sixes: the state stays fixed
        left_over = fast_join_pow(Sum, choice(0, 1, 5), remaining) # how many are still not six
        return left_over ^ Fork(num_rolls + 1, Id)
    finalize = IfThenElse(Proj[2] == 0, Proj[1], r + 1) # N from the final state
    resolved = (Proj[2] == 0)
    return initial, next_state, finalize, resolved

init, step, finalize, resolved = dicey_wait_system(10, 20)
res20 = evolve(init, step, 20)
E(res20 ^ resolved) # (a) probability all ten dice are six within 20 rolls
# 0.7677410624855379
init, step, finalize, resolved = dicey_wait_system(10, 100)
res100 = evolve(init, step, 100)
E(res100 ^ resolved)
# 0.9999998792532718
n100 = res100 ^ finalize # the Kind of N for d = 10, r = 100: 101 branches
n100.kernel(13) # the weight on one value
# 0.07050140658660858
```

value i	5	7	9	11	13	15	17	19	21	23	25
weight	0.0044759	0.020981	0.045123	0.063838	0.070501	0.066687	0.057056	0.045676	0.034967	0.025967	0.018887

Eleven branches of $\text{kind}(N)$ for $d = 10$, as (a) asks. The posted solution prints all 21 branches from 5 to 25. They are the same for $r = 100$ and $r = 1000$, because the weight on a value $i \leq r$ does not depend on r . Evolving $r = 1000$ takes most of a minute.

(b) Give each die its own color and follow one of them: it is rolled until it first shows a six. The game lasts as long as the slowest die, so N has the same Kind as $M = \text{Max}(R \star d)$, where R is the number of rolls one die needs (or $r + 1$). The maximum is monoidal, so `fast_join_pow` applies:

```
@kind_factory
def roll_until_6(r):
    # one die: rolls up to the first six, or r + 1
    @conditional_kind
    def transition(n, got_six):
        if got_six == 1:
            return constant(n, 1)
        return choice((n + 1, 1), (n + 1, 0), 5)
    return evolve(constant(0, 0), transition, r) ^ IfThenElse(Proj[2] == 0, r + 1, Proj[1])

m100 = fast_join_pow(Max, roll_until_6(100), 10)
Kind.equal(m100, n100)
# True
d = 10
[round((1 - (5/6) ** i) ** d - (1 - (5/6) ** (i - 1)) ** d, 7) for i in (5, 13, 25)]    # (d) the formula
# [0.0044759, 0.0705014, 0.0188874]
[round(n100.kernel(i), 7) for i in (5, 13, 25)]                                     # the evolved system
# [0.0044759, 0.0705014, 0.0188874]
```

ANSWER (a) The probability that the system is resolved after r rolls is 0.7677410624855379 for $r = 20$ and 0.999998792532718 for $r = 100$, and it prints as 1 for $r = 1000$ ($d = 10$). It increases with r because more rolls give more chances to get sixes. By (c) and (d) it equals $(1 - (5/6)^r)^d$.

(c) The maximum is at most j exactly when every component is at most j , and the components are independent with the same Kind, so $E\{M \leq j\} = [E\{R \leq j\}]^d$.

(d) $\{M = i\} = \{M \leq i\} - \{M \leq i - 1\}$, and $E\{R \leq i\} = 1 - E\{R > i\} = 1 - (5/6)^i$, since $R > i$ means the first i rolls all miss. So the kernel is $k_M(i) = (1 - (5/6)^i)^d - (1 - (5/6)^{i-1})^d$ for $i = 1, \dots, r$, and the rest of the weight, $1 - (1 - (5/6)^r)^d$, is on $r + 1$. With no limit on the number of rolls the formula holds for every positive integer i . That is the kernel of $\text{kind}(N_{d,\infty})$.

`hw6.py` has `dicey_wait_system` as a template that returns the plain tuple (`initial`, `next_state`, `r`, `finalize`, `resolved`). The handout and the posted solution show the same five pieces as a `NamedTuple` called `DwSystem`, read as `dw.init`, `dw.transition` and `dw.rolls`. The version above leaves `r` out of what it returns.

The posted `finalize` is `Proj[1]`, the number of rolls made, so the states that are still unresolved show the value r . The version above returns $r + 1$ for them, as the problem defines N . The two differ only on the unresolved weight, 1.2×10^{-7} for $r = 100$, which is why the posted tree for $r = 100$ has 1.4490E-7 on the value 100.

The posted (b) cleans the single-die Kind first, `clean(roll_until_6(r), tolerance=1e-9)`. `clean` drops the branches whose weight is below the tolerance and rescales the rest, which saves time when $r = 1000$. Its `roll_until_6` writes the value for "no six" as 1001, which is $r + 1$ for $r = 1000$.

10.5 Problem 5: optimal weighings

Twelve coins, one of which is heavier or lighter than the rest, and a two-pan balance. C is the identity of the odd coin, a value in $\{-12, \dots, -1, 1, \dots, 12\}$: the absolute value is the coin and the sign says lighter ($-$) or heavier ($+$). Its Kind is uniform, which here stands for our own uncertainty. A weighing (left, right) is a statistic of C with outcome -1 (left pan heavier), 0 (balanced) or 1 (right pan heavier).

```
c = uniform(-12, -11, ..., -1, 1, 2, ..., 12)
c.entropy # (b) lg 24
# Decimal('4.584962500721156181453738938')

@statistic_factory
def balance_outcome(left_coins, right_coins): # (c)
    left = set(kinds.sequence_of_values(left_coins))
    right = set(kinds.sequence_of_values(right_coins))
    @scalar_statistic(codim=1)
    def psi_ell_r(odd_coin):
        if abs(odd_coin) in left:
            return -1 if odd_coin > 0 else 1
        if abs(odd_coin) in right:
            return 1 if odd_coin > 0 else -1
        return 0
    return psi_ell_r

six = balance_outcome([1, 2, ..., 6], [7, 8, ..., 12])
c ^ six # the example weighing: only two outcomes
# -1: 1/2 1: 1/2
(c ^ six).entropy
# Decimal('1.0000000000000000000000000000001')
w1 = balance_outcome((1, 2, 3, 4), (5, 6, 7, 8))
c ^ w1 # four against four: three equally likely outcomes
# -1: 1/3 0: 1/3 1: 1/3
(c ^ w1).entropy
# Decimal('1.584962500721156181453738943')
c | (w1 == -1) # (d) the left pan was heavier: still uniform
# -8: 1/8 -7: 1/8 -6: 1/8 -5: 1/8 1: 1/8 2: 1/8 3: 1/8 4: 1/8
ck = conditional_kind({v: constant(w1(v)) for v in values(c)}) # the outcome given the odd coin
mutual_information(c, ck) # (e) equal to the entropy of the outcome
# Decimal('1.584962500721156181453738943')
w2 = balance_outcome((1, 2, 5), (3, 4, 6)) # (f) the posted second weighing after that outcome
(c | (w1 == -1)) ^ w2
# -1: 3/8 0: 2/8 1: 3/8
```

ANSWER (a) There are 24 possibilities and each weighing has 3 outcomes. Two weighings distinguish at most $3^2 = 9 < 24$ cases, which is not enough. Three give $3^3 = 27 > 24$, so three might be enough.

(b) $H(C) = \lg 24 = 4.585$ bits: the Kind is uniform on 24 values.

(c) The factory above. `kinds.sequence_of_values` turns `[1, 2, ..., 6]` into the list it stands for. (`kinds` is the `frplib` module, already loaded in the playground. The template in `hw6.py` imports the function itself, from `frplib.kinds import sequence_of_values`, and leaves the body of `psi_ell_r` for you to fill in.)

(d) An observation erases branches and leaves the relative weights of the rest unchanged, so starting from a uniform Kind every update is again uniform, on the possibilities that remain.

(e) If m possibilities remain and a weighing splits them into groups of m_{-1} , m_0 , m_1 , then $I(C; \psi(C)) = H(C) - H(C | \psi(C)) = \lg m - \sum (m_i/m) \lg m_i = -\sum (m_i/m) \lg(m_i/m) = H(\psi(C))$. That is the entropy of the balance outcome, which is largest, $\lg 3 = 1.585$, when the three groups are equal.

(f) So at each stage choose the weighing that splits the remaining possibilities into three parts as nearly equal as possible. First weigh four against four (8, 8, 8). Six against six can never balance and gives only 1 bit. The posted strategy is in the table below: whatever the first outcome, its second weighing splits the remaining 8 into 3, 2, 3, and a third weighing settles every case. Three is the fewest weighings that can guarantee an answer, so any strategy that always finishes in three is optimal, and there are many.

First outcome	Second weighing	Its outcome	Third weighing	Third is -1	Third is 0	Third is 1
-1 (left heavier)	1, 2, 5 against 3, 4, 6	-1	1 against 2	1 heavy	6 light	2 heavy

First outcome	Second weighing	Its outcome	Third weighing	Third is -1	Third is 0	Third is 1
		0	8 against 12	cannot occur	7 light	8 light
		1	3 against 4	3 heavy	5 light	4 heavy
0 (balanced)	6, 9 against 10, 11	-1	10 against 11	11 light	9 heavy	10 light
		0	12 against 11	12 heavy	cannot occur	12 light
		1	10 against 11	10 heavy	9 light	11 heavy
1 (right heavier)	1, 2, 5 against 3, 4, 6	-1	3 against 4	4 light	5 heavy	3 light
		0	8 against 12	8 heavy	7 heavy	cannot occur
		1	1 against 2	2 light	6 heavy	1 light

The posted strategy after weighing 1, 2, 3, 4 against 5, 6, 7, 8. Outcome -1 means the left pan is heavier, 0 balanced, 1 the right pan heavier. An entry such as 6 light says that coin 6 is the odd one and is lighter. All 24 cases were run through this table when the file was built.

- After a balanced first weighing, coins 1 to 8 are known to be good, which is why the table can use coin 6 as a reference in 6, 9 against 10, 11.
- The posted solution finds the strategy with a recursive search, `best_weighing_pair` inside `optimal_weighing_strategy`. It calls three helpers that the posted solution does not show and that are not in `hw6.py`: `coins_in_kind`, `weighing_pairs` and `as_conditional_kind`. Its core is the `mutual_information` line above: compute it for every candidate pair and keep the largest.

11 Search words

Words you might type into Ctrl-F that the guide files under a different name. Each entry points to the section that covers it. The list is alphabetical, reading across.

If you search for	Go to	If you search for	Go to
90 minutes, time budget, order of attack, timed plan	4.10	absorbing state, terminal state, frozen	3.6
aces, symmetry argument	1.8 , 7.3	activated, fresh, observed (life of an FRP)	1.1 , 1.2
adapter, observation processor, conditional FRP box	5.5	Alice and Bob meeting, arrival times, Meeting Up	12.4
Alice and Bob's trick, monoid trick, parallel statistic	3.2 , 1.11	allowed materials, exam rules, wifi, offline check	6.4
arbitrage price, risk-neutral price, Market.buy	1.8 , 3.10	Assistant Assistance, secretary problem, After-k	3.17
at least one, complement, not	5.3 , 7.3	automaton, position along a pattern, H T T H T	3.18
balance, twelve coins, odd coin, weighings	10.5	Balls and Boxes, Bayes table by hand	5.4 , 5.5
base rate, false positive, sensitivity, specificity	3.5 , 1.7	battle, strategies, quickening, TeamPowers (homework 4)	8.6
Bayes, prior, posterior, inverse problem	3.5 , 1.7	best choice, optimal strategy, decision, where to stand	3.17
big red button, same Kind same value, valid Kind, which are functions	12.1	binomial, number of successes	5.2
bits, coding, compression, Golomb, run lengths	1.12	boarding pass, airplane seats, last passenger	10.3
by hand, show your work, arithmetic	5.4 , 5.6	canonical form, normalize, compact, unfolded, equivalent	1.1 , 5.4
Ceiling, StDev, guard, ordered_sample: cheatsheet names that fail	6.2 , 2.2	chi-square, ChiSquare, runs test	3.12
circle_points, FromDegrees, pentagon, cube corners	12.2 , 12.3	clean join	1.6
clean(k, tolerance), negligible weights	2.4 , 10.4	clone, fresh FRP	1.3 , 2.6
combination, n choose k, permutation, factorial	5.1	conditional expectation, $E(X C)$	3.10 , 1.8

If you search for	Go to	If you search for	Go to
conditional probability, given	3.3 , 5.3	conserved expectation, fair fight, immortals	8.6
constrain, constraint processor (the older names)	1.2 , 1.7	contingent, depends on the earlier stage	3.4 , 2.5
craps, dice game, Lazy C Casino	3.16 , 5.6 , 12.5 , 12.10	data-driven choice, which Kind produced the data	12.1 , 3.11
deadline, by step n, within n steps	3.6 , 7.2	demo, sample, simulate, Monte Carlo	2.6 , 2.4 , 8.5
dicey wait, dice set aside, maximum of waiting times	10.4	Dirichlet problem, boundary values, lava room	3.9 , 1.11 , 6.3
distribution operator, D_	1.9 , 10.1	doubled cards, tournament, mouse, hunters (lecture 11)	7.4
draw the tree, Kind tree, unfold	5.5	either(a, b, r), the older spelling of choice	2.1
equilibrium, long run, stationary, steady state	1.11	error names: MismatchedDomain, KindError	6.2
event, indicator, Iverson braces	1.3 , 1.13 , 5.4	evolve requires either a Kind and Conditional Kind, ContractError	4.11 , 6.2
exactly at step n versus by step n, arrives exactly	8.3 , 3.6	expected value, mean, average, prediction	1.8 , 3.10
factory, kind_factory, statistic_factory, condition_factory	2.8	fair game, house edge, favors the casino	3.10 , 4.8
Find it First, Carlos and Rhiannon, boxes with prizes	12.9	first-step analysis, hitting probability, hitting time	3.9 , 5.6
forest, random forest, num_trees, force_unkinded	12.7	freeze line forgotten, E equals the number of steps, done state keeps moving	4.11
frequencies, counting the values of a statistic	10.1 , 2.4	fruit pattern, orchard, squirrels (homework 5)	9.3
Funny Dice, non-transitive dice	9.2	gambler's ruin, random walk, Making Bank, after_bets	3.6 , 3.9 , 5.6 , 12.8
geometric, waiting time, first success	5.2 , 3.8	H H versus H T, waiting for a pattern by hand	5.6 , 3.18
help: info, source, explain_error, textbook	6.3	history, path, trajectory	3.6 , 4.4 , 8.3
homework 1, homework 2, homework 3	12.1 to 12.11	hw2.py: triples, alice_bob_arrival, observation_from	12.3 , 12.4 , 12.6
hw3.py: approx_sum_choices, next_roll, flights_from	12.7 , 12.10 , 12.11	hw4.py: five_letters, robot_graph, rgraph_data, Team	8.2 , 8.3 , 8.5 , 8.6
hw5.py: squirrels_count, markov_system, runs3, seq50s	9.3 , 9.4 , 9.5	hypothesis test, null, alternative, reject, retain, alpha	1.10 , 3.12
independent, independence, independent join	1.6 , 3.14 , 5.4	index_of, guard inside a statistic, StatisticError	8.3 , 2.8
inference: probabilistic versus statistical	1.10	inverse of a statistic	4.1
Jensen's inequality, convex	1.8	join, stages, early and later	1.6 , 3.4
Keep pads its result, Keep(cond) ^ Count, pad=None	2.2 , 12.7	kernel, weight of a value	1.9 , 2.4
KL divergence, relative entropy	1.12 , 3.15	likelihood, log-likelihood, MLE, likelihood ratio	1.10 , 3.11 , 9.4
linearity of expectation, additivity, scaling	1.8 , 5.3	MAD, standard deviation, variance, spread, uncertainty	1.8 , 3.10
map, mind map, process diagram, seven steps, how to approach a problem	0	marginal, projection, component	1.3 , 2.2
Markov chain example, two states, stay or switch, whole path with >>	4.12	Markov decryption, cipher, bigram	1.11
Markov likelihood, switches and stays (homework 5)	9.4	Markov property, Markov chain, transition	1.11 , 4.12 , 3.6 , 7.1
mistakes to avoid in a state machine, checks after evolve	4.11	modeling, word problem, where to start, pattern language	7.1
monoidal statistic, fast_join_pow, when it is wrong	3.2 , 1.11	Monty Hall, switch or stay	3.17

If you search for	Go to	If you search for	Go to
multiverse, My_obs, Ten_Rolls, sum3balanced	12.6	mutual information, entropy, information	1.12, 3.15, 10.5
noisy channel, obscured words	8.2, 4.9	observation, update, erase branches	1.7, 3.3
operator precedence, parentheses	2.3, 2.9	path weight, multiply along the path, history[-1], step reads the last value	4.12, 9.4
pattern of flips, run of heads, streak	3.18, 3.8, 4.5	Penney, which pattern comes first	3.18
posted solution slips in homework 1 to 3	12.3, 12.6, 12.7, 12.8	predict the output, what does this print	2.9
procedure, yield, given, return	2.7	race, which happens first	5.6, 3.18
random graph, block graph, simulated null data	8.5	random sum, compound total, N customers	3.7, 4.7
recursion, recurrence, fixed point	3.8	repeat_before_x, codim=(1, infinity)	12.2
robot, path without revisits, Contains	8.3	rules of a Kind: Completeness, Level, Prefix, Positivity	1.1, 5.5
salesman, cities, revenue, additive expectation	12.11	select, choose_balls, bucket of balls	12.7
self-test, true or false	1.14	shortcut, symmetry, long calculation	7.3
show(x), tidy output for a list	2.4, 2.6	silent slips, no error but a wrong answer, test with $p = 3/4$	4.11, 6.1
state, state machine, what should the state be	1.11, 3.6, 7.2	stopping time, time until, duration, clock	3.6, 3.9
Sum of Your Choices, Normal approximation	12.7	swapped tuples in weighted_as, tail tuple first, order of the weights	4.11
symbolic, symbols, substitution, arbitrary	3.13, 2.1, 9.2	table of moves, one row per state, what to remember about the past	4.11, 7.2
tail sum, $E(N)$ as a sum of $P(N > n)$	3.18	threshold, significance level	3.12, 8.5
tolerance, run until resolved, while loop	4.5, 3.6	too big, too many branches, which branches survive	7.3, 4.3
total probability, total expectation	5.3	transform, statistic, feature	1.2, 2.2
triangle from a cut rod, Random Triangle	12.7	two heads in a row, restart patterns, same as the start, openings	4.11, 3.8
two-headed coin, which coin	8.1, 10.2	type I error rate, false rejection, alpha	1.10, 8.5
types, codimension, dimension, $m \rightarrow n$	1.3	unary minus, -__	6.2, 2.2
until problems: flip until, roll until, no fixed number of steps	4.11, 3.6, 3.8	updating with observations, the third of the Big 3	1.2, 1.7, 3.3
urn game, penultimate round (homework 4)	8.4	urn, balls, drawing without replacement, cards	3.4, 2.1, 7.2, 12.7
variant of a practice problem, adapt a template, new problem routine	4.10, 7.1	version, frplib 0.3.10, updating frplib	6.4
weights list shorter than the values	12.7	where do I start, which tool, flowchart, decision map	0, 7.1
WidgetCorp, supplier, resin, quality (homework 5)	9.1	Win on Repeat, repeated ticket, birthday problem	3.17
win statistic over 100 rolls, index_where	12.5	wiring diagram, physical interpretation	5.5, 1.2
with replacement, independent draws	3.1		

12 Homework 1 to 3

The three early homeworks, from the handouts and the posted solutions. The code runs here on the course's own `hw2.py` and `hw3.py`. Where a posted line does not run on `frplib 0.3.10`, or does not do what the handout asks, the entry says so and gives a line that does. Counts by hand, closed forms and cross-checks are this guide's additions, and their numbers are checked by code here. This section was added last and was not tried on `frplib 0.3.2`. The handouts ask for 0.3.3 or later (homework 2, question 5) and 0.3.5 or later (homework 3, for drawing forests).

12.1 Homework 1: FRPs, Kinds, functions

Problem	Posted answer
2. Big Red Button. A fresh FRP shows (10, 40, 20). The display turns off and you push the button again. What does it show?	(G) (10, 40, 20). An FRP produces one value, one time, and shows that value ever after.
3. Two fresh FRPs have the same Kind. Must they show the same value?	No. Each can produce any possible value of the Kind, so they can differ. Only when the Kind has size 1 (a constant) must they agree.
4. A Data-driven Choice. Sixteen FRPs of one Kind showed 4, 1, 3, 3, 2, 4, 3, 2, 2, 1, 2, 2, 3, 2, 2, 4. The Kind has values 1, 2, 3, 4 with weights 2, 4, 6, 2 (left) or 2, 6, 3, 3 (right). Which, and how sure are you?	(B), the right one, but not with high confidence. Left makes 3 the most common value and 4 much rarer. Right makes 2 the most common, with 3 and 4 about equal, and that is the pattern in the data. Sixteen values are few: repeat <code>Market.demo(16, left_kind)</code> and <code>Market.demo(16, right_kind)</code> and some demos of the left Kind look like this too.
5. Valid Kind? The root has children (4) (weight 6) and (1) (weight 4). Under (4) sit (1, 2) and (1, 3). Under (1) sit (4, 6) and (4, 1).	No. Node (1, 2) does not share a prefix with its parent (4) (the Prefix Rule, 1.1). Exchange the subtree at (4) with the subtree at (1) and it is valid.
6. Which are functions? (a) arrows from A to E into 1 to 5, (b) the same with some inputs left out, (c) a table whose input β appears on two rows, (d) a table foo 17, bar 21, zap 21, wow 32, (e) code that returns $x + 1$ when $1 \leq x \leq 10$ and otherwise loops forever.	(a) Yes: every input maps to exactly one output, and several inputs may share an output. (b) No: B and C map to no output. (c) No: β maps to two different outputs. (d) Yes. (e) No: it can fail to terminate.

For problem 4 the posted solution argues from repeated demos. A likelihood (**3.11**) puts a number on the same judgment:

```
import math
data = (4, 1, 3, 3, 2, 4, 3, 2, 2, 1, 2, 2, 3, 2, 2, 4)
left_kind = weighted_as(1, 2, 3, 4, weights=[2, 4, 6, 2])
right_kind = weighted_as(1, 2, 3, 4, weights=[2, 6, 3, 3])
dict(sorted(frequencies(data).items()))      # how often each value came up
# {1: 2, 2: 7, 3: 4, 4: 3}
ell4 = ijoin_log_likelihood(data)             # the data are fixed; it takes a Kind
ell4(left_kind), ell4(right_kind)
# (-21.888082966292956, -20.606020607451093)
math.exp(ell4(right_kind) - ell4(left_kind))  # how many times likelier the data are under right
# 3.6040649414062504
```

By hand the ratio is $(6/4)^7 \cdot (3/6)^4 \cdot (3/2)^3 = 1.5^{10}/16 \approx 3.6$, one factor for each 2, 3 and 4 in the data (the two Kinds agree on the value 1). If both Kinds were equally plausible beforehand, 3.6 to 1 is about a 78% chance for the right Kind: a lean, not a verdict, which is the posted answer.

12.2 Homework 2, problem 1: statistic workout

```
abs_sum_expr = Abs(Sum)                                # (a) as an expression; Sum ^ Abs is the same statistic

@scalar_statistic
def abs_sum(x):                                         # (a) as a custom statistic: x is the whole input tuple
    return abs(sum(x))

abs_sum_expr(3, -9, 2)
# <4>
abs_sum(3, -9, 2)
# <4>
diff41 = Proj[4] - Proj[1]                             # (b) fourth component minus the first
diff41(10, 20, 30, 45, 99)
# <35>
typeof(diff41)                                         # codimension 4 or more, dimension 1
# '[4..) -> 1'
even_minus_odd = Sum(Proj[2::2]) - Sum(Proj[1::2])     # slices a:b:d, counted from 1
even_minus_odd(1, 2, 3, 4, 5, 6, 7)                  # (2 + 4 + 6) - (1 + 3 + 5 + 7)
# <-4>
circle_points = FromDegrees ^ Fork(Cos, Sin)          # (c) degrees to radians, then <cos, sin>
circle_points(180)                                     # the second component is 0 up to rounding
# <-1, 1.224646799147353E-16>
```

(d) A condition factory: a function that returns a condition for the chosen x . The condition is true when, reading from the second component on, the first component comes up again before x does.

```
@condition_factory
def repeat_before_x(x):
    "tests whether the first component repeats before x appears in the input tuple"
    name = f'repeats_before[{x}]'
    doc = f'tests whether the first component repeats before {x} appears in the input'
    @condition(codim=(1, infinity), name=name, description=doc)
    def _repeat_before_x(inp):
        first = inp[0]                                # inp is a tuple: Python indexing, from 0
        for elt in inp[1:]:
            if elt == first:
                return True
            elif elt == x:
                return False
        return False                                  # neither showed up: an arbitrary choice
    return _repeat_before_x

repeat_before_x(7)(4, 2, 6, 7, 8, 9, 4)               # the 7 comes before the second 4
# <0>
repeat_before_x(7)(8, 3, 10, 12, 8, 9, 7)             # the second 8 comes before the 7
# <1>
```

- `Sum` returns a tuple and Python's `sum` returns a number. Built-in statistics are capitalized, and the lowercase Python function of the same name is a different thing.
- `codim=(1, infinity)` lets the condition take a tuple of any positive dimension. `name` and `description` only change what the playground prints for `repeat_before_x(4)`.
- The same loop, with two losing values in place of one x , is the craps statistic win in [12.5](#). The toolkit of built-in statistics is in [2.2](#) and factories are in [2.8](#).

12.3 Homework 2, problem 2: frplib workout

triples comes from hw2.py: every 3-tuple that can be made from the given numbers.

```
from hw2 import *                                # triples, alice_bob_arrival, observation_from, ...
V = frp(uniform(triples(-1, 1)))                 # (a) a random corner of the cube: 8 values
V2, V3 = clone(V), clone(V)                     # fresh, independent FRPs with the same Kind
size(kind(V)), dim(V)
# (8, 3)
pentagon = weighted_as([circle_points(90 - 72 * k) for k in range(5)], weights=[5, 4, 6, 7, 9])
pentagon                                         # (b)
#      ,---- 9/31 ---- <-0.95106, 0.30902>
#      |---- 7/31 ---- <-0.58779,-0.80902>
# <> -+---- 5/31 ---- <6.1232E-17, 1>
#      |---- 6/31 ---- <0.58779,-0.80902>
#      `---- 4/31 ---- <0.95106, 0.30902>
sq_distance = scalar_fn(1 + SumSq)              # (c) <x, y, z> to the number 1 + x^2 + y^2 + z^2
p = weighted_by(triples(-1, 0, 1), weight_by=sq_distance)
P = frp(p)
size(p), p.kernel((1, 1, 1)), p.kernel((1, 1, 0))
# (27, 0.04938271604938271, 0.037037037037037035)
```

- **(b)** The vertices sit on the unit circle 72° apart, starting from $(0, 1)$ at 90° . Clockwise means the angle goes down: $90^\circ, 18^\circ, -54^\circ, -126^\circ, -198^\circ$, with weights 5, 4, 6, 7, 9 in that order. The top vertex prints with $6.1232\text{E-}17$ for its first coordinate, which is 0 up to rounding.
- **(c)** The 27 points are 8 corners (weight $1 + 3 = 4$), 12 edge midpoints (3), 6 face centers (2) and the center (1). The weights add to $32 + 36 + 12 + 1 = 81$, so the canonical weights are $4/81 \approx 0.049383$, $3/81 \approx 0.037037$, $2/81 \approx 0.024691$ and $1/81 \approx 0.012346$, as in the posted Kind. `scalar_fn` is needed because a statistic returns a tuple and `weight_by` wants a number.

WATCH OUT The posted line for (b) is `weighted_as([circle_points(Pi/2 + 2*Pi*k/5) for k in range(5)], weights=[5, 4, 6, 7, 9])`. With `circle_points` as defined in problem 1 it does not build the pentagon. Its angles are in radians, but `circle_points` converts from degrees, so all five points land next to $(1, 0)$ (the first is $(0.99962, 0.027412)$). The angle also goes up, which is counterclockwise, where the handout asks for clockwise. Give the angles in degrees and step down by 72, as above.

12.4 Homework 2, problem 3: Meeting Up

Alice and Bob each arrive between 9:00 and 9:30 and wait five minutes. They meet when their arrival times are within five minutes of each other. `alice_bob_arrival` in hw2.py gives the Kind of the arrival times for each part, and the event that they meet is a condition applied to it.

```
meet = Abs(Proj[1] - Proj[2]) <= 5              # (a) values <a, b>, minutes after 9:00
meet(alice_bob_arrival('a'))
# 0: 0.67638   1: 0.32362
meet_b = Abs(Proj[1] - Proj[2] - Proj[3]) <= 5  # (b) values <a, b, d>: Bob really arrives at b + d
meet_b(alice_bob_arrival('b'))
# 0: 0.57434   1: 0.42566
meet_c = And(Abs(Proj[1] - Proj[2]) <= 5, Proj[3] == 1, Proj[4] == 1)  # (c) <a, b, u, v>: both get in
meet_c(alice_bob_arrival('c'))
# 0: 0.90591   1: 0.094087
strict_b = And(meet_b, Proj[2] + Proj[3] <= 30)(alice_bob_arrival('b')) # nobody waits after 9:30
strict_b
# 0: 0.57530   1: 0.42470
```

- **(a)** T has dimension 2 and $31 \times 31 = 961$ equally weighted values. The pairs with $|a - b| \leq 5$ number $31 + 2 \cdot (30 + 29 + 28 + 27 + 26) = 311$, so the weight on 1 is $311/961 \approx 0.32362$. `environment.set_max_denom(999)` makes the playground print it as $311/961$ (**2.4**).
- `meet(T)` is the event, an FRP whose values are 0 and 1. `kind(meet(T))` and `meet(kind(T))` are the same Kind, and the weight on 1 adds up the weights of the values for which the condition is true.
- **(b)** Posted answer 0.42566. The posted condition lets Bob arrive after 9:30, since $b + d$ can reach 40. The handout also says nobody waits after 9:30, and the last line above adds that rule: 0.42470. The difference is small, and the posted answer is the first one.

- (c) Posted answer 0.094087. And takes any number of conditions.

12.5 Homework 2, problem 4: craps at the Lazy C, part 1

Every player rolls two dice 100 times and a statistic reads the rolls in order. A first roll of 7 or 11 wins and 2, 3 or 12 loses. Otherwise the player wins if the first roll comes up again before a 7 or 11, loses if a 7 or 11 comes first, and draws if neither happens in the 100 rolls.

```
d6 = uniform(1, 2, ..., 6)
rolls = d6 * d6
Roll = frp(Sum(rolls))           # (a) one roll: the sum of two dice
Play = Roll ** 100              # (b) 100 independent rolls

@scalar_statistic
def win(first_roll, *remaining_rolls):           # (d) the * collects the other 99 rolls in a tuple
    "Returns 1, 0, -1 for a win/draw/loss at craps at Lazy C Casino."
    if first_roll in {7, 11}:
        return 1
    elif first_roll in {2, 3, 12}:
        return -1
    for roll in remaining_rolls:
        if roll == first_roll:
            return 1
        elif roll == 7 or roll == 11:
            return -1
    return 0

win(4, 2, 6, 7, 8, 9, 4, 10)           # the handout's losing example
# <-1>
win(8, 3, 10, 12, 8, 9, 11, 2)         # and its winning example
# <1>
Win = win(Play)                       # (e) the FRP for the outcome of one game
```

- (c) Each of the 100 components has 11 possible values, so the Kind of `Play` has size 11^{100} . Do not ask for it.
- (f) $E(\text{Win})$ is the risk-neutral price. The Kind is out of reach, so `frplib` returns a numerical approximation: about -0.1084 in the posted run, and -0.10215 there with `tolerance=0.005`. One run here took 22 seconds and gave -0.114 . The price is negative, so the bet favors the casino: you lose about 10 cents per dollar on average.
- The second posted version of `win`, the one with `index_of` and `index_where`, has `r == 1` in its predicate where 11 is meant. The loop version above is the safe one to copy.

The exact price takes the state method of homework 3 (12.10, 3.16): carry (first roll, rolls so far, outcome) through the other 99 rolls. This check is the guide's addition.

```
from hw3 import next_roll           # the conditional Kind given in homework 3
win1 = Cases({2: -1, 3: -1, 12: -1, 7: 1, 11: 1}, default=0)
craps_initial_state = Sum(d6 * d6) ^ Fork(__, 1, win1)           # <first roll, 1, outcome so far>
lazy_c = evolve(craps_initial_state, next_roll, 99)             # the state after all 100 rolls
as_float(E(lazy_c ^ Proj[3]))           # the exact price of the 100-roll game
# -0.10282310282310281
as_float(E(lazy_c ^ (Proj[3] == 0)))     # the weight on a draw
# 3.582650599820115e-17
```

12.6 Homework 2, problem 5: the likelihood principle

The kernel and the likelihood are the same number read two ways (**1.10, 3.11**). In the kernel $k(x)$ the Kind is fixed and the value varies. In the likelihood $L_x(k) = k(x)$ the observed value is fixed and the Kind varies over the candidates. The maximum likelihood estimate is the candidate with the largest likelihood, or the largest log-likelihood, which picks the same one.

```

candidates = [weighted_by(triples(1, 2, 3), weight_by=scalar_fn(w)) for w in (Sum, Max, Min, SumSq)]
my_earth = multiverse() # hw2.py: call once per session
My_obs = observation_from(candidates, universe=my_earth) # an FRP of one candidate, picked at random
my_likelihoos = [k.kernel(My_obs.value) for k in candidates] # (a) L(k) = k(x) for each k
best = arg_max(*my_likelihoos) # index of the largest, from 0 (arg_max is in hw2.py)
balanced = uniform(1, 2, ..., 6)
skewed = linear(1, 2, ..., 6, increment=0.25) # weights 1, 1.25, ..., 2.25, that is 4/39, ..., 9/39
sum3balanced = Sum(balanced ** 3) # (b) the sum of three rolls
sum3skewed = Sum(skewed ** 3)
One_Roll = observation_from([sum3balanced, sum3skewed], universe=my_earth)
Ten_Rolls = One_Roll ** 10 # dimension 10; its Kind is too large to ask for
ell = ijoin_log_likelihoos(Ten_Rolls.value) # a function: give it a Kind, get the log-likelihoos
ijoin_log_likelihoos(Ten_Rolls.value, as_float=False) # the first posted form
# TypeError: ijoin_log_likelihoos() got an unexpected keyword argument 'as_float'
ten = (10, 5, 15, 9, 12, 16, 13, 15, 11, 17) # what Ten_Rolls showed in one run here
ell10 = ijoin_log_likelihoos(ten)
ell10(sum3balanced), ell10(sum3skewed)
# (-28.391534844260857, -26.72219324504231)

```

- For those ten sums the estimate is `sum3skewed`: its log-likelihood is larger by about 1.67, so the data are $e^{1.67} \approx 5.3$ times likelier under it. Your FRP will show other values, and `ell(sum3balanced)` and `ell(sum3skewed)` decide in the same way.
- Log-likelihoods are negative because a likelihood is a canonical weight, a number between 0 and 1, and the log of such a number is below 0.
- More rolls separate the two Kinds better, because the proportions of the values vary less from sample to sample. Fewer rolls separate them worse.
- (c)**, in plain words: you hold several models of the world. A model under which what you saw would be very surprising deserves less trust. Maximum likelihood picks the model that makes the observation least surprising. It is a rule of thumb that tends to work, not a guarantee.
- The posted one-liner for (a) is `tup(k.kernel(My_obs.value, as_float=False) for k in candidates) ^ Fork(ArgMax, Id)`: the index of the best candidate, then all the likelihoods.

WATCH OUT On `frplib 0.3.10` the first posted form, `ijoin_log_likelihoos(Ten_Rolls.value, as_float=False)`, stops with the error shown above. Leave the keyword out. The value of an FRP can be passed as it is, or as `as_float(Ten_Rolls.value)`, which is the second posted form.

12.7 Homework 3, problem 1: joins workout

(a) Statistics of independent FRPs are independent.

```

X = frp(weighted_as(1, 2, ..., 8, weights=[5, 4, 8, 2, 1])) # eight values, five weights: see below
Y = frp(uniform((99, 100), (199, 200), (999, 1000)))
Kind.equal(kind(X * Y), kind(X) * kind(Y)) # the Kind of the mixture is the mixture of the Kinds
# True
psi = IfThenElse(__ > 4, -10, 10)
phi = Sum
Kind.equal(kind(psi(X) * phi(Y)), kind(psi(X)) * kind(phi(Y)))
# True

```

- Why this holds for any compatible ψ and ϕ : the value of $\psi(X)$ is computed from X alone, so it carries no more information than X does. X tells us nothing about Y , and so nothing about $\phi(Y)$, which is computed from Y alone. The general test is in **3.14**.
- When `weights` is shorter than the list of values, `frplib` repeats the last weight: X has weights 5, 4, 8, 2, 1, 1, 1, 1. The forest code in (e) relies on this with `weights=[a, 1]`.

(b) Random Triangle. Cut a rod of length 1 at two independent points. The pieces have lengths $\min$, $\max - \min$ and $1 - \max$, and they form a triangle exactly when each is shorter than $1/2$. A piece of exactly $1/2$ gives a flat triangle, not counted here.

```
triangle = And(Min < 1 / 2, Max - Min < 1 / 2, Max > 1 / 2) # 1 - Max < 1/2 is Max > 1/2
cuts = uniform(0.01, 0.02, ..., 0.99) ** 2
triangle(cuts)
# 0: 0.76002 1: 0.23998
E(triangle(cuts)) # the probability of a triangle
# 0.2399755127027854
Cut1 = frp(uniform(0.02, 0.03, ..., 0.49)) # the first cut, in the left half

@conditional_frp(codim=1, auto_clone=True)
def next_cut(cut1): # the second: past the middle, under 1/2 beyond the first
    if 0.01 < cut1 < 0.5:
        hi = float(cut1) + 0.49
        return uniform(0.51, 0.52, ..., hi)
    raise ValueError('assume 0.01 < cut < 0.5')

kind(triangle(Cut1 >> next_cut)) # now the pieces always form a triangle
# 1: 1
```

The posted PDF repeats the `Cut1` line with 0.01 as its first value. `next_cut` rejects 0.01, so keep 0.02, which is also what the solution file uses.

(c) The Sum of Your Choices. `binary() ** 128` has 2^{128} values, each with canonical weight 2^{-128} : far too many to build. The sum of all 128 choices is the sum of the first 64 plus the sum of the last 64, and the two halves are independent with the same Kind, so `Sum(binary() ** 128)` equals `Sum(Sum(binary() ** 64) * Sum(binary() ** 64))`. Halving again and again is what `fast_join_pow` does (**3.2**).

```
from hw3 import approx_sum_choices # a Normal approximation on 0, 1, ..., n
k0 = fast_join_pow(Sum, binary(), 128)
k1 = approx_sum_choices(128)
size(k0), k0.kernel(64), k1.kernel(64)
# (129, 0.07038609217001514, 0.0704319777223871)
max(abs(k0.kernel(val) - k1.kernel(val)) for val, _wgt in k0)
# 5.9439055779479666e-05
```

`k0` is symmetric around 64 and its weights fall off quickly on both sides. The largest gap between the two kernels, about $5.94e-05$, is at the middle, where the weights are about 0.07, so the approximation is good at every value.

(d) Sampling without Replacement. State (red, green, blue) left in the bucket. Number the balls 1 to n , pick a number uniformly, and take one off the color it falls in.

```
@conditional_kind
def select(state):
    r, g, b = state
    n = r + g + b
    if n == 0: # no balls left: the state stays put
        return constant(state)
    chosen = IfThenElse(__ <= r,
                        tup(-1, 0, 0),
                        IfThenElse(__ <= r + g, tup(0, -1, 0), tup(0, 0, -1)))
    return uniform(1, 2, ..., n) ^ (chosen + state)

@kind_factory
def choose_balls(n: int, r: int, g: int, b: int):
    initial = constant(r, g, b)
    return evolve(initial, select, n)
```

```

choose_balls(3, 4, 4, 4)
#      ,---- 0.018182 ---- <1, 4, 4>
#      |---- 0.10909  ----- <2, 3, 4>
#      |---- 0.10909  ----- <2, 4, 3>
#      |---- 0.10909  ----- <3, 2, 4>
#      |---- 0.29091  ----- <3, 3, 3>
# <> -|
#      |---- 0.10909  ----- <3, 4, 2>
#      |---- 0.018182 ---- <4, 1, 4>
#      |---- 0.10909  ----- <4, 2, 3>
#      |---- 0.10909  ----- <4, 3, 2>
#      `---- 0.018182 ---- <4, 4, 1>

```

By counting, with 220 ways to choose 3 balls of 12: three of one color in 4 ways ($4/220 \approx 0.018182$), two of one color and one of another in $6 \cdot 4 = 24$ ways ($24/220 \approx 0.10909$), one of each in $4 \cdot 4 \cdot 4 = 64$ ways ($64/220 \approx 0.29091$). More on draws without replacement in [3.4](#).

(e) Random Forests. A forest on nodes 1, 2, ... is the tuple of each node's parent, with 0 for a root. Node $m + 1$ picks its parent from 0, 1, ..., m , where 0 (start a new tree) has weight a and every existing node has weight 1.

```

@frp_factory
def forest(n, a=1):
    initial = force_unkinded(frp(constant(0))) # one node, one tree; unkinded, as the handout advises
    @conditional_frp(auto_clone=True)
    def next_node(cur_forest):
        m = len(cur_forest)
        parent = frp(weighted_as(0, 1, ..., m, weights=[a, 1]))
        return parent ^ Prepend(cur_forest) # the forest so far, then the new node's parent
    return evolve(initial, next_node, n)

num_trees = ForEach(__ == 0) ^ Sum # one tree for every node whose parent is 0
num_trees(0, 0, 1, 1, 1, 0, 2, 7) # eight nodes; the roots are nodes 1, 2 and 6
# <3>
dim(forest(11)) # n steps from one node: n + 1 nodes
# 12

@kind_factory
def forest_size(n, a=1):
    initial = constant(1, 1) # <number of nodes, number of trees>
    @conditional_kind
    def next_nums(nnodes, ntrees):
        return weighted_as((nnodes + 1, ntrees), (nnodes + 1, ntrees + 1),
                           weights=[nnodes / (a + nnodes), a / (a + nnodes)])
    return evolve(initial, next_nums, n)

forest_size(12) ^ Proj[2] # the posted Kind of the number of trees
#      ,---- 0.076923 ----- 1
#      |---- 0.23871  ----- 2
#      |---- 0.31019  ----- 3
#      |---- 0.22708  ----- 4
#      |---- 0.10554  ----- 5
#      |---- 0.033093 ----- 6
# <> -+---- 0.0072250 ----- 7
#      |---- 0.0011124 ----- 8
#      |---- 0.00012036 ----- 9
#      |---- 0.0000089561 ---- 10
#      |---- 4.3632E-7  ----- 11
#      |---- 1.2526E-8  ----- 12
#      `---- 1.6059E-10 ----- 13

```

WATCH OUT Two things in the posted solution to (e). First, its statistic is `num_trees = Keep(__ == 0) ^ Count`. On `frplib` 0.3.10 `Keep` pads its result back to the dimension of the input, so that statistic returns the number of nodes (8 for the forest above), not the number of trees. `ForEach(__ == 0) ^ Sum` counts the zeros, and so does `Keep(__ == 0, pad=None) ^ Count (2.2)`. Second, the posted `forest(n)` and `forest_size(n)` take n steps from a forest of one node, so they describe $n + 1$ nodes: the Kind above, which is the posted one, is for 13 nodes. For a forest of exactly 12 nodes, as the handout words it, take 11 steps: `forest_size(11) ^ Proj[2]` puts 0.083333, 0.25166 and 0.31507 on 1, 2 and 3 trees.

- Without `force_unkinded`, as the posted code has it, `forest(7)` took 12 seconds here, because `frplib` works out the Kind of every stage. With it the factory answers at once.
- The number of trees only needs (nodes, trees), not the forest, because whether node $m + 1$ starts a new tree depends only on m : it does with probability $a/(a + m)$. That is why `forest_size` is small where the Kind of `forest(n)` is not.
- With $a = 1$ the expected number of trees in a forest of N nodes is $1 + 1/2 + \dots + 1/N$, one term for each node (expectation is additive, **3.10**). For 13 nodes that is about 3.18. The weight on 13 trees is $1/13!$, about $1.6059e-10$: every node has to start a tree.

12.8 Homework 3, problem 2: Making Bank

A gambler with w dollars needs $w + g$. She bets a dollar at a time, each bet paying `choice(-1, 1, u)`, until her net wealth reaches $-w$ (ruin) or g (the goal). The state is her net wealth, and the two ends are frozen. This is the gambler's ruin of **3.6** and **3.9**.

```
@kind_factory
def after_bets(n, w, g, u=1, initial=constant(0)):
    "Kind of the gambler's net wealth after n bets."
    bet = choice(-1, 1, u) # weight u on +1 for each 1 on -1
    @conditional_kind(codim=1)
    def next_bet(net_wealth):
        if net_wealth in (-w, g): # (a) ruined, or at the goal: no more bets
            return constant(net_wealth)
        return bet ^ (__ + net_wealth)
    return evolve(initial, next_bet, n) # initial is an argument so that a run can be continued

@condition_factory
def is_resolved(w, g): # (c) a factory, because the condition depends on w and g
    "tests whether the gambler has finished betting"
    return ElementOf(-w, g)

k100 = after_bets(100, 10, 30, 1) # (b) the Kind the handout asks for
size(k100), k100.kernel(-10), k100.kernel(30) # number of values, weight on ruin, weight on the goal
# (21, 0.3197273206999828, 0.00265341374299573)
as_float(E(k100 ^ is_resolved(10, 30))) # the chance that she has finished by bet 100
# 0.32238073444297854
```

(c) The handout wants the smallest n with $E(\text{after_bets}(n, w, g, u) \wedge \text{is_resolved}(w, g))$ above $1 - 10^{-5}$. The plain way is one bet at a time, continuing from the Kind reached so far. The posted `find_n_resolved` is the same search made faster: it jumps ahead 200 bets at a time and then narrows down.

```
def first_n_resolved(w, g, u=1, tolerance=1e-5):
    n, k = 0, constant(0)
    unresolved = Not(is_resolved(w, g))
    while E(unresolved(k)) >= tolerance:
        k = after_bets(1, w, g, u, k) # one more bet, starting from the current Kind
        n += 1
    return (n, k)

n5, k5 = first_n_resolved(5, 5)
n5
# 235
clean(k5, 1e-5) # clean drops the branches lighter than the tolerance
# -5: 1/2 5: 1/2
as_float(E(after_bets(3808, 20, 20) ^ is_resolved(20, 20))) # the posted n for w = g = 20, u = 1
# 0.9999900448291873
```

ANSWER Posted answer for $w = 20$, $g = 20$, $u = 1$: $n = 3808$, and `clean(after_bets(3808, 20, 20), 1e-5)` has weight $1/2$ on -20 and $1/2$ on 20 . Confirmed here: the resolved weight first passes $1 - 10^{-5}$ at 3808 bets and is still below it at 3807.

WATCH OUT The posted `is_unresolved` returns `Not(is_resolved(-w, g))`. That flips the sign of w twice, so it tests for w and g in place of $-w$ and g , and a ruined gambler counts as still playing. Her weight never drops under the tolerance, so the posted `find_n_resolved(20, 20)` never returns. Write `Not(is_resolved(w, g))`, as above. With that one change the posted function returns $n = 3808$ in a few seconds, so the posted answer is right.

- The handout's table for (b) is not in the posted solutions. `after_bets(100, 10, 30, 1)` has 21 values, the even numbers from -10 to 30 , with 0.31973 on ruin and 0.0026534 on the goal after 100 bets.
- With $u = 1$ every bet is fair, so the expected net wealth stays 0 at every stage, and in the end the goal is reached with probability $w/(w + g)$, after $w \cdot g$ bets on average (**3.9**, which also has the formula for $u \neq 1$).

12.9 Homework 3, problem 3: Find it First!

Fifteen boxes in three rows of five, numbered along the rows. Two different boxes hold a prize. Carlos opens boxes down the columns (1, 6, 11, 2, 7, 12, ...) and Rhiannon along the rows (1, 2, 3, ...), one box each per round. Whoever opens a prize box in an earlier round wins, and the same round is a tie.

```
prizes = without_replacement(2, 1, 2, ..., 15) # 105 equally likely pairs of boxes
carlos = [1, 6, 11, 2, 7, 12, 3, 8, 13, 4, 9, 14, 5, 10, 15] # down the columns
rhiannon = list(irange(1, 15)) # along the rows

@scalar_statistic(codim=2)
def winner(a, b):
    "1 if Carlos finds a prize first, -1 if Rhiannon does, 0 for a tie"
    c = min(carlos.index(a), carlos.index(b)) # the round (from 0) in which Carlos opens a prize box
    r = min(rhiannon.index(a), rhiannon.index(b))
    return (c < r) - (r < c)

winner(prizes) # (b) the Kind of Z
# -1: 0.40952  0: 0.21905  1: 0.37143
```

- **(a)** Posted answer (B): Rhiannon is more likely to win. A player gains by getting the only look at a fresh box while the other opens one that has been opened before. Through every round Rhiannon is never behind, and often ahead, in fresh boxes seen first.
- **(b)** The weights are $43/105$, $23/105$ and $39/105$, the posted Kind. The posted solution says its code (a Kind `prizes` and a statistic `winner`) is in `hw3_sol.py`, but the uploaded copy of that file does not have it, so the statistic above is this guide's.
- **(c)** `FRP.sample(10_000, frp(winner(prizes)))` or `Market.demo` tabulates many plays. The posted table had 41.89%, 21.49% and 36.62%, close to the Kind.

12.10 Homework 3, problem 4: craps, part 2

The same game, one roll at a time, with no limit of 100 rolls. The state is $\{f, n, w\}$: the first roll, the number of rolls so far, and the outcome so far (-1 lost, 1 won, 0 still going). **3.16** works the game with a conditional Kind of its own. This entry runs the posted solution on `next_roll` from `hw3.py`.

```
def step(current_kind): # (d)
    return current_kind >> next_roll ^ Proj[4, 5, 6]

def run_craps(init_kind, tolerance=1e-7): # (e) roll until a draw is all but impossible
    r = 0
    state = init_kind
    while True:
        state = step(state)
        r += 1
        draw_prob = E(state ^ (Proj[3] == 0))
        if draw_prob < tolerance:
            break
    return (state, r)

full_game, r = run_craps(craps_initial_state, tolerance=1e-7)
```

```

full_outcome = clean(full_game[3], tolerance=1e-7)
r
# 41
full_outcome
# -1: 0.55141 1: 0.44859
E(full_outcome) # the price of a $1 bet
# -0.1028230805158485

```

- **(a)** From (f, n, w) with $w = 1$ or -1 the next state is the same state: the game is over. From $(f, n, 0)$ it is $(f, n + 1, -1)$ if the next roll is 7 or 11, $(f, n + 1, 1)$ if it equals f , and $(f, n + 1, 0)$ otherwise.
- **(b)** The initial state is `craps_initial_state` from **12.5**: `Sum(d6 * d6) ^ Fork(__, 1, win1)`. `Fork` turns the number 1 into the statistic that always returns 1.
- **(c)** In `next_roll` the three values `vs` are the three possible next states and `ws` counts, out of the 36 equally likely rolls of two dice, how many lead to each: 8 give a 7 or an 11 (lose), `win = min(first_roll - 1, 13 - first_roll)` give the first roll again (1 way for 2 or 12, 2 ways for 3 or 11, 3 for 4 or 10, and so on), and the other $28 - \text{win}$ leave the game going. It returns a `Kind` because the next state is random.
- **(d)** `current_kind >> next_roll` is the `Kind` of (current state, next state), six components. `Proj[4, 5, 6]` keeps the next state. `Proj[4:]` does the same.
- **(e)** $r = 41$, the outcome has 0.551412 on a loss and 0.448588 on a win, and the price is about -0.102823 : the bet favors the casino.

12.11 Homework 3, problem 5: Wandering Salesman

He starts in one of six cities, equally likely. Each day he earns `revenue[city]` where he is, then flies to a city chosen uniformly from `flights_from[city]`. After 100 days he flies home from the 101st city without collecting there. The state is (city, revenue so far).

```

from hw3 import flights_from, revenue
starting_state = uniform(1, 2, ..., 6) * constant(0)

@conditional_kind
def travel(state):
    city, earned = state
    return uniform(flights_from[city]) ^ Fork(Id, earned + revenue[city]) # fly on, bank the revenue

city_rev = evolve(starting_state, travel, 100) # 10 to 20 seconds: 4031 states by the end
E(Proj[2](city_rev)) # (a) expected net revenue
# 0.1810841983852364
Proj[1](city_rev) # (b) the city he flies home from
# 1: 0.11765 2: 0.19608 3: 0.23529 4: 0.17647 5: 0.27451

```

Part (a) does not need the revenue in the state. Expectation is additive, so the expected total is the sum over the 100 days of the expected revenue of that day's city, and the city alone is a state with six values. This shortcut is the guide's addition. It runs at once and gives the same number:

```

@conditional_kind(codim=1)
def next_city(city):
    return uniform(flights_from[city])

day, total = uniform(1, 2, ..., 6), 0 # the Kind of today's city, and the running expectation
for _ in range(100):
    total = total + E(day ^ Cases(revenue)) # expected revenue in today's city
    day = next_city // day # the Kind of tomorrow's city

as_float(total)
# 0.18108419838523646

```

- The revenue is added when he flies on, so on reaching the 101st city all 100 days are in the total and that city's own revenue is not.
- City 6 is missing from (b) because no flight goes to city 6. The weights are 6/51, 10/51, 12/51, 9/51 and 14/51. By day 100 the `Kind` of the city no longer changes from one day to the next.